THE UNIVERSITY OF QUEENSLAND

AUSTRALIA

A Study of Multi-Constraint Shortest Path Queries in Road Network

Ziyi Liu

Master of Computer Science

0000-0002-2468-3104

A thesis submitted for the degree of Doctor of Philosophy at

The University of Queensland in 2023

Information Technology & Electrical Engineering

Abstract

The shortest path problem is fundamental in route planning. It focuses on optimising one weight w, such as minimum distance, travel time of driving, or fuel consumption, in a weighted graph $G = (V, E)$. Specifically, a path p is the shortest path between two vertices s and t if its weight $w(p)$ is minimal compared to all other paths from s to t. In real-life route planning, shortest-path queries are not sufficient to satisfy multiple user requirements. Hence, this multi-criteria route planning is studied and applied in more practical cases. Looking for an optimal path based on multi-criteria information is an important research problem and is defined as the *Multi-Constraint Shortest Path (MCSP)* problem. Typically, the optimal result returned by an *MCSP* query is the shortest path on one criterion, whilst other criteria satisfy some pre-defined constraints. However, the previous solutions to *MCSP* are far from efficient. Motivated by the limitations of existing works. We study in-depth the different types of query in the field of *MCSP*.

We first study the *Constrained Shortest Path (CSP)* problem, which aims to find the shortest path between two nodes in a road network subject to a given constraint on another attribute. It is typically processed as a skyline path problem on the two attributes, resulting in a very high computational cost, which can be prohibitive for large road networks. The main bottleneck is to deal with a large number of partial skyline paths, which further makes the existing index-based methods incapable of obtaining the complete exact skyline paths. In this thesis, we propose a novel skyline path concatenation approach to avoid the expensive skyline path search, which is then used to efficiently construct a 2-hop labelling index *FHL* for the *CSP* queries.

Then, we dig into the more complex problem *MCSP*, which aims to find the shortest path between two nodes in a network subject to a fixed number of constraints. Consider that the number of intermediate skyline paths becomes larger as the network size increases and the constraint number grows, causing the dramatic growth of computational cost and further making the existing index-based methods hardly capable of obtaining complete exact results. We propose a novel high-dimensional skyline path concatenation method to avoid the expensive skyline path search, which supports the efficient construction of hop labeling index for *MCSP* queries.

Subsequently, we work on the flexibility of *MCSP* queries. Our aim is to answer the *MCSP* queries with any combination of constraints. To achieve it, we propose a skyline-cube-based *FHL* index that can handle flexible *MCSP* efficiently. Firstly, we analyse the relation between low and high-dimensional skyline paths theoretically and use a cube to organise them hierarchically. After that, we propose methods to derive the high-dimensional path from the lower ones, which can adapt to the flexible scenario naturally and reduce the expensive high dimensional path concatenation. Then we introduce efficient methods for both single and multi-hop cube concatenations and propose pruning methods to further alleviate the computation. Finally, we improve the *FHL* structure with a lower

height to speed up construction and query.

Our last work focuses on the approximate *CSP*. Consider that the *FHL* index to answer *CSP* queries will take a long time in index construction. To reduce the construction time and size of the *FHL* index, we propose different approximate strategies applied in each phase of *FHL*.

We conduct extensive experiments on real-life road networks to test our methods on each type of *CSP* queries. The results demonstrate the superiority of our methods compared to state-of-the-art solutions.

Declaration by author

(All candidates to reproduce this section in their thesis verbatim)

This thesis is composed of my original work, and contains no material previously published or written by another person except where due reference has been made in the text. I have clearly stated the contribution by others to jointly-authored works that I have included in my thesis.

I have clearly stated the contribution of others to my thesis as a whole, including statistical assistance, survey design, data analysis, significant technical procedures, professional editorial advice, financial support and any other original research work used or reported in my thesis. The content of my thesis is the result of work I have carried out since the commencement of my higher degree by research candidature and does not include a substantial part of work that has been submitted to qualify for the award of any other degree or diploma in any university or other tertiary institution. I have clearly stated which parts of my thesis, if any, have been submitted to qualify for another award.

I acknowledge that an electronic copy of my thesis must be lodged with the University Library and, subject to the policy and procedures of The University of Queensland, the thesis be made available for research and study in accordance with the Copyright Act 1968 unless a period of embargo has been approved by the Dean of the Graduate School.

I acknowledge that copyright of all material contained in my thesis resides with the copyright holder(s) of that material. Where appropriate I have obtained copyright permission from the copyright holder to reproduce material in this thesis and have sought permission from co-authors for any jointly authored works included in the thesis.

Publications included in this thesis

1. **Ziyi Liu**, Lei Li, Mengxuan Zhang, Wen Hua, Pingfu Chao, and Xiaofang Zhou, Efficient Constrained Shortest Path Query Answering with Forest Hop Labeling, *IEEE 37th International Conference on Data Engineering (ICDE)*, pp. 1763-1774, 2021
2. Xuanyi Zhang, **Ziyi Liu**, Mengxuan Zhang and Lei Li, An Experimental Study on Exact Multi-constraint Shortest Path Finding, *Australasian Database Conference (ADC)*, Springer, pp. 166–179, 2021
3. **Ziyi Liu**, Lei Li, Mengxuan Zhang, Wen Hua, and Xiaofang Zhou, Multi-constraint shortest path using forest hop labeling, *The VLDB Journal (2022)*, Springer, pp. 1-27, 2022
4. **Ziyi Liu**, Lei Li, Mengxuan Zhang, Wen Hua, and Xiaofang Zhou, FHL-Cube: Multi-Constraint Shortest Path Qerying with Flexible Combination of Constraints, *Proceedings of the VLDB Endowment*, Issue 11, pp 3112–3125, 2022

Submitted manuscripts included in this thesis

1. **Ziyi Liu**, Lei Li, Mengxuan Zhang, Wen Hua, and Xiaofang Zhou, Approximate Skyline Index for Constrained Shortest Pathfinding with Theoretical Guarantee, submitted to *ICDE* on July 2023.

Contributions by others to the thesis

No contributions by others.

Statement of parts of the thesis submitted to qualify for the award of another degree

No works submitted towards another degree have been included in this thesis.

Research involving human or animal subjects

No animal or human subjects were involved in this research.

Acknowledgments

Start this section on a new page [the template will handle this for you].

First and foremost, I would like to express my profound gratitude to my supervisors, Prof. Wen Hua, Prof. Lei Li, and Prof. Xiaofang Zhou, whose unwavering support, guidance, and encouragement have been instrumental in completing this thesis. Your expertise, patience, and enthusiasm for the subject matter have been truly inspiring, and I am grateful for the opportunity to have worked under your mentorship.

I would like to extend my heartfelt appreciation to my thesis committee members, Dr Jiwon Kim, Prof. Guido Zuccon, and Dr Miao Xu, for their invaluable feedback, constructive criticism, and insightful suggestions throughout the development of this work. Your expertise and dedication have significantly enriched my understanding and helped refine my research.

I am deeply thankful to my colleagues in General Purpose South Building (78) at UQ for their stimulating discussions, insightful suggestions, and camaraderie throughout my time in the program. Your intellectual curiosity and collaborative spirit have made this journey a truly enjoyable and rewarding experience.

I would also like to express my gratitude to the administrative staff of the graduate school, who have provided essential support and ensured that my research could progress smoothly.

My friends, both inside and outside of academia, have played a crucial role in keeping me grounded, providing much-needed encouragement and respite from the rigours of research. Thank you for being there for me during the highs and lows of this journey.

Last but by no means least, I owe an immense debt of gratitude to my family. To my parents, Runze Liu and Chunhong Zhao, your unwavering faith in me, love, and sacrifices have been the bedrock upon which I have built my dreams. To my partner, Jiajia Chen, your patience, love, and encouragement have been my constant source of strength, and I could not have done this without you.

In conclusion, I dedicate this thesis to all those who have supported, inspired and believed in me throughout this journey. Thank you for making it possible for me to reach this milestone.

Financial support

This research was supported by UQ research training scholarship and UQ research higher degree scholarship. Also, it was partially supported by the Australian Research Council under Grant No. DP200103650 and LP180100018, Hong Kong Research Grants Council (grant 16202722), and was partially conducted in the JC STEM Lab of Data Science Foundations funded by The Hong Kong Jockey Club Charities Trust.

Keywords

constrained shortest path, skyline, hop labelling, tree decomposition, forest

Australian and New Zealand Standard Research Classifications (ANZSRC)

ANZSRC code: 080604, Database Management, 80%
ANZSRC code: 080201, Analysis of Algorithms and Complexity, 20%

Fields of Research (FoR) Classification

FoR code: 460106, Spatial Data and Applications, 80%
FoR code: 460505, Database Systems, 20%

Order for the Remainder of the Thesis

Remainder of the thesis should be in the following order

- Dedications (if applicable)
- Table of Contents
- List of Figures and Tables
- Main text of the thesis
- Bibliography or List of References

Contents

List of Figures

List of Tables

Chapter 1

Introduction

In this chapter, we briefly introduce the research in this thesis, including background, problem statements, contributions, and the organisation of the thesis.

1.1 Background

Route planning plays a crucial role in daily life, with various path problems having been explored over the past decades. Real-world route planning often involves multiple criteria based on user preferences. For instance, a user might be constrained by a budget, affecting their willingness to incur toll charges on highways, bridges, tunnels, or congestion zones. Some major cities advise reducing the number of significant turns (e.g., left turns in countries with right-side driving)[1] due to their increased accident risk. Other criteria might include tunnel height restrictions, road capacity, slope gradients, elevation increases, tourist drive lengths, and the number of transportation changes. Despite these multifaceted needs, many algorithms focus solely on optimizing a single objective, such as minimizing distance [1–3], travel time [4, 5], fuel consumption [6, 7], or battery use [8]. However, combining these criteria provides routes with diverse implications, better catering to user requirements. For instance, *TollGuru*[2] integrates travel time, toll charges, and fuel consumption in its route planning. This multi-criteria approach offers a more comprehensive solution compared to the traditional shortest path method. Moreover, identifying an optimal path based on multiple criteria is an important research area in both transportation [9, 10] and communication fields, particularly in *Quality-of-Service (QoS)* constrained routing [11–15]. In transportation literature, this problem is often termed *MCSP*, drawing from the *Multi-Constrained Optimal Path (MCOP)* concept in *QoS* routing [11].

[1] https://www.foxnews.com/auto/new-york-city-to-google-reduce-the-number-of-left-turns-in-maps-navigation-directions

[2] https://tollguru.com

Criteria for route planning can be divided into *additive criteria* and *non-additive criteria*. For *additive criteria*, constraints applied to a path depend on the summation of corresponding values from the edges or links that comprise the path. This encompasses factors such as travel time and fuel consumption in transportation, or delay time and administrative weight in *QoS*. Contrarily, *non-additive criteria* evaluate constraints on individual edges to determine a path's feasibility, like a transportation tunnel's minimum height or bandwidth considerations in *QoS*. Addressing *MCSP* problems with non-additive criteria can be straightforward, often achieved by pruning edges not meeting the necessary constraints [16]. Yet, dealing with *MCSP* problems involving additive criteria is a more intricate endeavor, meriting detailed research. Most existing studies have predominantly centered on additive criteria. Thus, our analysis will focus on *additive criteria*, with a formal presentation of the problems in Section 1.2.

However, we could hardly find one result that is optimal in every criterion when there is more than one optimisation goal. One way to implement multi-criteria route planning is through path/route skyline [17–19], which provides a set of paths that cannot dominate each other in all criteria. However, skyline paths computation comes with significant time complexity $O(c_{max}^{n-1} \times |V| \times (|V| \log |V| + |E|))$, where $|V|$ is the vertex number, $|E|$ is the edge number, c_{max} is the highest criteria value, and n is the number of criteria [20] (c_{max}^{n-1} is the worst case skyline number). Additionally, it is impractical to provide users with a large set of potential paths from which to choose. Therefore, *Multi-Constraint Shortest Path (MCSP)*, as another way to consider multiple criteria, is widely applied and studied. Specifically, it finds the best path based on one objective, whilst it requires other criteria that satisfy some predefined constraints. For example, suppose that we have three objectives: distance d and costs c_1, c_2, and we set the maximum constraints C_1 and C_2 on the corresponding costs. Then this *MCSP* problem finds the shortest path p whose cost $c_1(p)$ is not greater than C_1 and $c_2(p)$ is not greater than C_2. Let's delve into the examples of *MCSP* queries for clarity:

Example 1: A backpacker travelling between two cities aims to identify the most eco-friendly route that doesn't exceed a 5 *dollar carbon footprint and offers at least 10 km of pedestrian paths. While the carbon footprint is a straightforward measure, the pedestrian path distance, as denoted in certain urban planning guidelines, is crucial for backpackers, often overlooked in traditional routing. For instance, a pathway through a park might be lengthier than a main road, but offers a more immersive experience.*

Example 2: An electric vehicle (EV) driver, journeying between two charging stations, seeks a route with minimal energy consumption. However, they must ensure that there are at least three fast charging stations en route, with the entire journey costing under 15 *dollars in charging fees. Since energy consumption is pivotal for EV drivers, a slightly longer journey is acceptable if it ensures battery longevity and available charging options.*

Example 3: An IT manager wants to establish a data route with the highest data transfer rate.

This route should have an associated cost under 50 *cloud credits (a digital currency in some cloud platforms) and a maximum latency of 100 milliseconds, ensuring efficient real-time data transfers.*

To our knowledge, prevailing *MCSP* algorithms predominantly utilize linear programming or graph search techniques. Their inherent complexity often restricts scalability in expansive networks [10, 11, 21]. The *Constrained Shortest Path (CSP)* problem, representing a simpler scenario with a singular constraint, is foundational for *MCSP* resolutions. Solutions for *CSP* can be organized into categories: *exact/approximate* and *index-free/index-based*. The *exact CSP* solution offers a precise result that aligns with users' specifications [20, 22–24]. However, being NP-Hard [25, 26], it is computationally intensive. As a remedy, *approximate CSP* algorithms have been introduced [13, 20, 26–28]. They prioritize reduced computation times over optimal path lengths, though the achieved acceleration remains relatively modest compared to exact methods [23, 29]. To further bolster query efficiency, several indexing methods have emerged, expanding on the foundational structures of shortest path problems. For instance, *CSP-CH* [30] augments the conventional *CH* index [31] to accommodate exact *CSP* queries. Although these indices often demand considerable construction time and can lack flexibility with fluctuating weights or costs [32–34], they excel in efficient query response. Notably, the *2-hop labelling* approach [35–37] stands out as the contemporary gold standard for shortest path indexing.

In this thesis, our objective is to offer solutions for addressing various *MCSP* queries, ensuring both expedited index construction and enhanced query processing efficiency. Initially, we introduce the *Forest Hop Labeling (FHL)*, a 2-hop labeling-based index structure, tailored to address *CSP* queries proficiently. Subsequently, we evolve *FHL* to cater to *MCSP* queries subject to a predetermined set of constraints. Progressing further, we unveil the *FHL-Cube* designed to tackle a broader spectrum of *MCSP* queries. Specifically, the *FHL-Cube* is engineered to respond to the versatile *MCSP* challenges, empowering users to define diverse constraint combinations in alignment with their immediate needs. Culminating our efforts, we present the α*-FHL*, an approximate indexing solution, exemplifying reduced memory demands and swift index construction.

1.2 Problem Statement

1.2.1 2-Hop Labelling on CSP

It is non-trivial to extend the 2-hop labelling to answer the *CSP* queries. As proven in [35], it is already NP-H to find the optimal labels with the size of $\Omega(|V||E|^{1/2})$ for the single-criterion shortest path. It would be much harder for the *CSP* problem for the following reasons.

First, the constraint C is unknown before the query is issued, which means that the index has to be able to cope with all possible values of C. Therefore, storing the skyline path information in the

index is inevitable. As discussed previously, the search for the skyline path is very time-consuming, and therefore the existing *CSP* solutions all suffer from low efficiency. Consequently, none of the existing indexes is constructed by the exact skyline path but resorts to the approximate path search. The *CSP-CH* [30] chooses to contract a limited number of vertices and adopts heuristic testing before each skyline path expansion, which leads to a subset of skyline shortcuts with a smaller index size but slower query time. *COLA* [38] resorts to graph partitioning and approximate skyline path search to reduce intermediate result size, but sacrifices query accuracy. Moreover, its performance is still limited by the skyline path search when the approximate ratio is small, and it has to build different indexes for different approximation ratios. Therefore, the skyline path search is the bottleneck of existing methods and prevents the existence of a full exact index, without which a fast exact *CSP* query answering is impossible. In this work, we propose a *skyline path concatenation* algorithm to completely avoid the expensive skyline path search and construct a 2-hop index that stores the complete skyline paths without compromising query accuracy or efficiency.

Second, the number of skyline paths becomes larger as the path grows, so it can hardly scale to larger networks. Therefore, we propose the *forest hop labelling* framework, which first partitions the graph into regions and then builds indexes within and across regions in parallel.

Third, concatenating the skyline path is not an easy task. Since concatenation of the skyline path does not necessarily generate a skyline path, it requires $O(mn \log mn)$ time to validate the skyline results, where m and n are the sizes of the two sets of skyline paths to be concatenated. To speed up this process, we provide several insights into the skyline path concatenation and propose an efficient concatenation method to incorporate the validation into the concatenation and meanwhile prune the concatenation space. Although the worst-case complexity remains the same when all the concatenation results are all skylines, our algorithm is much more efficient in practice.

Last but not least, the final index is essentially for a skyline path query, since it contains all the skyline path information. Therefore, it would take approximately $m \times n \times h$ times to concatenate the skyline before answering the query, where h is the number of hops. This is much higher than the traditional 2-hop index, which only requires h times of calculation. In other words, the query performance of the index deteriorates dramatically and loses the promise of its high efficiency. Therefore, we propose a *rectangle-based pruning* technique to prune the useless concatenation as early as possible. Furthermore, for the *CSP* query, we propose a constraint-based pruning technique for faster query answering and also provide a pruning technique for forest labels. As shown in the experiments, our approach can answer the *CSP* query within 1 ms, and the proposed pruning techniques can further reduce it by two orders of magnitude, whilst the existing methods are thousands of times slower.

1.2.2 Hop Labelling on MCSP

The *Multi-Constraint Shortest Path (MCSP)* problem aims to find the shortest path between two nodes in a network subject to a given set of constraints. Answering *MCSP* queries with *2-Hop Labelling*-based index inherits all the challenges of the version of *CSP*. Moreover, it is typically processed as a *skyline path* problem. However, the number of intermediate skyline paths becomes larger as the network size increases and the constraint number grows, causing dramatic growth of computational cost and further making the existing index-based methods hardly capable of obtaining complete exact results. This thesis extends *Forest Hop Labeling* to *CSP* with a single constraint. Because the path concatenation algorithm and the pruning technique were only designed for one constraint, its multi-constraint version in [39] was only a straightforward and slow solution. Hence, we generalise the path concatenation methods into a multi-cost scenario and extend the pruning techniques to a multi-constraint scenario to achieve higher performance. Although they are generalisations of *CSP*, the problem in the multi-dimensional space is much harder than the 2-dimensional one such that *CSP* is only a very special case of *MCSP* with unique properties. Therefore, we provide new insightful properties of high-dimensional skyline path concatenation and pruning to facilitate more efficient index construction and query answering in *MCSP*.

1.2.3 Hop Labelling on Flexible MCSP

The flexible *MCSP* requires answering queries of combinations of arbitrary criteria, so the first challenge lies in how to build an index to support the flexibility of the query. Even though the higher dimensional *MCSP* queries take larger index space with longer construction and query time than the lower ones, it is observed that the query results of the low dimension also belong to those of the high dimension. This insight provides us with a chance to derive the high-dimensional skyline paths by accumulating those from the low dimension. To this end, it is critical to distinguish between low-dimensional skyline paths and high-dimensional ones and establish their relations. Specifically, we first propose *exclusive skyline path* to determine the lowest dimension of a skyline path, so that the skyline paths can be organised hierarchically into a *cube* by categorising them into different subspaces. Then, we extend *FHL* to *FHL-Cube* by changing the index values from paths of only one space to a cube that organizes all spaces. Next, we establish their relations and propose theorems to derive high-dimensional results from lower ones. Finally, the higher dimensional cuboid (the basic component of the cube) is smaller than the lower ones (reverse to the *FHL* as indicated by the database sizes), and the flexible *MCSP* can be handled naturally.

In addition, the cuberisation of skyline paths involves cube concatenation in both index construction and query processing, which would cause numerous intermediate skyline results with heavy computation. The difficulty here is to improve the cube concatenation efficiency such that index

construction and query processing can be accelerated as well. Naively, we concatenate the skylines of the corresponding cuboids from two-dimension to higher dimensions, since the subspaces are organized hierarchically. However, there are multiple two-dimension subspaces waiting for concatenation. Hence, we dedicate ourselves to pruning the unnecessary computation in cube concatenation through subspace pruning and hop pruning, respectively. To be specific, we determine the path range of each subspace through calculating the lower and upper bounds of each dimension, such that the subspaces whose path range is dominated by others could be safely pruned without affecting the correctness. In terms of the hop pruning, we precompute the lower bound and the upper bound of the path results concatenated by each hop and those hops with their lower bound surpassing others' upper bound can be safely pruned. Moreover, it is proved that those pruned subspaces or pruned hops of one specific dimension also apply to all its higher dimensions. Finally, both the index construction and query processing of *FHL-cube* are highly improved owing to the proposed pruning techniques of cube concatenation.

Finally, the tree height of the *tree decomposition* structure during the index construction is usually large, which seriously deteriorates the index performance since a larger tree height indicates more cube concatenation in both index construction and query processing. To alleviate this problem, we analyse the forest structure and propose several principles to optimise the index structure. As for the query processing, our index is flexible for any combination of criteria because of the skyline path relation established between the low dimension and the high dimension.

1.2.4 2-Hop Labelling on Approximate CSP

In the index-free *CSP* algorithms, each visited vertex maintains a set of skyline paths from the source. We should consider which paths can be saved to compose the final skyline path in further searches, resulting in a large amount of memory consumption. That is why the exact index-free solution can hardly efficiently process the *CSP* query in the real-life road network. Then exact index-based approaches are proposed, but there are larger challenges, since finding a "harmonious" trade-off between query efficiency and index size is difficult. For instance, *CSP-CH* [30] only keeps a few skyline results on its shortcuts to save the construction and index size, but its query time suffers. *Forest Hop Labelling (FHL)* [39–41] can produce the fastest query processing by avoiding the online search. In contrast, its index is several orders of magnitude larger than other index-based methods, since it has to store a set of exact skyline indices as labels for each vertex.

On the other hand, approximate *CSP* algorithms are developed to save computational time and indexing space by compromising the optimality of the path length. Specifically, approximate solutions preset an approximate ratio α and ensure that the length of the final path returned is no longer than α times the optimal path length. Most index-free approximate methods have limited speedup compared to exact solutions, as they still suffer from expensive computation in large road networks. So far, no

approximate solution has been applied directly to the *CSP* index. *COLA* [38] is the state-of-the-art approximate solution. However, its performance improvement is limited because of the approximate search,i.e., the approximation is applied to the search-based query processing rather than the index.

Motivated by this, we propose α-*FHL*, an approximate *CSP* index that is space efficient and quick to construct. However, it is non-trivial to achieve this goal. We explain the technical challenges in the following aspects.

Approximate Framework. Let F be the solution to construct *FHL* index, and Λ° represents the index result of graph G. We employ the approximate ratio $\alpha \geq 1$ as a parameter of F. The ideal approximate result of $F(\alpha, G)$ is to prune Λ° to $\alpha \otimes \Lambda^\circ$, s.t., $F(\alpha, G) = \alpha \otimes \Lambda^\circ \subseteq \Lambda^\circ$. In $\alpha \otimes \Lambda^\circ$, all skyline paths between the same two vertices cannot α-dominate each other. The special case is when $\alpha = 1$ such that $F(1, G) = 1 \otimes \Lambda^\circ = \Lambda^\circ$, which refers to the exact *FHL* index. However, $\alpha \otimes \Lambda^\circ$ means that α is acting on Λ°. Thus, one way to achieve $|\alpha \otimes \Lambda^\circ|$ is to obtain the exact index and then prune it by α. It is costly for index construction. The construction time is even longer than the exact *FHL*. Therefore, we propose the approximate framework of *FHL* for pruning the index in different phases of indexing. Specifically, *FHL* indexing includes two phases: 1) Skyline Tree Decomposition; and 2) Skyline Label Assignment. Accordingly, in our approximate framework, we design 1) a bottom-to-up approximate tree decomposition to form trees with approximate skyline shortcuts; and 2) a top-to-down approximate label assignment manner. Note that our approximation techniques take place around the structure of Tree Decomposition. However, it has not been investigated in the literature.

Allocation of α. It is challenging to reasonably adopt α for pruning in computing the *FHL* index, because 1) we cannot directly use α to calculate the approximate index because adopting α in each iteration of the labelling leads to an over-approximate ratio in the results. Thus, the answer to a *CSP* query is far from optimal; 2) on the contrary, a small approximate ratio applied in each iteration will result in insufficient pruning power. Thus, it will produce a less effective and redundant index, which will further impact the efficiency of query processing; 3) since each label may experience different times of approximation, it is difficult to maintain the same pruning power over them; 4) a fixed allocation plan is not enough to solve the problem. Specifically, if we compute a label with abundant approximate power, the subsequent labels are approximated insufficiently. For the reasons mentioned above, we propose efficient methods to allocate α and also recycle the approximate power overused. Furthermore, we theoretically analyse the relations between the allocation of α and the tree decomposition structure.

Skyline Operator. Skyline path concatenation is an important operator in both index construction and query processing, but it is very time-consuming, especially in large-scale graphs. When we compute a label $L(v, u)$, it requires the concatenation of the skyline paths in $L(v, w_i)$ and $L(w_i, u)$, where w_i is a hop between v and u. If there are multiple intermediate hops, we must compute the concatenated paths through each of them. For each hop, we cannot guarantee that the concatenated results from

v to u are still skyline paths. Therefore, we need $O(mn\log(mn))$ time to verify their approximate dominance relations, where m and n are the sizes of the two sets of skyline paths to be concatenated. For multiple hops, we have to generate a set P_s that includes all concatenated results through different hops and then select the final approximate skyline paths of P_s. Suppose that the size of hops is h, we will concatenate around $m \times n \times h$ times. Although α-*FHL* can reduce concatenation times by approximating each label, we still need dominance verification in both single-hop and multiple-hop concatenations. To speed up this process, we designed an efficient approximate algorithm α-SPC_1 for single-hop concatenation and a pruning technique α-SPC_n to reduce the number of intermediate hops.

1.3 Contribution

1.3.1 2-Hop Labelling on CSP

We propose a skyline path concatenation-based *CSP-2Hop* labelling method that avoids the expensive skyline path search and achieves efficient *CSP* query answering and index construction. We further propose a *forest hop labelling* to extend the *CSP-2Hop* to larger networks. Afterwards, we present several concatenation insights and propose an efficient method to incorporate skyline validation into concatenation and reduce the concatenation space. We also design a rectangle-based pruning technique to further speed up the multi-hop concatenation during index construction and query answering, and a constraint pruning method to expedite *CSP* and forest *CSP* specifically. Finally, we thoroughly evaluate our method with extensive experiments on real-life road networks, and the results show that it outperforms the state-of-the-art methods by several orders of magnitude.

1.3.2 Hop Labelling on MCSP

We first propose a skyline path concatenation-based *MCSP-2Hop* method that avoids the expensive skyline path search to achieve efficient *MCSP* query answering and index construction. Several concatenation insights are further presented to reduce the concatenation space and incorporate multi-dimensional skyline validation into concatenation. Then, we further propose *forest hop labelling* for scalability. We also design a n-Cube pruning technique to further speed up the multi-hop concatenation during index construction and query answering, and a constraint pruning method to speed up *MCSP* specifically. Finally, we thoroughly evaluate our method with extensive experiments on real-life road networks and the results show that it outperforms the state-of-the-art methods by several orders of magnitude.

1.3.3 Hop Labelling on Flexible MCSP

We introduce a novel flexible *MCSP* problem and propose the *FHL-Cube* index for its query processing. Furthermore, we achieve query flexibility by organising and deriving paths with different dimensions hierarchically as a cube, and further propose pruning techniques for faster cube concatenation. Then, we optimize the boundary label structure and concatenation method for faster index construction and querying. Finally, we conduct extensive evaluations on real-world road networks to verify the superiority of our approach compared to state-of-the-art methods.

1.3.4 2-Hop Labelling on Approximate CSP

We provide an approximate framework for *FHL* with a theoretical guarantee of the correctness of our approximations. Then, we propose a threshold to approximate the indexing labels selectively. Additionally, we design a recycling technique to reuse the redundant approximate power on labels. Afterwards, we propose an efficient operator to concatenate skyline paths derived from different approximate ratios so that the concatenated skyline results approximated by a specific ratio can be found rapidly. Moreover, we thoroughly evaluate our methods with extensive experiments in real-life road networks, and the results show that they outperform state-of-the-art methods on query processing with a practical index size and construction time.

1.4 Thesis Organisation

The rest of this thesis is organised as follows: In Chapter 2, we review existing works on solving the problems of the shortest path, *CSP*, *MCSP*, and approximate *MCSP*. In Chapter 3, we first present our *FHL* index, which can answer any *CSP* query with a constraint efficiently in real road networks. Then, we extend *FHL* to answer any *MCSP* query with a set of constraints. In Chapter 4, we introduce the problem of flexible *MCSP* and propose *FHL-Cube* to organise multi-dimensional skyline path index in cubes. It can efficiently answer *MCSP* queries with any combination of constraints. In Chapter 5, we note that the hop labelling-based index of *CSP* may be huge in large-scale networks. Therefore, we propose different approximate manners to optimise *FHL* in terms of scalability. Compared to *FHL*, our approximate version achieves faster index construction and less memory consumption and then improves the performance of *CSP* queries. Finally, Chapter 6 includes the conclusions and the future research directions suggested by the thesis.

Chapter 2

Literature Review

In this section, we first define path problems we talked about in the report, in terms of the criterion number, including shortest path problem, constrained shortest path problem, and multi-criteria problem. We then provide a brief description of the existing techniques related to each field above.

2.1 Introduction

2.2 Shortest Path

We discuss the shortest path problem and its solutions on query answering. We define a graph $G(V,E,W)$, where V is a set of vertices and $E = (u,v) \in V \times V$ is a set of edges. Edge edge (u,v) has a non-negative weight $w(u,v) \in W$, which can be represented as the distance, travel time, fuel consumption, toll charge, and so forth. We use the distance as the example to represent the weight in the report. A path p from s to t is a sequence of vertices $p =< v_0, v_1, .., v_k >$ with $v_0 = s, v_k = t$ and $\forall(v_i, v_{i+1}) \in p(v_i, v_{i+1}) \in E$. The length of a path is $w(p) = \sum_{i=0}^{n} w(v_i, v_{i+1})$, and the shortest path is the path with the minimum $w(p)$.

The shortest path problem can be classified into several categories based on the number of the source and destination.

Definition 2.1 (**Single-Source Shortest Path Problem**). *Searching the shortest paths from one single source vertex s to all other vertices in the graph. The query is denoted as* $q(s,V)$.

Definition 2.2 (**Single-Destination Shortest Path Problem**). *Contrary to the Single-Source problem, querying the shortest paths from all the other vertices to the one destination vertex t, which is denoted as* $q(V,t)$.

Definition 2.3 (**Point-to-Point Shortest Path Problem**). *Querying the shortest path between a source vertex s to a destination vertex t, which is denoted as* $q(s,t)$. *We call the given vertex pair* (s,t) *as a OD pair for such query.*

Definition 2.4 (**All-Pairs Shortest Path Problem**). *Looking for the shortest paths between each pair of vertices in a given network. The query is denoted as* $q(V,V)$.

In this section, we mainly discuss the point-to-point shortest path algorithm as it is the most commonly used in reality and the basis of the other problems. Moreover, the shortest path query or problem mentioned in the following means the point-to-point version. The solutions on the shortest path problem can be classified into Index-Free and Index-Based algorithms. The Index-Free algorithms only have the query processing stage to conduct different online search methods. Compared with Index-Free algorithms, Index-Based algorithms have an extra stage, which is called index construction. Such methods can produce faster query processing in terms of the cost of index construction. In the next sections, we will explain several representative techniques in the two types of algorithms.

2.2.1 Index-Free

Dijkstra's Algorithm

Given a graph $G(V,E,W)$ and query $q(s,t)$. *Dijkstra's* Algorithm [1] search start with s and find the shortest path one by one with increasing order of path length. The algorithm will terminate when the algorithm visits t. The mechanism is:

1. Mark the start vertex s as reached.
2. Respectively, calculate the weight from s to its adjacent vertices and mark them as reached. Then we select the vertex that has the smallest weight and mark it as settled.
3. Respectively, calculate the weight between s and the vertices adjacent to the settled vertex. Then we select the vertex with the smallest weight so far and mark it as settled.
4. Repeat step 3 until the end vertex t is marked as settled.

There are three states of vertices when we implement *Dijkstra's* Algorithm, which are unreached, reached or settled. During the movement of the algorithm, if $\forall v \in V$ is not found, we mark v as unreached and set $d(v)$, which is the current best distance of the shortest path $p(s,v)$ from s to v, to infinity. Once the search has found v, we mark v as reached and update $d(v)$. For every edge from u to v, which has weight $w(u,v)$, we update $d(v)$ through a basic operation in the shortest path computation called relaxation. In a relaxation function $Relax(u,v,w)$, if $d(v) > d(u) + w(u,v)$, we update $d(v)$ to

$d(v) = d(u) + w(u,v)$. Moreover, the algorithm uses a priority queue Q to manage the reached vertices. The priority queue always takes out the minimum d of reached vertices, the set of d are the keys in Q. If a reached vertex v is popped from Q, $d(v)$ becomes fixed and considered as the optima result of $p(s,v)$, we mark v as settled.

Hence, in Step 1, the movement starts with s, we mark s as reached and update $d(s)$ to 0. Other vertices are initialised to infinity. In Step 2, we do relaxation for each adjacent vertex of s, and add them into the priority queue as reached vertices. Then the priority queue will pop the vertex having the smallest weight. Then we change the state of this vertex to settled. In step 3, we relax each vertex adjacent to the settled vertex and add them into the priority queue if they have not yet been added. Afterwards, we mark the newly added vertices as reached. Then the priority queue pops the vertex having the smallest weight so far. This vertex becomes the newly settled vertex, and then we repeat step 3. The termination rests upon the query that has the end vertex or not. If the end vertex t is not given in the query, the algorithm will terminate when the priority queue is empty. In this case, we can identify the shortest path and distance of every vertex that is not marked as unreached. Otherwise, the algorithm will terminate when the given t is settled.

Dijkstra's algorithm applies a brute-force search for exploring the optimal shortest-path. The method is regarded as a greedy-like algorithm. *Dijkstra's* algorithm uses the procedure of successive approximation in terms to *Bellman Ford's* optimality principle [42]. Thus, *Dijkstra's* algorithm is able to solve the dynamic programming equation by the reaching method [43–45].In addition, apart from improving the performance of *Dijkstra's* algorithm by directly re-designing the search into bidirectional manner [46, 47], [48] proposes a goal-directed search method to enhance the *Dijkstra's* by pre-computing the shortest distances. The algorithm divides the graph into k clusters without overlapping. Then, it stores the start and endpoint and the shortest path between each pair of clusters. The running time of query answering can achieve $\sqrt{k}$ time faster than *Dijkstra's*.

A* Algorithm

A^* Algorithm [49] is based on the principle of *Dijkstra's*, but it provides a heuristic function h to estimate the distance from the current node to the destination. This improvement can avoid a part of redundant searches.

Given a graph $G(V,E,W)$ and query $q(s,t)$, A^* Algorithm will follow the steps of *Dijkstra's*, but calculate the keys in priority queue Q according to the equation $f(s,v) = d(v) + h(v,t)$. For $\forall v \in V$ is found, this equation will calculate the key $f(s,v)$ through adding the current shortest distance $d(v)$ from s to v with a heuristic function $h(v,t)$, which is a lower bound for distance from v to t. For instance, when we search the shortest path on G, we can use straight-line distance or Euclidean distance from vertex v to the destination.

There are several variants of the A^* algorithm with the help of different techniques. For instance, landmarks [50] adds an extra prepossessing stage to select several landmarks first. Then, it computes and stores the shortest path between the vertices of all these landmarks. Moreover, the algorithm provides a constant-time lower-bound technique using the triangle inequality property and the computed distances. The technique includes three components, which include $A*$ algorithm, the landmark chosen, and the triangle inequality.

[51] proposes a solution based on the concept of reach. The algorithm stores a reach value and the Euclidean coordinates of all vertices. It takes advantage of combining the A^* algorithm. Compared with the work of [50], it can achieve better performance when the number of landmarks is a few and become worse when the number increases. The disadvantage of [51] is that the algorithm is depended on domain-specific assumptions. Moreover, it has longer prepossessing complexity and impracticality in a dynamic setting [52].

[53] proposes an approximate landmark-based technique to estimate the shortest distance in large-scale networks. Its landmark selection technique is more accurate. It means that the technique can take up about 250 times less space than choosing landmarks randomly.

[54] proposes a beacon-based algorithm, which is designed for triangulation. By using the triangle inequality, the unmeasured distances can be deduced by the method. Given a constant number of beacons, the algorithm provides a method of triangulation-based reconstruction to show that the multiplicative error is $\frac{1+\delta}{1-\varepsilon}$.

2.2.2 Index-Based

Contraction Hierarchies(CH)

Even though the shortest path problem can be solved by the *Dijkstra's* Algorithm, it is impractical to work on a large-scale road network as a significant number of vertices. *CH* as a speedup method is optimised to impose properties of graphs representing road networks. It is first proposed in [31]. Apart from the statistic networks, the idea of CH has also extended to time-dependent networks [55] [56]. The speedup technique is achieved by the idea of shortcuts in the phase of index construction. By adding a sequence of shortcuts between vertices in the road network, it can skip over several non-crucial nodes during the shortest path query processing. The query processing is achieved by the *Dijkstra*-based search on a highly hierarchical graph. For instance, in highway-node routing, several highway junctions are considered as more important nodes and given higher levels in the hierarchy than a junction resulting in a dead-end. The hierarchical structure imposes shortcuts to reserve the distance between a pair of such important junctions in index construction. Thus, it avoids considering all the paths between the junctions in query processing.

CH is a specific case of highway-node routing [31], which means its contraction hierarchy provides a level for each vertex. The number of levels can reach up to n, which is the maximum number of vertices [52]. By searching in an upwards manner over the hierarchical graph, the query performance can be improved, and space complexity can be reduced. Therefore, CH is a method using the idea of node ordering for assigning levels and hierarchy construction for searching. To achieve these aims, [31] proposes a method to discover a way to order nodes for contraction, criteria used for adding shortcuts, and the search method on the resulting hierarchy.

On the node ordering, a significant node ordering admits the search to achieve the topmost node in the hierarchy in a logarithmic number of steps. On the contrary, it will result in inefficiency if the node ordering requires a linear number of steps. One challenge on producing a significant node ordering is due to the coefficients of different vertices. In detail, when node v is contracted, the priorities of other nodes might be changed. Contracting a node here means adding shortcuts between pairs of more important neighbours. To handle the challenge, CH proposes the technique to lazily update the priorities to identify the next contracted node in each contracting time. For instance, before contracting a node v, the algorithm updates its priority and check whether it exceeds the priority of v', which is the second priority vertex. If true, v is reinserted into the priority queue, and v' is contracted in the next round. During the iterations, *CH* provides a technique to recompute the priority of the neighbours of the currently contracted vertex. Also, all priorities are reevaluated periodically for rebuilding the priority queue. In the hierarchical graph, several heuristics are used to identify the priority of vertices.

The hierarchical graph needs to minimise the number of edges which added to the graph after finishing the contractions, as the number of shortcuts can mainly affect the efficiency of pre-processing and query answering. The heuristics used in *CH* can provide the optimal node ordering, which is bottom-up and top-down heuristics. The bottom-up heuristics can greedily identify the contracting order. It applies in situations in which the node ordering is previously unknown. However, once the current contraction has been completed, the next node can be selected for contraction. For top-down heuristics, it pre-computes the entire node ordering and then starts the operation on contractions. As the optimal node ordering has been already given in advance, the top-down heuristics produce better results with the cost of more pre-processing time.

In node contractions, we can work out every shortest path between the node's immediate neighbours and insert shortcuts for them. When contracting an arbitrary node v, we need to determine whether the shortcuts are needed. In detail, we should insert the shortcut edge (u,w), iff path $< u,v,w >$, which includes $(u,v) \in E$ and $(v,w) \in E$, is the only shortest path from u to w. The simply way to achieve this is using *Dijkstra's* Algorithm for computing then mark the node as settled. Figure 2.1 shows an example graph on the left side and a contracted graph of C on the right side. To contract C, we can insert shortcuts from A to E and A to B, which are shown by the dash lines, as the path $< A,C,E >$ and $< A,C,B >$ are the shortest paths. We do not need to add shortcuts from B to E, as $< B,C,E >$ is

longer than the existing path $< B, D, E >$.

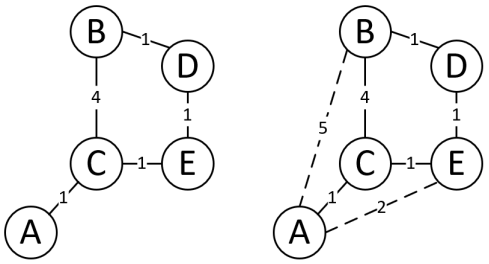


Figure 2.1: an example node contraction

Also, contracting a vertex v does not mean to delete the vertex completely, but the vertex can be ignored in the rest of the contractions. The process of identifying if the shortest path between u and w contains v is called a witness search, which is the most important part of node contraction [57]. It can be performed by computing a path from u to w by a forward *Dijkstra*-based search within the nodes without contracting.

There have been several improvements in shortcut determination. To contract a vertex v, we denote the set of incoming vertices of v as u_i and the outgoing vertices as w_j. We can stop the search for shortcuts on v when we settle a vertex with a distance greater than $d(u_i, v) + w_max(v, w_j)$. In fact, utilizing Another a 1-hop backward search from each of the w_i gives an even better bound. The third method is to apply a heuristic. For each *Dijkstra-based* search from each of u_i, we can set a limit on the number of hops, which means the distance in number of edges from u_i, and on the number of vertices settled. With the heuristic, it is possible to skip the shortest path from u_i to w_j, which does not pass v. Thus, it will insert the shortcut (u_i, w_j) unnecessarily. Even the unnecessary shortcuts do not affect the correctness, but the performance of query processing worsens when such shortcuts keep increasing. In this case, we will face a trade-off, which means the heuristic can save much time in index construction at the cost of several unnecessary shortcuts.

In the query processing, the search is similar to *Dijkstra's* Algorithm, but has extra conditions. The search on the contracted graph, or called overlay graph, can only follow edges going to nodes with a higher contraction order than the current node. We give the definition of the query algorithm as follows. Given a source s and a target t, we can conduct a full *Dijkstra's* search from s forwards and consider the edges (u, v) with the priority $u < v$ in upward graph, which is defined as $G_\uparrow = (V, u, v : u < v)$. On the contrary, we conduct a backward version of *Dijkstra* search from t and consider the edges (u, v) with $u > v$ in downward graph, which means $G_\downarrow = (V, u, v : u > v)$. Let V_s be the set of vertices settled in two directional *Dijkstra* search. The shortest path distance $d(s, t)$ is calculated by $min\{d(s, v) + d(v, t) : v \in V_s\}$.

2-Hop Labelling

2-Hop Labelling first proposes in [35] to answer the distance query accurately with only one intermediate hop vertex. It answers the shortest distance query by table lookup and addition. Specifically, $\forall u \in V$, we assign a label set $L(u) = \{v, w(v,u)\}$ to store the minimum weights between a set of *hop vertices* v to u. To answer a query $q(s,t)$, we first determine the intermediate hop set $H = L(s) \cap L(t)$. Then we can find the minimum weight via the hop set: $min_{h \in H}\{w(s,h) + w(h,t)\}$.

Pruned Landmark. It is the first algorithm that uses the *2-hop labelling* method. As the technique guarantees, the search spaces can be pruned dramatically, and it is used in the small-world graphs in general [36] [58] [59]. In the index construction, the label sets are generated iteratively. For each iteration, it generates in-labels from other vertices to a vertex u by conducting a forward *Dijkstra*'s from u. It then generates out-labels from u to other vertices by conducting a backward *Dijkstra*'s to u. Moreover, it sets rules on pruning techniques. In detail, if the existing labels have already answered the shortest distance between u and v, the search out of v can be pruned and the edges of v can be ignored in searches. Hence, the time complexity of index construction is $2|V|$ and the search spaces can be narrowed down as the label size increases.

Hop Doubling. [60] proposes a hop-doubling labelling method to answer point-to-point distance querying in massive scale-free graphs, which is non-trivial for numerous applications. The problem of point-to-point distance query has been well studied for road networks, but most of them focus on the solutions of small graphs representing high index construction costs and large index storage spaces.

Although the *2-hop labelling* method is considered an efficient way to solve the problem, there are still have several challenges. First, before the *Hop Doubling*, it can become hard to find a method that can provide a small label size. For the principles of index construction on *2-hop labelling*, the worst case of the total label size is $O(|V|^2)$. In general cases, any *2-hop labelling* indexes the total size of $w(|E||v|^{\frac{1}{2}})$ in $G(V,E)$. Therefore, the high complexity is impractical for large-scale networks. Second, before the work, we cannot find a good bounded complexity on the computing time and the runtime memory space required for the label index construction.

[60] provides bounds on the index size, the computation costs and I/O costs based on the properties of unweighted scale-free graphs. The method outperforms the other the-state-of-the-art techniques on both querying time and indexing time. The method reduces the increasing rate of label size at each iteration. Therefore, it results in highly effective labelling for the index. The main contributions can be summarised as follows: First, they propose a novel *2-hop labelling* indexing method for point-to-point distance querying on unweighted directed graphs and have developed I/O efficient algorithms for index construction when the given graph and the index cannot fit in the main memory. Second, in terms of the properties of unweighted scale-free graphs, they provide the complexity bounds for the index in different areas. In detail, the index size is $O(h|V|)$, the computational cost is

$O(|V|logM(|V|/M+log|V|))$, and the I/O cost is $O(|V|log|V|M\times|V|/B)$, where h is a small constant, M is the memory size and B is the disk block size. Third, according to the performance of the method on large real-world scale-free networks, they verify the efficiency on both query processing and index construction.

Tree Decomposition

In graph theory, a tree decomposition is a mapping of a graph into a tree structure. We can impose it to define the treewidth of the graph and speed up solving certain computational problems in graphs. We give the definition in the following:

Definition 2.5 (**Tree Decomposition**). *Given a graph $G=(V,E)$, a tree decomposition of G, denoted as T_G, is a rooted tree in which each node $X\in V_G$ is a subset of V_G. T_G must have the following tree properties [61]:*

1) $\bigcup_{X\in V_G}X=V$;

2) for each edge $(u,v)\in E$, *there is a node* $X\in V_G$ *such that vertices* $\{u,v\}\subseteq X$;

3) $\forall v\in V_G$, *and the set* $\{X|v\in X\}$ *forms a connected subtree of* T_G

We can have the following insights in terms of the properties above: First, every node in T_G must have at least one vertex, and every vertex in G can exist in more than one tree node. Moreover, arbitrary edge (u,v) can be embedded in one or more tree nodes but does not mean that any two vertices u, v in a tree node must be an edge. Besides, node X can act as a separator, if we remove vertices in X from its subtree resulting in the remaining vertices in the subtree being independent.

We can describe a tree decomposition by its treewidth and tree height. The definition is as follows:

Definition 2.6 (**Treewidth and Treeheight**). *Given a tree decomposition T_G, the treewidth of T_G is $max_{X\in T_G}|X|-1$. The tree height of T_G is the maximum depth from the leaves to the root of T_G.*

For instance, Figure 2.2 shows an example tree decomposition. The first vertex in each tree node is its representative vertex. This structure has a *Cut Property* that can be used to assign labels: $\forall s,t\in V$ and X_s,X_t are their corresponding tree nodes without an ancestor/descendent relation, and their cuts are in their *lowest common ancestor (LCA)* node [62]. For example, the tree nodes of v_9 and v_7 are X_9 and X_7, and their *LCA* is $X_1=\{v_1,v_5,v_8,v_{12}\}$. Then these four vertices form a cut between v_2 and v_7. If s is t's descendant, then t can be viewed as a cut between itself and s. Therefore, the *LCA* of a vertex pair contains all the cuts between them. Accordingly, we assign labels to a vertex with all its ancestors in T_G. It should be noted that the vertices in each tree node are also labelled because of the

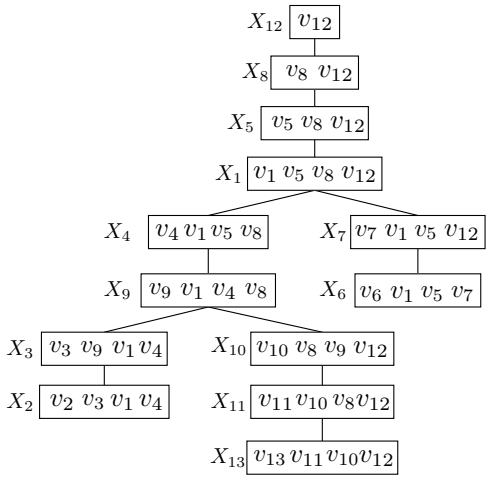


Figure 2.2: Tree Decomposition Example

Ancestor Property: $\forall X_v \in T_G, X_u (u \in X_v \backslash \{v\})$ is an ancestor of X_v in T_G. For example, v_6 has labels $\{v_7, v_1, v_5, v_8, v_{12}\} \supseteq \{v_1, v_5, v_7\}$.

The 2-hop labeling index [35] answers distance query without any search. *PLL* [33, 36] and *PSL* [63] are designed for *small-world* networks since several large degree vertices work as gateways to help reduce the search. *Tree-Decomposition*-based methods *H2H* [37] and *TEDI* [64] are suitable for road networks because of the low average degree and the small tree width. *CT-Index* [65] combines *H2H* and *PLL* to deal with the periphery and core of a graph separately.

2.2.3 Summary

In this section, we discuss the techniques for solving the shortest path problem. Compared with index-based methods, the index-free methods have worse performance on query processing, but they provide the basic ideas for the development of index-based methods. Moreover, index-free methods can solve the shortest path problem in dynamic networks because of their nature. On the contrary, index-based methods have to spend extra time on pre-processing for each change of networks. The lager index they generate, the longer time they spend on pre-processing. However, they can achieve better performance on query processing when the index size is increasing. Also, all the other types of path planning problem can be regarded as extended problems of the shortest path, such as constrained shortest path, time-dependent path planning and multi-criteria path planning.

2.3 Constrained Shortest Path

In the previous section, the path problems are related to one criterion, such as finding the path with the shortest distance, fastest travel time, or lowest fuel consumption. However, these corresponding

solutions will fail to consider when the route planning problem has over one criterion. For instance, in our daily lives, we may face various scenarios, such as a person may have a limited budget to pay the toll charge of highways or bridges; some mega-cities might require drivers to reduce the number of left turns in the right driving case to decrease the accident rate. Thus, in these cases, we may raise a question of how we can find the path as short as possible and meet the above requirements at the same time. To answer the question, we first give the definitions for such problems as follows:

Definition 2.7 (Bi-Dimensional Skyline Path Problem). *Given a graph $G(V,E)$, each $e \in E$ has a weight $w(e)$ and a cost $c(e)$. A path p from s to t has a weight $w(p)$ and a cost $c(p)$. p_i dominates p_j if $w(p_i) \leq w(p_j)$ and $c(p_i) \leq c(p_j)$, and the equal conditions do not hold at the same time. The bi-dimensional skyline path problem is to find a set of paths that are not dominated by other paths.*

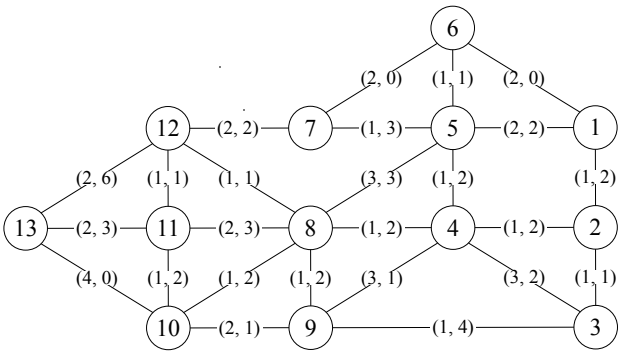


Figure 2.3: An Example of Skyline Path. Edge Labels are $(w(e), c(e))$.

In this thesis, we use $P(s,t)$ to denote the set of skyline paths from s to t. we denote a path p as a skyline path $\overline{p}$ if it is in a skyline path set. Consider the example in Figure 2.3. We enumerate four paths from v_5 to v_9:

1. $p_1 = \langle v_5, v_4, v_9 \rangle$, $w(p_1) = 4$, $c(p_1) = 3$
2. $p_2 = \langle v_5, v_8, v_9 \rangle$, $w(p_2) = 4$, $c(p_2) = 5$
3. $p_3 = \langle v_5, v_4, v_8, v_9 \rangle$, $w(p_3) = 3$, $c(p_3) = 6$
4. $p_4 = \langle v_5, v_8, v_4, v_9 \rangle$, $w(p_3) = 7$, $c(p_3) = 6$

According to the definition, we can identify that p_2 is dominated by p_1, and p_4 is dominated by the other three paths. Hence, p_2 and p_4 are not skyline paths. Meanwhile, as p_1 and p_3 cannot dominate each other, we obtain two skyline paths from v_5 to v_9: $P(v_5, v_9) = \{\overline{p}_1, \overline{p}_3\}$.

Given the results of a *Bi-Dimensional Skyline Path* query. Users can select the most suitable one from them. Also, we can make the query result become more user friendly by the *Constraint Shortest*

Path, which is regarded as a particular case of the *Bi-Dimensional Skyline Path*. We give its definition as follows.

Definition 2.8 (Constraint Shortest Path (CSP)). *Given a graph $G(V,E)$, each $e \in E$ has a weight $w(e)$ and a cost $c(e)$. A path p from s to t has a weight $w(p)$ and a cost $c(p)$. The constraint shortest path finds a path with the minimum $w(p)$ whilst its cost $c(p) \leq C$, where C is a pre-defined cost constraint.*

Refer to our work. We mainly introduce the solutions on CSP with *bi-criteria* in the following sections. Moreover, the CSP problem is the basis of the multi-criteria CSP. Moreover, answering such a CSP query, such as the algorithm in related work, is already a challenging task. Finally, combing a large number of criteria may lead to no feasible path, which will reduce the usability of the routing system.

2.3.1 Exact Index-Free Algorithms

The first work that studies the *CSP* problem is [66], which proposes a dynamic programming algorithm. [22] converts it to an *integer linear programming (ILP)* problem and solves it with a standard *ILP* solution. However, both approaches are not scalable to large real-life road networks. Amongst the practical solutions, the *Skyline Dijkstra* algorithm can be utilised to answer *CSP* queries by setting a constraint on the cost during the skyline search. In this way, the search space is further pruned as the CSP result is a subset of the skyline paths. Another practical solution to answer *CSP* is through *k-Path*. The straightforward way is testing the top-k paths one by one until one path satisfies the cost constraint. *Pulse* [67] improves this procedure by applying constraint pruning during the *k-Path* search. However, its search strategy follows a *DFS* manner, which limits its performance. *cKSP* [68] uses the pruned *Dijkstra* search in its k-path generation and incorporates the skyline dominance relation to prune the infeasible ones. *eKSP* [24] is the state-of-the-art approach that further replaces *cKSP*'s search with sub-path concatenation to reduce the cost of k-path generation.

Skyline-Dijkstra

The solution is named by *Skyline-Dijkstra* as it uses the idea of *Dijkstra's* in general. Compared with *Dijkstra's*, *Skyline-Dijkstra* stores a set of paths for each vertex rather than a single shortest path during the iterative search. To achieve it, *Skyline-Dijkstra* assigns a label set $L(v)$ to each vertex v. Initially, each $L(v)$ is set to empty and then updated for maintaining the current set of skyline paths from the start s to v in each interaction. *Skyline-Dijkstra* also organises the search by a priority queue Q. However, in this case, each element in Q has two criteria including weight and cost. Thus, Q can select either

weight or cost as its ranking order. For instance, we maintain the Q in ascending order of the costs of the elements. The principles of the search are shown as follows:

1) Initially, Q includes a *s*-*s* path p_0.

2) In each iteration, *Skyline-Dijkstra* pops the top path p , which has the minimum cost, from Q. Suppose v is the last vertex in p. We need to check whether v is equal to the destination t.

3) if$v \neq t$, it means that p has not reached the destination. The method enumerates each path p', which can be obtained by adding an edge (v, v') at the end of p. Afterwards, it checks the cost of p'.

4) If $c(p) > C$ or dominated by other paths in $L(v')$, discard p'. Otherwise, add p' to Q and $L(v')$ as well. Then, conduct validation of skyline paths in L(v') for eliminating the dominated paths.

5) It terminates when Q is empty. Then select the path with minimum weight in the label set $L(t)$.

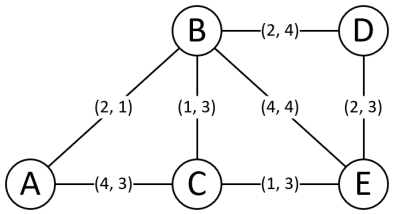


Figure 2.4: Skyline-Dijkstra Example

For instance, we implement the algorithm on the example road network which shown in Figure 2.4. Given a CSP query $q(A,E,6)$, where 6 is the cost constraint. *Skyline-Dijkstra* initialises Q with a *A*-*A* path p_0, which includes both zero cost and weight. Then, $p_0 =< A,A >$ is popped out. By checking the neighbours of A, we can get the path $p_1 =< A,B >$ with $w(p_1) = 2$ and $c(p_1) = 1$, the path $p_2 =< A,C >$ with $w(p_2) = 4$ and $c(p_2) = 3$. We assign p_1 to $L(B)$ and p_2 to $L(C)$. Consequently, we push p_1 and p_2 into Q. In the next round, p_1 is popped out from Q as $c(p_1)$, which is the minimum. We can obtain $p_4 =< A,B,C >$ with $w(p_4) = 3$ and $c(p_4) = 4$, $p_5 =< A,B,D >$ with $w(p_5) = 4$ and $c(p_5) = 5$, $p_6 =< A,B,E >$ with $w(p_6) = 6$ and $c(p_6) = 5$. To maintain the labels, we add p_4 to $L(C)$, p_5 to $L(D)$, and p_6 to $L(E)$. We push the new paths into Q except for p_6 as it has already reached E. According to the rules of Q, p_2 pops out and obtains $p_7 =< A,C,E >$ with $w(p_7) = 5$ and $c(p_7) = 6$. When Q becomes empty, we check $L(E)$ and find out that p_7 is the optimal result.

The time complexity of query processing is $O(w_{max}|V||E|log(w_{max}|V|))$, where V is the vertex number, E is the edge number and w_{max} is the longest shortest path in the network. Therefore, the algorithm is very time-consuming in a large-scale road network.

Yen's Algorithm

The simplest way to extend the top-k shortest path problem on CSP is to test the top-k paths one by one and then find one path that satisfies the cost constraint. The well-known algorithm for solving the problem is *Yen's Algorithm* [69]. It first computes the most shortest-path from the start s to the destination t as the first path. Afterwards, it discovers the shortest path p at each vertex as the deviation node to generate candidates. These new paths are used in the next single-source shortest path discovery. Amongst the candidates, it selects the path with minimal cost and repeats the discovery processing. It terminates when k different shortest paths are identified. The time complexity of $Yen's$ algorithm can reach $O(kn(m+nlogn))$. For each of the k shortest paths, it spends $O(n)$ on the single-source shortest path discovery, where n is the number of nodes and m is the number of edges.

cKSP

Compared with $Yen's$ algorithm, although *cKSP* cannot reduce the time complexity $O(kn(m+nlogn))$ in the worst case, it handles the reduction of each factor. The fundamental point is to avoid the redundant computation cost of each candidate path generation by pre-computing the shortest paths to the destination t.

cKSP first pre-computes the shortest path tree rooted at the target node. In addition, to speed up the loop detection in candidate paths, it assigns interval encodings on each tree node. Afterwards, it generates a transformed graph constructed by annotating each edge e with side cost. The processing guarantees an earlier termination than previous algorithms in the shortest path discovery. In the candidates' generation, instead of *Dijkstra*- based search on the whole graph, it generates each shortest path by concatenating two sub-paths. As the second sub-path to t is already located in the pre-computed shortest-path tree, it only needs to spend time on the first sub-path discovery.

Furthermore, *cKSP* provides two pruning techniques to guarantee an earlier termination. When the number of paths in the candidate set reaches k, the iterations can stop even though the iteration time is less than k. Moreover, it sets a threshold $cost_{est}(k)$ to ensure that the current candidate path discovering can stop when its cost exceeds $cost_{est}(k)$, where $cost_{est}(k)$ is an estimated cost for the k^{th} shortest path. The adaptation of such estimation is handled by the polices, including *lazy* and *eager*.

The extensive experiment shows that *cKSP* has higher efficiency and better scalability than $Yen's$ and the replacement version [70]. Moreover, *cKSP* can be over two orders of magnitude faster than *Yen's* in large-scale networks.

eKSP

eKSP takes advantage of the binary partition strategy of a recent k-shortest path algorithm [67, 71], which admits posing numerous extra path constraints in the networks without other difficulties.

The algorithm provides several pruning techniques to enhance the efficiency of k-shortest path search. According to extensive experiments, *eKSP* is the method that outperforms the existing k-shortest path algorithms. The pruning techniques include fixed cycles, dominance, and infeasibility, which are discussed as follows.

On the pruning of fixed cycles, the method chooses to search for the paths without cycles as the optimal CSP is a basic path. When a path contains a loop, the binary partition strategy can selectively scan a path, including a loop and determine when to identify simple or non-simple new paths. For instance, if a fixed sub-path containing a loop is found in a new path, the algorithm stops to exploring the region covering the path. Nevertheless, if a fixed sub-path containing a loop is in non-simple paths, the algorithm needs to explore the paths by the principles of binary partition strategy. Therefore, the method scans the entire path to find a loop once a new path is found. Thus, there are three situations that exist: first, it is the simple path; second, the path contains a loop, which is not totally fixed; third, the path contains a loop in the fixed sub-path. The algorithm can only identify a potential new path based on the current path in the previous two situations. The algorithm does not search the region in the last situation as discussed.

On the pruning of dominance, in each iteration of the algorithm using the binary partition strategy, it can determine the paths containing a fixed sub-path from several vertices that belong to the s-t path to destination vertex t. Suppose there are two determined paths P_{it}^{k} and P_{it}^{p}, where P_{it}^{k} denotes the k^{th} path containing a fixed sub-path from vertex i to vertex t and is already determined by the algorithm. Similarly, P_{it}^{p} denotes the p^{th} path, and $p < k$. Thus, the sub-paths $P_{si}^{k} = P_{si}^{p}$ are equal to $P_{si}(T*)$, which means the path from s to t in the shortest path tree $T*$. Moreover, $c(P_{it}^{p}) \leq c(P_{it}^{k})$ as the optimal distance of p less or equal to the optimal distance of k. In the case, the sub-path P_{it}^{p} strongly dominates the sub-path P_{it}^{k} when $w(P_{it}^{p}) \leq w(P_{it}^{k})$.

It exits an optimal solution of the CSP problem and is not strongly dominated by any other path from vertex i to j as the resource consumption values are non-negative [72]. Therefore, the algorithm only scans paths containing fixed non-dominated sub-paths. Moreover, for each vertex i, the algorithm keeps a first non-dominated label in the shortest path tree. This label is overwritten every time when a partially fixed path from i to t shows lower resource cost than the one saved. Otherwise, the second non-dominated label in the tree needs to be replaced.

The last pruning is on infeasibility. It aims to prune the paths that cannot meet the resource constraint. With the help of the binary partition strategy, if the resource consumption in the sub-path from i to t in the candidate path plus the minimum resource consumption in the optimal path from s to i is greater than the resource constraint, the algorithm will discard this path, which means that there are no feasible paths that meet the resource constraint to be found from this path.

2.3.2 Approximate Index-Free Algorithms

The approximate *CSP* allows the resulting length to be no longer than $(1+\alpha)$ times the optimal path. The first approximate solution is also proposed by [20] with a complexity of $O(|E|^2 \frac{|V|^2}{\alpha-1} \log \frac{|V|}{\alpha-1})$. [28] improves it to $O(|V||E|(\log\log \frac{w_{max}}{w_{min}} + \frac{1}{\alpha-1}))$. *Lagrange Relaxation* is also applied to obtain the approximation result [22, 27, 73] with $O(|E|\log^3|E|)$ iterations of path finding. As shown in [29] and [67], these approximate algorithms are even orders of magnitude slower than the *k-Path*-based exact algorithms. *CP-CSP* [13] approximates the *Skyline Dijkstra* by distributing the approximation power $\sqrt{[}n]\alpha$ to the edges. However, this approximation ratio would decrease to nearly 1 when the graph is big, so its performance is only slightly better than *Sky-Dijk* when the graph is small.

Lagrange Relaxation

Lagrange relaxation is the earliest-used index-free method. It applies the idea of mathematical optimisation and approximates the problem of constrained optimisation by a simpler problem. Therefore, *Lagrangian relaxation*-based methods [22, 74–76] are the approximate solutions to the original problem and provide useful information.

The main points of the method include dualising the side constraints into the objection function. In terms of Lagrangian duality theory, the original problem can be extended to a dual Lagrangian problem. The dual Lagrangian problem provides the corresponding optimal value as the lower bound of the fundamental problem. In other words, it is a problem of maximising the Lagrangian function of the dual variables. The advantage of *Lagrangian relaxation*, it can be still useful in a large-scale network. The disadvantage is that it gets near-optimal solutions that cannot satisfy the accuracy requirements.

[22] first proposes a *Lagrangian relaxation*-based method to answer CSP queries and return near-optimal solutions. [73] proposes a *Lagrangian relaxation*-based method, which is combined with enumerating near-shortest paths to deal with the CSP problem. [77] solves the dual Lagrangian problem by iteratively eliminating edges and vertices. [74] focus on a stochastic constrained shortest path problem with an integrated algorithm based on *Lagrangian relaxation*.

2.3.3 Exact Index-Based Algorithms

There are many shortest path indexes, but only *CH* [31] has been extended to *CSP* [30]. Because the query constraint can be arbitrary, the *CSP* index has to capture all the possible solutions. Hence, its shortcut is a set of *skyline paths* instead of one shortest path. Consequently, its construction and query answering are both *Sky-Dijk*-based, resulting in long index construction, large index size, and slow query answering. Hence, *CSP-CH* only keeps a limited number of skyline paths in each shortcut to reduce the construction time, but its query performance suffers.

CSP-CH

CSP-CH imposes the *Dijkstra*-based speedup techniques to improve the performance of CSP query and reduce the space consumption on index construction. Moreover, *CSP-CH* proposes several pruning techniques for shortcut generation. It results in a trade-off between the pre-processing time and graph sparseness.

In the phase of node order, to achieve a sparse graph after contracting, *CSP-CH* imposes the edge-difference as the heuristic to produce an efficient node ordering. The method to compute the edge-different of a vertex v uses the number of added shortcuts minus the edges removed when contracting v. Moreover, the algorithm also employs a weighted version of edge-difference [57] to consider the distance/weight-optimal paths with low total cost.

In the phase of index construction, *CSP-CH* takes advantage of skyline paths. Therefore, the number of shortcuts between a pair of vertices in *CSP-CH* can be multiple, rather than single in *CH*. Also, as every sub-path of a skyline path must be a skyline path [30], the algorithm needs to maintain all skyline paths on a local level to ensure that the *CH*-graph can be used to answer arbitrary CSP query correctly. When contracting each vertex v, the algorithm does not need to add a shortcut two of its neighbours u and w, iff there is an alternative path from u to w that dominates the path $p =< u, v, w >$. Such a path is called a dominating witness.

The naive method to find a series of dominating witnesses uses *Skyline-Dijkstra*. For each *Skyline-Dijkstra* search between two neighbour vertices u and w, p us a skyline path from u to w, the shortcut $sc = (u,w)$ has $w(sc) = w(u,v) + w(v,u)$ and $c(sc) = c(u,v) + c(v,w)$. Once w pops out from the priority queue in the search, the search will stop and check if $w(sr)$ exceeds $w(p)$. If so, the shortcut is added. Otherwise, it means that the search has already found a dominating path, and the shortcut needs to be discarded. However, implementing the *Skyline-Dijkstra* search may result in an extremely long time of index construction. Based on the naive method, *CSP-CH* proposes a technique to avoid partial computations on the generation of shortcuts.

CSP-CH first limits the shortcuts on the *lower convex hull(LCH)* [78] of all paths. If a skyline path p is a part of LCH, it means that we cannot find a dominating path and $sc = (u,w)$ needs to be inserted. On the contrary, if there exists a dominating path p', p' can be regarded as a part of LCH. To generate candidate paths on the LCH, the algorithm sets a parameter $\lambda \in [0,1]$, which combines the two criteria into a single criterion by $\lambda w(e) + (1-\lambda)c(e)$. The value of the new criterion is assigned to each edge and generates a new graph G^{λ}. As the original problem becomes the point-to-point shortest path problem in G^{λ}, the conventional *Dijkstra*-based algorithms can be easily implemented to find an optimal path p'. By evaluating p' in G, the algorithms will check the dominance relation between p and p'. If p' cannot dominate p, the algorithm tests another value of λ in the range.

For restricting the time of updating λ, the algorithm provides a heuristic. It starts with $\lambda_1 = 0$

and $\lambda_2 = 1$. By using these two values in the equation for computing the new criterion, we obtain the cost-minimal path p_1 and the weight-minimal path p_2. If the dominating path cannot be found, the algorithm further tests $\lambda_3 = \frac{c(p_2)-c(p_1)}{w(p_1)-w(p_2)+c(p_2)-c(p_1)}$. By the λ_3, the algorithm can simply find a new shortest path p_3 in the updated graph G^{λ}. Thus, p_3 may be as same as p_1 or p_2. If so, the method has already searched the entire *LCH*. Otherwise, p_3 includes an intersection point with both p_1 and p_2. These two intersection points are regarded as candidates for the support points. Actually, such points are vertices in the original graph and are pushed into a priority queue Q. Hence, the algorithm pops out the top λ in Q and repeats the previous processing. The search is terminated when Q is empty. In this case, the entire *LCH* has been checked.

The final CH graph G' will contain all vertices and edges in the original and all the shortcuts added by the heuristic search method above. To answer CSP queries, it imposes a bi-directional *Skyline Dijkstra* search from both the start and the target vertex in G' for computing the shortest path satisfying the given constraint cost.

2.3.4 Approximate Index-Based Algorithms

The only approximate index method is *COLA* [38], which partitions the graph into regions and creates approximate skyline labels for this 3-hop structure. Similar to *CP-CSP*, the approximate labels are also constructed in the *Sky-Dijk* fashion. In order to avoid the diminishing approximation power $\sqrt{\lceil n \rceil |V|}$, it views α as a budget and concentrates it on the important vertices that are associated with a large number of paths. A similar pruning technique is also applied in the time-dependent pathfinding [4]. In this way, the approximation power is released, so it has a faster construction time, a smaller index size, and a faster query answering time.

COLA

[74] proposes *COLA*, which is the first practical solution for answering CSP queries with approximate results in large-scale networks. In general, *COLA* provides a method to divide the road network into several partitions efficiently. Then, it generates an overlay graph on top of the partitions, which has a smaller size than the original graph. As the index construct is conducted on the smaller graph, it results in better performance on query processing.

On the section of overlay graph, the method partitions G into a set of edge-disjoint sub-graphs $T = \{G_1, G_2, .., G_i\}$. Therefore, the union of all G_i is equal to G. Once a sub-graph is partitioned, there exist several boundary vertexes. A vertex v is defined as a boundary vertex if v is included in more than one sub-graphs. There are several feasible strategies in [79–81] to build an overlay graph

An overlay graph can compress the input graph G' by the strategies as follows: First, G' only stores the boundary vertices of the sub-graphs. Second, paths in G are represented by edges in G'. In detail,

for each edge e' in G' that starts at vertex v and ends at vertex v', it must have a path p from v to v' in G. Thus, $w(p) = w(e')$ and $c(p) = c(e')$. Note that v and v' are neighbours in G', but such a spatial relation is not necessary in G. Third, it can reduce several edges in G' by removing the corresponding paths that are dominated by other paths in G. We can adopt different ways to build an overlay graph, as the partitioning of G can be varying.

After pre-computing the overlay graph, which has a smaller size than G, it can construct a low-cost index and improve the performance on CSP query processing. In details, to answer a CSP query $q(s,t,C)$, the algorithm first identifies the sub-graphs that contain s and t, which are denoted as G_s and G_t. Afterwards, it generates an extended graph G^q by merging G_s, G_t, and G'. The query processing of *COLA* is implemented on G^p.

The index construction is conducted in each overlay graph $G' = (V', E')$. The algorithm assigns an in-label set and an out-label set for each vertex $v' \in G'$, which is denoted as $L_{in}(v')$ and $L_{out}(v')$. *COLA* provides the pruning strategies on the labels of v', as follows: First, each entry in $L_{in}(v')$ has a corresponding path from another vertex in G' to v'; Second, each entry in $L_{out}(v')$ has a corresponding path from v' to another vertex in G'; Third, given an arbitrary OD pair $s', t' \in V'$, for any path P from s' to t', the algorithm can generate the index containing an out-label in $L_{out}(s')$ with weight w'_0 and cost c'_0 and an in-label in $L_{in}(t')$ with weight w'_i and cost c'_i. Such index need to satisfy the equation $w'_o + w'_i \leq w(P)$ and $c'_0 + c'_i \leq \alpha \times c(P)$, where α is the approximate ratio.

In query processing, *COLA* uses the method called α-*Dijk* for computing skyline paths. The search of α-*Dijk* is mainly used in intra-partition search and index construction of *COLA*. The search of α-*Dijk* is similar to *Skyline Dijkstra* but provides the pruning strategy with the relaxed α-dominance, which is defined in [74]. The query processing concentrates the pruning power on the 'important' vertices. These vertices usually are included in a large number of paths. Such vertices usually are treated as landmarks in a real road network. Concentrating the pruning power on these vertices can effectively reduce the total number of paths during the search. Thus, the performance of query processing can be improved.

Summary

Compared with the shortest path problem, the constrained shortest path problem is more related to real life, as it considers extra criteria in path planning, which can adapt to a more complex environment. In the current, only a small number of works dig into this problem. Most of them apply the *Dijkstra*-based search in different phases. We notice that such online search has a significant effect on query performance. Therefore, it provides a chance for us to improve the query efficiency, and avoiding *Dijkstra*-based search is one feasible way. Moreover, achieving better performance on exact query answering than current approximate approaches is a significant contribution.

2.4 Multi-Dimensional Skyline Path

Definition 2.9. ***Multi-Dimensional Skyline Path Problem*** *Given a graph $G(V,E)$, each $e_i \in E$ has a d dimensional value vector$(e_i^0,...,e_i^{d-1})$. A path p_i from s to t also have a d-dimensional value vector $p_i = (p_i^0,...,p_i^{d-1})$, where each p_i^j is the sum of all path edge values from the same dimensional. For any two paths p_i and p_j, we say p_i dominates p_j if $p_i \neq p_j$ and $p_i^k \leq p_j^k, \forall k \in [0,d-1]$. The multi-dimensional skyline problem is to find a set of paths that cannot be dominated by other paths.*

The methods of the Multi-Dimensional Skyline Path Problem can be classified into scalar and Pareto methods. In general, the scalar method uses a weighted objective function [10]. The idea of this approach is to transform the problem to a single-objective problem by existing heuristics [9]. However, the drawback is that such methods are not easy to achieve coherence on a set of criteria [82]. On the other side, the Pareto method uses the idea of Pareto dominance, which is defined in the previous **Multi-Dimensional Skyline Path Problem**. The exact ways for finding the entire set of Pareto-optimal paths can be divided into labelling and ranking algorithms. The basic idea of labelling has been discussed in the previous section. For a more user-friendly method, *Multi-Constraint Shortest Path(MCSP)* is proposed and regarded as a particular case of *Multi-Dimensional Skyline Path*.

Definition 2.10 (**Multi-Constraint Shortest Path**). *Given a n-dimensional graph $G(V,E)$, a MCSP query $q(s,t,\overline{C})$ returns a path with the minimum $w(p)$ whilst each $c_i(p) \leq C_i \in \overline{C}$, where $i \in [1,|\overline{C}|]$.*

It is first studied by [20], but can only focus on bi-objective. The extensions for solving it can be found in [83–85]. The main idea of ranking algorithms is to produce a set of the k-shortest path first and then to discard the paths dominated by other criteria in the path set. Note that the fundamental solutions on solving *MCSP* are similar to the methods on Bi-Dimentional Skyline Path Problem, which is a specific case of *MCSP*. It is an easy extension for solving *MCSP* on a given pair of a source and a destination. However, there are many more challenges in answering an arbitrary OD pair, as the skyline paths will increase dramatically when the number of criteria increases. In the following section, we will introduce several existing solutions to *MCSP*.

2.4.1 ARSC

ARSC is proposed in [17], and it solves the *MCSP* problem the queries that are not given in networks previously. It means that a set of potentially non-dominated paths have to be produced first and then the users need to select an appropriate one amongst them. Thus, the problem is highly dynamic. Moreover, it is impractically to pre-compute all possible solutions as similar to the bi-objective case, as a large number of skyline paths will dramatically increase and exceed a reasonable space limitation. *ARSC* solves the problem by dynamically searching paths and pruning the paths that are not able to

be extended into elements of the skyline as fast as possible [17]. The algorithm proposes a lower bound forward estimation for each optimisation criterion. Once another path to the same destination dominates the estimation, the algorithm will stop searching the path. Given a multi-attribute graph G, it applies a vector to store different optimisation criteria for each edge. The algorithm first conducts skyline processing on such feature vectors and then uses the estimation above for pruning. Note that the current stage can only prune the s-t path p if there is an already-known s-t path p' which dominates the forward estimation of p. Therefore, the current stage may produce a large computation as every path that has a chance to be extended into a skyline path must be maintained and processed. Hence, the algorithm further proposes a second local pruning criterion in terms of the observation that any sub-path of a skyline path can also be the Pareto optimal. In detail, especially when given a query $q(s,t)$ for answering the skyline paths from s to t. The search has currently found the path $p' = (s,..,v_i)$. Suppose p' is not a skyline path from s to v_i, the algorithm will stop exploring the expansion of p', as it cannot be the sub-path of a skyline path from s to t.

For checking whether p' can be excluded as the dominated path from s to v_i during the search, it is expensive to generate the complete path set $P(s,v_i)$. Thus, the algorithm only checks the dominance amongst paths that have already been generated during the skyline path search processing. For each path, $p = (s,..,v_i)$ that is found, the algorithm computes the value of all criteria and combines them in an attribute vector $p.attr[]$. Afterwards, it stores the vector and the corresponding path p in a list $v_i.L$ for the vertex v_i, if p cannot be dominated by the paths in $v_i.L$.

The algorithm requires a priority queue Q to maintain each vertex v stored in it. Each v also stores its sub-path skyline in a list $v.Sub$. This list has skyline paths for all searches ending at vertex v. In Q, the vertices are sorted in ascending order in terms of a given score function. It determines that v_i is prior to v_j iff there exists at least one sub-path that owns a higher score than each sub-path in $v_i.Sub$.

Also, the algorithm maintains a global list S_l to organise all paths as candidates for skyline paths. Initially, it inserts the start s into Q. In the current, the sub-path skyline of s only has one path, including one vertex s. Q will pop out the top vertex v_i each time. For each sub-path p in $v_i.Sub$, the algorithm will check whether another path from the same destination dominates p. If not, p can lead to a skyline path in further search. Therefore, the algorithm uses forward estimation to compute the lower bounding attribute vector $p.lb[] = p.attr[] + (D_1(v_i,t),...,D_d(v_i,t))^T$, where $D_1(v_i,t)$ is the distance estimation to lower bound the distance between v_i and t in terms of the attribute $l \in (1,...,d)$ by means of the reference vertex embedding which can be found in [86]. The lower bounding vector can be used to estimate the attribute vector of all paths from s to t, including sub-path p. Afterwards, the algorithm will identify whether $p.lb[]$ is dominated by any found paths in S_l. If so, p can be pruned to stop further searching based on p. All the sub-paths $p \in v_i.Sub$ that are not dominated are expanded by one hop in their direction. The already generated sub-paths are marked with processed flags. For each expanding sub-path p by moving one hop to vertex v_j, if v_j is equal to the destination $t$efficiently , S_l obtains

a completely explored to t and updates itself. In this case, if p is dominated by any path in S_l, p is pruned. If not, p is inserted in S_l. If p dominates any path in S_l, the dominated path is removed from S_l. On the other hand, if v_j is not equal to t, the algorithm checks whether p is dominated by each sub-path $p' \in v_j.Sub$ before inserting p in $v_j.Sub$. If so, p is pruned and discarded directly. On the contrary, p' is removed from $v_j.Sub$ if it is dominated by p. If p is a skyline path to v_j, it is inserted in $v_j.Sub$. Finally, the algorithm updates Q if the current $v_j.Sub$ results in a new smaller score which is computed by the score function.

2.4.2 Branch-and-Bound Skyline (BBS)

The *Nearest Neighbours (NN)* algorithm uses the divide-and-conquer framework on datasets indexed by R-trees [87]. It can answer the initial skyline points efficiently and adapt to varying data distributions and dimensions. However, it needs duplicate elimination once the number of dimensions d is greater than 2. Also, several tree nodes have to be visited multiple times, and a large space is occupied. Based on the case, [88] proposes *BBS*, which is a progressive algorithm based on *NN* search. This algorithm can perform unique access to the R-tree nodes that may have skyline points. Moreover, it significantly reduces the space consumption and only accesses a node one time.

Similar to the *NN* algorithm [89, 90] and the algorithm for convex hulls [91], BBS is the method based on the branch-and-bound paradigm. Specifically, it searches from the root of the R-tree. All the entries are organised by a heap and sorted with the increasing order of their shortest distances. The heap will iteratively pop out the entry which has the minimum distance. The algorithm expands the pooped entry so that its children can be inserted each time. The search of BBS is similar to the best-first *NN* algorithm of [90].

2.4.3 High Dimensional Skyline

Existing skyline computation is executed in either full space or multidimensional subspaces. Specifically, *Skyey* [92] is a sorting-based skyline algorithm, which computes the cubes from the full space and utilizes a depth-first manner to search skylines on each non-empty subspace. But it can hardly extend to high dimensional or large-scale graphs. *Stellar* [93] computes the skyline cubes from the full space, but can avoid looking through all the subspaces. It defines the seed skyline groups and the corresponding seed lattices to shape the skyline groups and their decisive subspace. However, the performance of *Stellar* will go worse if the number of seed skyline groups is high, because it must test the domination in each seed skyline group for the points that are not a full space skyline. In practice, the full space skylines only account for very small part in the final skyline groups. We can easily find that each skyline group may has very few points and still has large amount points need to be tested on the domination. Orion [94] reduces the domination tests significantly by deriving the skylines in

higher subspaces from their subspace skylines. In terms of the derivation rules they propose, it defines two types of skylines that can be identified without domination. However, it still need to spend high cost of domination test on finding the skylines out of the derivation rules. In summary, it is difficult to extend the existing skylines techniques to our *MCSP* query because we need to consider about the concatenation method between skyline cubes and the efficiency on finding the concatenated skyline cube.

Summary

Note that the current approaches to multi-criteria shortest-path problems depend on the concept of skylines. On the query processing, the more criteria the algorithms consider, the longer time they spend on skyline dominance. For index-based algorithms, they have to generate a huge index even though the index size is minimal. Therefore, several algorithms, such as *CSP-CH*,*eKSP*, and *COLA*, may answer queries efficiently in the bi-criteria shortest-path problems. However, they are impractical to be extended into multi-criteria shortest-path problems as the complexity of skylines.

2.5 Comparative Analysis of CSP Algorithms

This section compares the *CSP* algorithms in Table 2.5. Specifically, we can see the exact *CSP* solutions in this table. The previous three are index-free based. *Sky-Dijkstra* by pruning over the constraint is the labelling algorithm based on dynamic programming. *cKSP* and *eKSP* are k-path enumerations by testing the paths individually. However, the query processing of index-free-based methods is too slow in practice. A few solutions are index-based, which have better performance on query processing. For example, *CH* is the shortest path index extended to solve *CSP* with its shortcuts being skyline paths. However, it suffers from long index construction time and large index size.

On the approximate solutions, Lagrange Relaxation is the earliest used approximate index-free method. It applies the idea of mathematical optimization and approximates the constrained optimisation problem by a simpler problem. Because of its high complexity, it is even much slower than the k-path-based exact algorithms. *CP-CSP* approximates the skyline Dijkstra by distributing the approximation power to the edges. However, its approximation ratio will decrease to nearly 1 when the graph is big. The performance is only slightly better than *Skyline Dijkstra*. *COLA* partitions the graph into regions and precomputes approximate skyline paths between regions. Similar to *CP-CSP*, the approximate paths are also computed in the *Skyline Dijkstra* fashion. To avoid diminishing approximation power, it views α as a budget and concentrates it on the important vertices. It can solve exact *CSP* as well when we set α to 0, but the complexity of the exact version is similar to *Skyline Dijkstra*.

Table 2.1: Comparison of Path Algorithms

Algorithm	**Description**	**Remarks**
Exact Algorithms		
Sky-Dijkstra	Dijkstra's path search.	Computationally intensive.
cKSP	Pruned Dijkstra for k-path generation with skyline dominance.	Many paths for distant OD pairs.
eKSP	k-Path with sub-path concatenation.	Efficient than cKSP.
CSP-CH	SP index for constrained path. Uses Dijkstra.	Large shortcuts; slow queries.
Approximate Algorithms		
Lagrange Relaxation	Simplifies constrained optimization.	Slower than k-path exact methods.
CP-CSP	Approximated Skyline Dijkstra using $\alpha^{1/n}$.	Approximation ratio will decrease to nearly 1 when the graph is big.
COLA	Partition-based; precomputed skylines.	Efficient queries; exact for $\alpha = 0$.

Remark

Based on our literature review, the issue mainly affecting the efficiency of *CSP* query answering is skyline path search. Some index-free methods may spend hours answering *CSP* queries in large-scale road networks, even for the most straightforward version *CSP* queries. The index-based methods, such as *CSP-CH* and *COLA*, use some heuristics or hierarchical structures. However, their querying methods are still based on online skyline path search, leading to visiting a large number of vertices and conducting skyline dominance verification frequently, which limits their query performance. Motivated by these challenges, we introduce a hop-based *CSP* index in the following Chapter to bypass the costly skyline path search, thereby ensuring efficient *CSP* query response and swift index construction.

The following publication has been incorporated as Chapter 3.

1.**Ziyi Liu**, Lei Li, Mengxuan Zhang, Wen Hua, Pingfu Chao, and Xiaofang Zhou, Efficient Constrained Shortest Path Query Answering with Forest Hop Labeling, *IEEE 37th International Conference on Data Engineering (ICDE)*, pp. 1763-1774, 2021

Contributor	Statement of contribution	%
Ziyi Liu	Conception and design	60
	Analysis and interpretation	60
	Drafting and production	70
	proof-reading	50
Lei Li	Conception and design	15
	Analysis and interpretation	20
	Drafting and production	30
	supervision, guidance	40
	proof-reading	20
Mengxuan Zhang	Analysis and interpretation	5
	Drafting and production	10
	proof-reading	5
Wen Hua	Conception and design	5
	supervision, guidance	25
	proof-reading	10
Pingfu Chao	Conception and design	5
	proof-reading	10
Xiaofang Zhou	Conception and design	10
	Analysis and interpretation	15
	supervision, guidance	35
	proof-reading	5

2.Xuanyi Zhang, **Ziyi Liu**, Mengxuan Zhang and Lei Li, An Experimental Study on Exact Multi-constraint Shortest Path Finding, *Australasian Database Conference (ADC)*, Springer, pp. 166–179, 2021

Contributor	Statement of contribution	%
Xuanyi Zhang	Conception and design	50
	Analysis and interpretation	50
	Drafting and production	50
	proof-reading	50
Ziyi Liu	Conception and design	30
	Analysis and interpretation	15
	Drafting and production	30
	proof-reading	10
Mengxuan Zhang	Analysis and interpretation	15
	Drafting and production	10
	supervision, guidance	20
	proof-reading	20
Lei Li	Conception and design	20
	Analysis and interpretation	20
	Drafting and production	10
	supervision, guidance	80
	proof-reading	20

3. **Ziyi Liu**, Lei Li, Mengxuan Zhang, Wen Hua, and Xiaofang Zhou, Multi-constraint shortest path using forest hop labeling, *The VLDB Journal (2022)*, Springer, pp. 1-27, 2022

Contributor	Statement of contribution	%
Ziyi Liu	Conception and design	60
	Analysis and interpretation	60
	Drafting and production	60
	proof-reading	50
Lei Li	Conception and design	15
	Analysis and interpretation	20
	Drafting and production	30
	supervision, guidance	50
	proof-reading	20
Mengxuan Zhang	Conception and design	5
	Analysis and interpretation	5
	Drafting and production	10
	proof-reading	10
Wen Hua	Conception and design	10
	supervision, guidance	35
	proof-reading	20
Xiaofang Zhou	Conception and design	10
	Analysis and interpretation	15
	supervision, guidance	15

Chapter 3

Multi-Constraint Shortest Path Querying

The *Multi-Constraint Shortest Path (MCSP)* problem aims to find the shortest path between two nodes in a network subject to a given constraint set. It is typically processed as a *skyline path* problem. However, the number of intermediate skyline paths becomes larger as the network size increases and the constraint number grows, which brings about the dramatic growth of computational cost and further makes the existing index-based methods hardly capable of obtaining the complete exact results. In this paper, we propose a novel high-dimensional skyline path concatenation method to avoid the expensive skyline path search, which supports the efficient construction of the hop labelling index for *MCSP* queries. Specifically, insightful observations and techniques are proposed to improve the efficiency of concatenating two skyline path sets. An n-Cube technique is designed for pruning the concatenation space amongst multiple hops. A *constraint pruning* method is used to avoid unnecessary computation. Furthermore, we propose a novel forest hop labelling enabling parallel label construction from different network partitions to scale up to larger networks. Our approach is the first method to achieve accuracy and efficiency for *MCSP* queries. Extensive experiments on real-life road networks demonstrate the superiority of our method over state-of-the-art solutions.

3.1 Introduction

Route planning is an important application in our daily life, and various path problems have been identified and studied in the past decades. In real-life route planning, multiple criteria depend on users' preferences. For example, a user may have a limited budget to pay the toll charge of highways, bridges, tunnels, or congestion. Some mega-cities require drivers to reduce the number of big turns (e.g., left turns in the right driving case[1] as they have a higher chance to cause accidents). There could

[1] https://www.foxnews.com/auto/new-york-city-to-google-reduce-the-number-of-left-turns-in-maps-navigation-directions

be other side-criteria such as the minimum height of tunnels, the maximum capacity of roads, the maximum gradient of slopes, the total elevation increase, the length of tourist drives, the number of transportation changes, etc. However, most of the existing algorithms only optimise one objective rather than the other side-criteria, such as minimum distance [1–3], travel time of driving [4, 5] or public transportation [95, 96], fuel consumption [6, 7], battery usage [8], etc. In fact, it is these criteria and their combination that endow different routes with diversified meanings and satisfy users' flexible needs. *TollGuru*[2] is a routing application that considers travel time, toll charge, and fuel consumption together. In other words, this *multi-criteria* route planning is more generalised, whilst the traditional *shortest path* is only a special case. Moreover, looking for an optimal path based on *multi-criteria* information is an important research problem in the fields of both transportation [9, 10] and communication (*Quality-of-Service (QoS)* constraint routing) [11–15]. It is defined as *MCSP* in transportation, which inherits its definition from *Multi-Constrained Optimal Path(MCOP)* in *QoS* routing [11].

The criteria can be classified into *additive criteria* and *non-additive criteria*. For additive criteria, the constraints toward a path act on the sum of the corresponding criterion value of edges/links along the path, such as travel time and fuel consumption in transportation, or delay time and administrative weight in *QoS*. On the other hand, non-additive criteria consider the constraints at edges for determining a path via them or not, such as the minimum height of tunnels in transportation or bandwidth in *QoS*. We can easily solve the *MCSP* problem with non-additive criteria by pruning all edges that do not meet the constraints [16], but the *MCSP* with additive criteria is more complicated and deserves more effort in research. In fact, nearly all the related works only focus on the additive criteria. Therefore, we focus on the *additive criteria* and state the problem formally in Section 4.2.

However, we could hardly find one result that is optimal in every criterion when there is more than one optimisation goal. One way to implement the multi-criteria route planning is through path/route skyline [17–19], which provides a set of paths that cannot dominate each other in all criteria. However, skyline paths computation is very time-consuming with time complexity $O(c_{max}^{n-1} \times |V| \times (|V| \log |V| + |E|))$, where $|V|$ is the vertex number, $|E|$ is the edge number, c_{max} is the largest criteria value, and n is the number of criteria [20] (c_{max}^{n-1} is the worst case skyline number). In addition, it is impractical to provide users with a large set of potential paths to let them choose from. Therefore, the *Multi-Constraint Shortest Path (MCSP)*, as another way to consider the multiple criteria, is widely applied and studied. Specifically, it finds the best path based on one objective whilst requiring other criteria satisfying some predefined constraints. For example, suppose we have three objectives such as distance d and costs c_1, c_2, and we set the maximum constraints C_1 and C_2 on the corresponding costs. Then this *MCSP* problem finds the shortest path p whose cost $c_1(p)$ is no larger than C_1 and $c_2(p)$ is no larger than C_2. In the following, we show some examples of the *MCSP* queries:

[2]https://tollguru.com

Example 1: A tourist travelling from one place to another wants to find the fastest path that takes less than $10 in toll charges and travels through at least 20km of tourist drive. Whilst the toll charge is easy to understand, the length of tourist drive [3] is very important for a tourist's experience. Still, it fails to be captured by the single-criteria routing problem. For instance, a road by the beach may be longer and slower than a highway parallel to it, but it is more attractive for a tourist than for a commuter.

Example 2: A truck driver wants to plan a route from one deposit location to another with the lowest fuel consumption but has to arrive within 10 hours, make at least four places to stop for breaks and fuel, and keep toll charge under $20. As fuel consumption dominates the logistic company's operation cost, sacrificing the travel time is acceptable as long as the items are delivered on time. Besides, the rest stops are mandatory for long trips.

Example 3: The network controller aims to find a communication route with the maximum throughput (total bandwidth of the demands routed successfully) that has a cost of less than 10 (the cost can be defined in dollars or as a function of buffer utilisation [15]) and a total delay of shorter than 2 seconds . Thus, the constraint on the cost and total delay would benefit the real-time performance.

To the best of our knowledge, the existing *MCSP* algorithms are mostly based on linear programming or graph searches, and their high complexity prohibits them from scaling to large networks [10, 11, 21]. As a special and simpler case with only one constraint, the *CSP* problem is the basis for solving the *MCSP*, which can be classified into the following categories based on *exact/approximate* and *index-free/index-based* solutions. The *exact CSP* result provides the ground-truth and satisfies the user's actual requirement [20, 22–24], but it is extremely slow to compute as an NP-H problem [25, 26]. Therefore, the *approximate CSP* algorithms [13, 20, 26–28] are proposed to reduce the computation time by sacrificing the optimality of the path length. However, their speedup compared with the exact solutions is very limited [23, 29]. To further improve the query efficiency, a few *indexes* [30, 38] have been proposed by extending the existing index structures of the shortest path problem. For example, *CSP-CH* [30] extends the traditional *CH* index [31] to support exact *CSP* query processing. Although the index construction time is usually long, which makes them inflexible when the weight or cost changes [32–34], the index-based methods enjoy the high efficiency of query answering. Since *2-hop labelling* [35–37] is the current state-of-the-art shortest path index, in this work, we aim to provide a *2-hop labelling*-based index structure for exact *MCSP* queries with both faster index construction and more efficient query processing.

However, it is non-trivial to extend the *2-hop labelling* to answer the *MCSP* queries. Since it is already NP-H to find the optimal labels with size of $\Omega(|V||E|^{1/2})$ for the single-criterion shortest path as proven in [35], it would be much harder for the *MCSP* problem due to the following reasons:

Firstly, since constraints can be of any possible combination whilst being unknown beforehand, the skyline path information is inevitably needed in the index to be able to deal with any scenario. However,

[3] https://www.qld.gov.au/transport/safety/holiday-travel/scenic

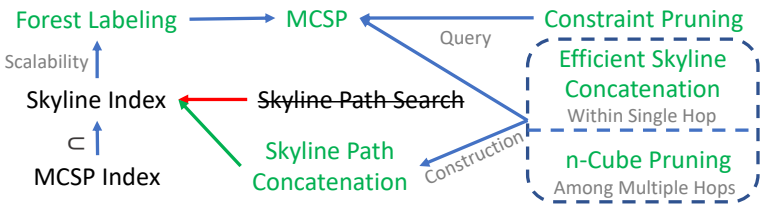


Figure 3.1: Framework Overview

a skyline path search is very time-consuming as discussed before, and hence the existing *MCSP* solutions all suffer from low efficiency. Consequently, none of the existing indexes is constructed by the exact skyline path but resorts to the approximate path search. The *CSP-CH* [30] chooses to contract a limited number of vertices and adopts heuristic testing before each skyline path expansion, which leads to a subset of the skyline shortcuts with smaller index sizes but slower query time. *COLA* [38] resorts to the graph partitioning and approximates skyline path search to reduce intermediate result size, but it sacrifices the query accuracy. Therefore, skyline path search is the bottleneck of existing methods and hinders the development of a full exact index, without which a fast exact *MCSP* query answering is impossible. In this work, we propose a *skyline path concatenation* algorithm to completely avoid the expensive skyline path search and construct a *2-hop labelling* index that stores the complete skyline paths without compromising query accuracy or efficiency.

Secondly, direct concatenation of two skyline paths does not necessarily generate a skyline path; therefore, another $O(mn \log mn)$ time is needed to validate the skyline results, where m and n are the size of the two skyline path sets to be concatenated. To speed up this process, we provide several insights of the multi-dimensional skyline path concatenation and propose an efficient concatenation method to incorporate the validation into the concatenation and meanwhile prune the concatenation space. Although the worst-case complexity remains the same when all the concatenation results are all skylines, our algorithm is much more efficient in practice.

Thirdly, the skyline path computation can hardly scale to larger networks. As the path length increases in large networks, the number of skyline paths becomes larger. Consequently, both the index size and index construction time increase dramatically. To address the scalability issue, we propose the *forest hop labelling* framework which partitions the networks into multiple regions and constructs the index parallelly both within and across the regions.

Last but not least, the final index is essentially for the skyline path query since it contains all the skyline path information. Hence, it would take approximately $m \times n \times h$ times of skyline concatenations before answering the query, where h is the number of hops. This is much larger than the traditional *2-hop labelling* index which only needs h times of calculation. In other words, the query performance of the index deteriorates dramatically and loses the promise of its high efficiency. Therefore, we propose an *n-Cube pruning* technique to prune the useless concatenation as early as possible. Furthermore,

we propose a *constraint-based pruning* technique for faster *MCSP* query answering and also provide pruning techniques for forest label construction. As shown in the experiments, our approach can answer the *MCSP* query within 1 ms and the proposed pruning techniques can further reduce it by two orders of magnitude, whilst the existing methods are thousands of times slower. Our contributions are illustrated in Figure 3.1 and summarised below:

- We first propose a skyline path concatenation-based *MCSP-2Hop* method that avoids the expensive skyline path search to achieve efficient *MCSP* query answering and index construction. Several concatenation insights are further presented to reduce the concatenation space and incorporate multi-dimensional skyline validation into concatenation.
- We further propose *forest hop labelling* for scalability. We also design an n-Cube pruning technique to further speed up the multi-hop concatenation during index construction and query answering, and a constraint pruning method to speed up *MCSP* specifically.
- We thoroughly evaluate our method with extensive experiments on real-life road networks and the results show that it outperforms the state-of-the-art methods by several orders of magnitude.

This paper extends the work [39] where we introduced the *Forest Hop Labeling* for *CSP* with only one constraint. Because the path concatenation algorithm and pruning technique were only designed for one constraint, its multi-constraint version in [39] was only a straightforward and slow solution. Therefore, in this paper, we 1) generalise the path concatenation methods into the multi-cost scenarios and 2) extend the pruning techniques to the multi-constraint scenario to achieve higher performance. Although they are the generalisations of the *CSP*, the problem in the multi-dimensional space is much harder than the 2-dimensional one such that *CSP* is only a very special case of *MCSP* with unique properties. Therefore, we provide new insightful properties of the high dimensional skyline path concatenation and pruning to facilitate the more efficient index construction and query answering in *MCSP*.

In the remainder of this paper, we first define our problem formally and introduce the essential concepts in Section 4.2. Sections 3.3 and 3.4 present our *MCSP-2Hop* and *forest hop labelling* with their index structures and query answering methods, and Section 3.5 presents the speedup techniques. We report our experimental results in Section 4.7.

3.2 Problem Definition

A road network is a n-dimensional graph $G(V,E)$, where V is a set of vertices and $E \subseteq V \times V$ is a set of edges. Each edge $e \in E$ has n criteria falling into two categories: weight $w(e)$ and a set of costs

$\{c_i(e)\}$, $i \in [1, n-1]$. A path p from the source $s \in V$ to the target $t \in V$ is a sequence of consecutive vertices $p = \langle s = v_0, v_1, \ldots, v_k = t \rangle = \langle e_0, e_1, \ldots, e_{k-1} \rangle$, where $e_i = (v_i, v_{i+1}) \in E, \forall i \in [0, k-1]$. Each path p has a weight $w(p) = \sum_{e \in p} w(e)$ and a set of costs $\{c_i(p) = \sum_{e \in p} c_i(e)\}$. We assume the graph is undirected whilst it is easy to extend our techniques to the directed one. Then we define the *MCSP* query below:

Definition 3.1 (MCSP Query)**.** *Given a n-dimensional graph $G(V,E)$, a MCSP query $q(s,t,\overline{C})$ returns a path with the minimum $w(p)$ whilst each $c_i(p) \leq C_i \in \overline{C}$, where $i \in [1, |\overline{C}|]$.*

As analysed in Section 5.1, it is quite slow to answer a *MCSP* query by online graph search. Therefore, in this paper, we resort to index-based method and study the following problem:

Definition 3.2 (Index-based MCSP Query Processing)**.** *Given a graph G, we aim to construct a hop-based index L that can efficiently answer any MCSP query $q(s,t,\overline{C})$ only with L, which is stored in a lookup table and can avoid the online searching in road networks.*

Next, we prove that skyline paths are essential for building any *MCSP* index:

Theorem 3.1. *The skyline paths between any two vertices are complete and minimal for all MCSP queries.*

Proof. We first prove the completeness. Suppose the skyline paths from s to t are $\{\overline{p}_1, \ldots, \overline{p}_k\}$, then each cost dimension c_i can be divided into $k+1$ intervals (suppose the path values on c_i are different. If some of them are the same, we can have interval number lower than $k+1$): $[0, c_i^1), [c_i^1, c_i^2), \ldots, [c_i^k, \infty)$, where each c_i^j is the j^{th} cost value of criterion c_i sorted increasingly, and the entire space can be decomposed into $(k+1)^{(n-1)}$ subspaces. Then for the subspaces with any interval equals to $[0, c_i^1)$, there is no path satisfying all the constraints at the same time. This is because c_i^1 has the smallest value along c_i. Otherwise, c_i^1 should be replaced by the smallest one. For the subspaces with no interval falling into $[0, c_i^1)$, there is always at least one path dominating all the other paths in such a subspace, and the one with the smallest distance is the result.

Next, we prove this skyline path set is minimal. Suppose we remove any skyline path $\overline{p}_j = \{c_i^j\}$ from the skyline result set. Then the *MCSP* query whose constraint is within the subspace of $[c_1^j, c_1^{j+1}) \times [c_2^j, c_2^{j+1}) \times \cdots \times [c_k^j, c_k^{j+1})$ finds no path in this subspace and the result becomes empty, which is incorrect. Hence, $\overline{p}_j$ cannot be removed and the skyline path set is minimal for all the *MCSP* queries between s and t. □

In summary, building an index for *MCSP* queries is equivalent to building the index for multi-dimensional skyline path queries.

2-hop labelling is an index method that does not involve any graph search during query answering. It answers the shortest distance query by table lookup and summation. Specifically, $\forall u \in V$, we assign

Table 3.1: MCSP-2Hop Labels of Figure 2.3

	v_3	v_7	v_{11}	v_{10}	v_9	v_4	v_1	v_5	v_8	v_{12}
v_2	(1, 1)				(2, 5),(4, 3)	(1, 2)	(1, 2)	(2, 4),(4, 3)	(2, 4)	(3, 5),(7, 4)
v_3					(1, 4),(6, 3)	(2, 3),(3, 2)	(2, 3)	(3, 5),(4, 4)	(2, 6),(3, 5) (4, 4)	(3, 7),(4, 6) (5, 5)
v_6		(2, 0)					(2, 0)	(1, 1)	(3, 5),(4, 4) (5, 3)	(4, 2)
v_7							(3, 5),(4, 0)	(1, 3),(3, 1)	(3, 3)	(2, 2)
v_{13}			(2, 3) (5, 2)	(3, 5) (4, 0)	(4, 9),(5, 6) (6, 1)	(4, 9),(5, 7) (6, 4),(9, 2)	(6, 13),(7, 11) (8, 8),(9, 6) (12, 5)	(5, 11),(6, 9) (7, 6),(8, 5) (10, 4)	(3, 7),(4, 5) (5, 2)	(2, 6),(3, 4) (6, 3)
v_{11}				(1, 2)	(3, 3)	(3, 4)	(5, 8) (7, 3)	(4, 6),(5, 5) (6, 4)	(2, 2)	(1, 1)
v_{10}					(2, 1)	(2, 4),(5, 2)	(4, 8),(6, 7) (7, 6),(8, 5)	(3, 6),(4, 5) (6, 4)	(1, 2)	(2, 3)
v_9						(2, 4),(3, 1)	(3, 7),(5, 5)	(3, 6),(4, 5)	(1, 2)	(2, 3)
v_4							(2, 4),(4, 3)	(1, 2)	(1, 2)	(2, 3)
v_1								(2, 2),(3, 1)	(3, 6),(5, 5) (6, 4),(7, 3)	(4, 7),(6, 2)
v_5									(2, 4),(3, 3)	(3, 5),(4, 4) (5, 3)
v_8										(1, 1)
v_{12}									(1, 1)	

a label set $L(u) = \{v, w(v,u)\}$ to store the minimum weights between a set of *hop vertices* v to u. To answer a query $q(s,t)$, we first determine the intermediate hop set $H = L(s) \cap L(t)$. Then we can find the minimum weight via the hop set: $min_{h \in H}\{w(s,h) + w(h,t)\}$. In this work, we select *H2H* [37], the state-of-the-art *2-hop labelling*, to support *MCSP* queries since it can avoid graph traversal in both index construction and query processing. Specifically, *Tree Decomposition* [97] is the cornerstone of *H2H*, which maps a graph into a tree structure.

3.3 MCSP-2Hop Labelling

In this section, we first present the *MCSP-2Hop* index structure and query processing, followed by the index construction, correctness proof and path retrieval.

3.3.1 Index Structure

Unlike the traditional *2-hop labelling* whose label is only a vertex and its corresponding distance value, *MCSP-2Hop*'s label is a set of skyline paths. Specifically, for each vertex $u \in V$, it has a label set $L(u) = \{(v, P(u,v))\}$, where v is the hop vertex ($\{v\}$ is u's hop set) and $P(u,v)$ is the skyline path set. We use $L = \{(L(u)|\forall u \in V\}$ to denote the set of all the labels. If L can answer all the *MCSP* queries in G, then we say L is a *MCSP-2Hop* cover. The detailed labels of the example graph in Figure 2.3 are shown in Table 3.1, with the rows being the vertices and columns being the hops. With the help of the tree decomposition in Figure 2.2, we can see each vertex stores all its ancestors on tree as labels. For example, X_{12}, X_8 and X_5 are the ancestors of X_1, then they appear in v_1's labels. Because X_2, X_{13}, and

X_6 are the leaves, they will not appear in any vertex's labels. Some cells have several values because they are skyline paths. The details of the index construction are presented in Section 3.3.3.

3.3.2 Query Processing

Given a *MCSP* query $q(s,t,\overline{C})$, we can answer it with the *MCSP-2Hop* in the same way as the single-criteria scenario: we first determine the common intermediate hop set $H = L(s) \cap L(t) = \{h | h \in X_{LCA(s,t)}\}$ by retrieving the vertices in the *LCA* of s and t, where $LCA(s,t)$ returns the *LCA* of X_s and X_t, each vertex in the *LCA* is regarded as a hop, we use h to represent them specifically. Then for each $h_i \in H$, we compute its candidate set $P_c(h_i)$ by concatenating the paths $P(s,h_i)$ and $P(h_i,t)$. By taking the union of $P_c(h_i)$ from all hops in H, we obtain the candidate skyline path set $P_c(H)$:

$$\begin{aligned} P_c(H) &= \{P_c(h_i) = P(s,h_i) \oplus P(h_i,t) | \forall h_i \in H\}, \\ &= \{p_j \oplus p_k | \forall p_j \in P(s,h_i) \wedge \forall p_k \in P(h_i,t) \wedge \forall h_i \in H\} \end{aligned}$$

where $\oplus$ is *path concatenation* operator that takes the sum of paths' weights and the corresponding costs. Each $P_c(h_i)$ contains all the pairwise skyline concatenation results, and $P_c(H)$ is their union.

During the concatenation, we maintain the current optimal result (i.e., the shortest one satisfying the constraint $\overline{C}$) and keep updating it. Finally, after all the candidates are generated and validated, we return the optimal one as the *MCSP* result. For example, given a query $q(v_9, v_7, \{6\})$, the *LCA* of v_9 and v_7 in the tree is X_1, and it contains vertices $H = \{v_1, v_5, v_8, v_{12}\}$. For hop v_1, $L(v_9)[v_1] = \{(3,7),(5,5)\}$ and $L(v_7)[v_1] = \{(3,5),(4,0)\}$. Therefore, we can get four candidates $P_c(v_1) = \{(6,12),(7,7),(8,10),(9,5)\}$. Only $(9,5)$ has a cost smaller than 6, so $(9,5)$ is the current best result. For hop v_5, we can get candidates $P_c(v_5) = \{(4,9),(5,6),\ (6,7),(7,4)\}$. Then $(5,6)$ replaces $(9,5)$ as it has a smaller weight and meets the constraint condition as well. For hop v_8, we can get candidates $P_c(v_8) = \{(4,5)\}$, and the current best result is updated to $(4,5)$. Finally, for hop v_{12}, we obtain the candidate $P_c(v_{12}) = \{(4,5)\}$, which equals the current best and is the final result

As shown in the above example, although only 4 hops are needed, we actually run 10 path concatenation operations. Specifically, unlike the *2-hop labelling* of the single-criterion case where only $|H|$ times of adding is needed, the concatenation of the skyline paths makes hop computation much more time-consuming. Suppose the label of s to h has m skyline paths and the label of h to t has n skyline paths. Then for each intermediate hop, we have to compute mn times, and the total time complexity grows to $O(mn|H|)$ instead of $O(|H|)$. Therefore, pruning over this large amount of computation is crucial to the query performance and we will elaborate it in Section 3.5.

3.3.3 Index Construction

The index construction is made up of two phases: a bottom-up *Skyline Tree Decomposition* that gathers the skyline shortcuts, creates the tree nodes and forms the tree, and a top-down *Skyline Label Assignment* that populates the labels with query answering, as shown in Algorithm 7. It should be noted that both of these two phases do not involve the expensive skyline path search and only use the skyline path concatenation. In Algorithm 7, we use the function *Skyline* to describe the operator of skyline path concatenation on two sets of skyline paths from the start to hops and hops to the end, respectively. Then, it can return a set of skyline paths from the start to the end. We will discuss the details of the skyline concatenation in Section 3.5.

Algorithm 1: MCSP-2Hop Construction

Input: Graph $G(V,E)$
Output: MCSP-2Hop Labeling L

// Tree Node Contraction
$i \leftarrow 0$;
while $v \in V$ *has the minimum degree and* $V \neq \phi$ **do**
  $X_v = N(v)$, $r(v) \leftarrow i++$;
  for $(u,w) \leftarrow N(v) \times N(v)$ **do**
    $P(u,w) \leftarrow Skyline(P(u,v) \oplus P(v,u), P(u,w))$;
  for $(u,w) \leftarrow N(v) \times N(v)$ **do**
    $P(u,w) \leftarrow e(u,w) \cup (P(u,w))$, $E \leftarrow E-(u,v)-(v,w)$;
  $V \leftarrow V - v$;
// Tree Formation
for v *in* $r(v)$ *increasing order* **do**
  $u \leftarrow min\{r(u)|u \in X_v\}, X_v.Parent \leftarrow X_u$
// Label Assignment
$X_v \leftarrow$ Tree Root, $Q.insert$(Tree Root)
while $X_v \leftarrow Q.pop()$ **do**
  for $u \in \{u|X_u$ *is ancestor of* $X_v\}$ **do**
    $P(v,u)) \leftarrow Skyline(P(v,w) \oplus P(w,u)), \forall w \in X_v$;
    $L(v) \leftarrow (u, P(v,u))$;
  $Q.insert(X_u.children)$, $L \leftarrow L \cup L(v)$;

Skyline Tree Decomposition

We contract the vertices in the degree-increasing order, created by *Minimum Degree Elimination* [18,63]. For each $v \in V$, we add a skyline shortcut $P'(u,w)$ to all of its neighbour pairs (u,w) by $P(u,v) \oplus P(v,w)$. If (u,w) does not exist, then we use $P'_s(u,w)$ as its skyline path $P(u,w)$ directly. Otherwise, we compute the skyline of $P(u,w)$ and $P'(u,w)$ in a linear time sort-merge way, which sorts the two path sets based

on w first and obtain the new skyline paths by comparing them in the distance-increasing order. After that, a tree node X_v is formed with v and its neighbours $N(v)$, and all their corresponding skyline paths from v to $N(v)$. The tree node X_v is assigned an order $r(v)$ according to the order when v is contracted. After contracting v, v and its edges are removed from G, and the remaining vertices' degrees are updated. Then we contract the remaining vertices in this way until none remains.

Theorem 3.2. *The time complexity of the tree node contraction phase is* $O(|V|(max(|X_i|)^2 \cdot n \cdot c_{max}^{2n-2} \log^{n-1} c_{max} + \log|V|))$, *where* $max(|X_i|)$ *is the tree width,* c_{max} *is the maximum criteria value of all the skyline paths, and* c_{max}^{n-1} *determines the worst-case of skyline path number.*

Proof. For the contraction on a tree node X_v, we conduct concatenation for all-pair neighbours of v, and thus $O((max(|X_i|)^2)$ times of concatenation operations are executed. As for the concatenation, c_{max} is the largest maximum criteria value of all the skyline paths, whilst c_{max}^{n-1} denotes the worst case of skyline path number of all the skyline path sets. This complexity was originally from [19] with c_{max} denoting the smaller maximum criteria value in the two-dimensional case. However, the smaller one cannot be used to bound in the higher dimensional case, so we use the largest one for the upper bound. Intuitively, for one skyline path set $P(u,v)$, each path $p \in P(u,v)$ has the largest cost $c_i^{max}(p)$ for each of its n dimensions. Then the largest $c_i^{max}(p)$ is an upper-bound c_{max} (loose but bounded) of the 2-dimensional skyline path number between c_i and any other dimension c_j and between any two dimensions. Then suppose we have a 3-dimensional case c_i, c_j and c_k, then we have at most $O(c_{max}^2)$ skyline paths; it would be easy to generalise this to n dimensions with the worst-case $O(c_{max}^{n-1})$ skyline paths in $P(u,v)$. Then for all the skyline path sets, which at this stage are the edges created in all the tree nodes $\{X_v\}$, the maximum skyline path number is the upper bound of the skyline path number in each skyline path set globally.

During concatenation, we have $O(c_{max}^{n-1} \times c_{max}^{n-1}) = O(c_{max}^{2n-2})$ concatenated results to choose from, and it takes $O(c_{max}^{2n-2} \log^{n-1} c_{max}^{2n-2}) = O(n \cdot c_{max}^{2n-2} \log^{n-1} c_{max})$ time to find the skyline paths from the $O(c_{max}^2)$ candidates. Afterwards, tree node ordering (find the one with the minimum updated degree) takes $O(log|V|)$ time. As we have $|V|$ vertices that need to be contracted, the total complexity of the contraction phase is $O(|V|(max(|X_i|)^2 \cdot n \cdot c_{max}^{2n-2} \log^{n-1} c_{max} + \log|V|))$. □

We use the graph in Figure 2.3 as an example. Firstly, we contract v_2 with $N(v_2) = \{v_1, v_3, v_4\}$. Because there is no edge between v_1 and v_3, we add a shortcut (v_1, v_3) by $w = w(v_1, v_2) + w(v_2, v_3) = 2$ and $c = c(v_1, v_2) + c(v_2, v_3) = 3$. Similarly, there is no edge between v_1 and v_4, so we add a new edge (v_1, v_4) with $w = w(v_1, v_2) + w(v_2, v_4) = 2$ and $c = c(v_1, v_2) + c(v_2, v_4) = 4$. As for v_4 and v_3, we first generate a shortcut by $P(v_4, v_2) \oplus P(v_2, v_3) = (2, 3)$. Then because there already exists an edge $(3, 2)$ from v_4 to v_3, we compute the new skyline paths between the shortcut and the edge and get the new skyline paths $P(v_4, v_3) = \{(3, 2), (2, 3)\}$. After that, we remove v_2 from the graph, leaving v_1's degree

changed to 4, v_4's degree remaining 5, and v_3's degree remaining 3. We also obtained a tree node $X_{v_2} = \{v_2, v_3, v_1, v_4\}$ with the skyline paths from v_2 to $N(v_2)$ and order $r(v_2) = 1$. This procedure runs on with the next vertex with the smallest degree. When all the vertices finished contraction, we have an order of : $\langle v_2, v_3, v_6,$
$v_7, v_{13}, v_{11}, v_{10}, v_9, v_4, v_1, v_5, v_8, v_{12}\rangle$.

Subsequently, we connect all the nodes to form a tree by connecting each tree node to its smallest-order neighbour. For example, X_{10} has a neighbour set of $\{v_8, v_9, v_{12}\}$, and X_9 has the smallest order of them, so X_{10} is connected to X_9 as its child node.

Skyline Label Assignment

It should be noted the obtained tree preserves all the skyline path information of the original graph because every edge's information is preserved in the shortcuts. In fact, we can view it as a *superset* of the *MCSP-CH* result. Therefore, we can assign the labels with skyline search on this tree directly. However, it would be very time-consuming as the *Sky-Dijk* is much slower than *Dijkstra*'s. To this end, we use the top-down label cascading assignment method that advantages the existing labels with query answering to assign labels to the new nodes.

Specifically, a label $L(u)$ stores all the skyline paths from u to its ancestors. Therefore, we start from the root of the tree to fill all the labels. For the root itself, it has no ancestor, so we just skip it. Next, for its child u, because the root is the only vertex in X_u other than u, we add the root to u's label. Then for any tree node X_v, it is guaranteed that all its ancestors have obtained their labels. Then for any of its ancestor a_i, we compute its skyline paths by taking the skyline paths from $\{P(v,u) \oplus P(u,a_i) | \forall u \in X_v\}$.

Theorem 3.3. *The time complexity of the label assignment phase is* $O(|V| \cdot h \cdot max(|X_i|) \cdot n \cdot c_{max}^{2n-2} \cdot \log^{n-1} c_{max})$, *where h is the tree height, and the space complexity of the labels is* $O(h \cdot |V| \cdot c_{max}^{n-1})$.

Proof. On the label assignment for a tree node X_v, we compute concatenation between the skyline paths from v to $\forall u \in X_v$ and the skyline paths from u to one of its ancestors, which takes $O(max(|X_i|))$ times of concatenations. Moreover, the extreme case on the number of ancestors towards a tree node is the tree height h. Therefore, we need $O(h \cdot max(|X_i|))$ concatenations in total. Because each concatenation takes $O(c_{max}^{2(n-1)} \log^{n-1} c_{max}^{2(n-1)}) = O(n \cdot c_{max}^{2(n-1)} \log^{n-1} c_{max})$ time, the label assignment on all the tree node takes $O(h \cdot |V| max(|X_i|) \cdot n \cdot c_{max}^{2n-2} \log^{n-1} c_{max})$ time. For space complexity, the worst case of the skyline path number towards a pair in a tree node is $O(c_{max}^{n-1})$. The worst case of the vertices number in a tree node is the tree height h, and we have the $|V|$ label set. Thus, the total space complexity is $O(h \cdot |V| \cdot c_{max}^{n-1})$. □

For example in Figure 2.2, suppose we have already obtained the labels of X_1, X_5, X_8, and X_{12}, and we are computing the labels of X_4, which has an ancestor set of $\{v_{12}, v_8, v_5, v_1\}$. We take the

computation of $L(v_4)[v_{12}]$ as an example. X_4 has three vertices $\{v_1, v_5, v_8\}$ apart from v_4, so we compute the skyline paths via these tree hops, which are denoted by $\{h_1, h_5, h_8\}$. To show the process clearly, we do not apply the pruning first. We use $P_c(h_i)$ to denote the set of all the concatenated paths via hop h_i and use $P_s(h_i)$ to denote the set of skyline paths via hop h_i.

$$\begin{aligned}
P_c(h_1) &= P(v_4, v_1) \oplus P(v_1, v_{12}) \\
&= \{(2,4),(4,3)\} \oplus \{(4,7),(6,2)\} \\
&= \{(6,11),(8,6),(8,10),(10,5)\} \\
P_s(h_1) &= \{(6,11),(8,6),(10,5)\}
\end{aligned}$$

$$\begin{aligned}
P_c(h_5) &= P(v_4, v_5) \oplus P(v_5, v_{12}) \\
&= \{(1,2)\} \oplus \{(3,5),(4,4),(5,3)\} \\
&= \{(4,7),(5,6),(6,5)\} \\
P_s(h_5) &= \{(4,7),(5,6),(6,5)\}
\end{aligned}$$

$$\begin{aligned}
P_c(h_8) &= P(v_4, v_8) \oplus P(v_8, v_{12}) \\
&= \{(1,2)\} \oplus \{(1,1)\} = \{(2,3)\} \\
P_s(h_8) &= \{(2,3)\}
\end{aligned}$$

Because $(2,3)$ dominates all the others, we put $(2,3)$ into the label of $L(v_4)[v_{12}]$.

3.3.4 Correctness

In this section, we show that the *MCSP-2Hop* constructed in Algorithm 7 is correct, i,e., the *MCSP* query between any source vertex s and target vertex t can be correctly answered.

As proved in Theorem 5.2, the skyline paths between s and t are enough to answer any *MCSP* query, so we only need to prove the candidate set $P_c(H)$ covers all the skyline paths between s and t. Firstly, we introduce the following lemma:

Lemma 3.4. *If a path p is a skyline path, the sub-paths on p are all skyline paths.*

Proof. The proof of it can be found in [30]. □

Theorem 3.5. *$P_c(h)$ covers all the skyline paths from s to t that travel via hop h.*

Proof. Suppose p^* is a skyline path from s to t via h and $p^* \notin P_c(h)$. According to Lemma 3.4, p^*'s sub-paths $p^*(s,h)$ and $p^*(h,t)$ are also skyline paths from s to h and from h to t. If $p^*(s,h) \notin P(s,h)$ or $p^*(h,t) \notin P(h,t)$, it contradicts the definition that $P(s,h)$ contains all the skyline paths from s to h, and similar for $P(h,t)$. Therefore, $P_c(h)$ covers all the skyline paths from s to t via h. □

Next, we need to prove the union of all the hops $h \in H$ covers all the skyline paths between s and t. Because our *2-hop labelling* is based on the graph cut, we only need to prove the following theorem:

Theorem 3.6. *If the hop set H is the set of cuts between s and t, then $P_c(H) = \bigcup_{h_i \in H} P_c(h_i)$ covers all the skyline paths from s to t.*

Proof. Similar to Theorem 3.5, we also prove it with a contradiction. Suppose p^* is a skyline path and $p^* \notin P_c(H)$. Because H is the graph cut set, p^* has to travel through one of the cut vertex $h_i \in H$, which has a candidate set $P_c(h_i)$. However, $p^* \notin P_c(h_i)$ contradicts with Theorem 3.5. So every skyline path from s to t exists in $P_c(H)$. □

Finally, we prove the labels generated by Algorithm 7 is a *MCSP-2Hop*.

Theorem 3.7. *L constructed by Algorithm 7 is MCSP-2Hop, that is $\forall (v, P(u,v)) \in L(u), (u, v \in V)$, the path set $P(u,v)$ covers skyline paths between u, v.*

Proof. Firstly, we define graph $G(v)$ as the assembly of tree nodes from X_v to the root of *skyline tree decomposition* T_G. It can be easily obtained that $G(v)$ is *skyline preserved* by referring to the *distance preserved property* of tree decomposition in [37]. Next, we prove that $P(u,v)$ covers skyline paths between (u,v) through induction. We suppose that height h of the tree root is zero and the height of one tree node is its parent's height plus one. For the root with $h = 0$, its label is *MCSP-2Hop*. For X_v with height $h = 1$, the labels in $L(v)$ are also *MCSP-2Hop* because of the skyline preserved property. Then suppose that labels in $L(v)$ with height $h = i (i > 1)$ are *MCSP-2Hop*, we attempt to prove that labels in $L(u)$ with $h = i+1$ are also *MCSP-2Hop*. Since $G(u)$ is skyline preserved with $X_u \setminus \{u\}$ being $u's$ neighbour set in $G(u)$, then all the skyline paths must pass through vertices in this set. According to the top-down label assignment, the labels are obtained by concatenating the two paths with $X_u \setminus \{u\}$ being the hops, so the computed labels in $L(u)$ are also *MCSP-2Hop*. Therefore, L constructed by Algorithm 7 is *MCSP-2Hop*. □

3.3.5 Path Retrieval

To retrieve the actual result path, we need to preserve the intermedia vertex in the labels. Specifically, during the *Tree Node Contraction* phase, we need to store the contracted vertex of each skyline path in each shortcut. During the path retrieval, suppose one of the skyline results is the concatenation of $s \to h$ and $h \to t$, then they are actually two shortcuts so we only need to insert the intermediate vertices recursively until all no shortcut remains to recover the original path.

3.4 Forest Hop Labelling

Due to the nature of the skyline, the *MCSP-2Hop* size could be orders of magnitude larger than the distance labels, especially when the paths are long. Therefore, we present our *Forest Hop Labelling* to reduce the index size by partitioning the graph into regions and building a small tree for each region, and a boundary tree between regions.

3.4.1 Graph Partition

We partition G into a set $C = \{G_1, G_2, \ldots, G_{|C|}\}$ of vertex-disjoint subgraphs such that $\bigcup_{i\in[1,|C|]} G_i = G$. If an edge appears in two different subgraphs, it is a *Boundary Edge*, and its end vertices are called *Boundary Vertex*. Graph partitioning is a well-studied problem and we can use any existing solutions. In our implementation, we use the *Natural Cut* method [79], which is also used by *COLA* [38], because such an approach can create only a small number of boundary vertices. The influence of the partitioning method on our forest labelling index structure is another important problem and we will study it in future work.

We use Figure 2 as an example to illustrate the procedure of our *Forest Hop Labeling* construction. As shown in Figure 3.2, we partition the whole network into two subgraphs by cutting the edges connecting them. Then, we can identify that the boundary vertices in Figure 3.2-(a) are $\{v_7, v_8, v_9\}$, and the boundary vertices are $\{v_3, v_4.v_5, v_6\}$.

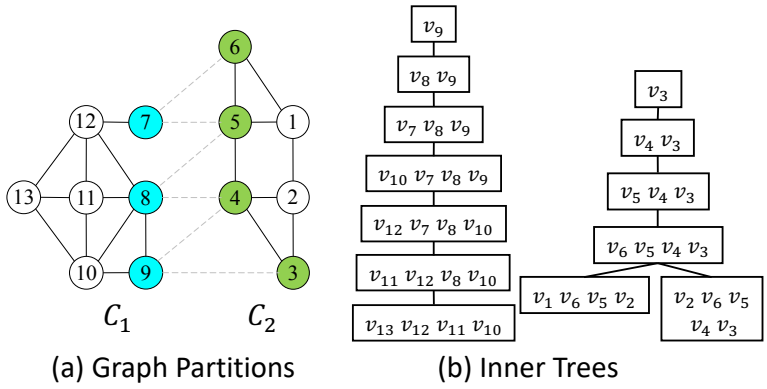


Figure 3.2: Example of Graph Partition

3.4.2 Inner Partition Tree

Conceptually, we build a small tree (*MCSP-2Hop*) for each partition and these procedures can run in parallel. However, these trees cannot be constructed directly with only the subgraph information because the actual paths may take a detour out of the current partition and come back later. Therefore,

we need to pre-compute the all-pair boundary results for each partition because these boundaries form the cut of a partition naturally and their all-pair results can cover all the detours. In other words, a complete graph of the boundaries is overlapped to the partition with each edge is a skyline path set. This procedure is also fast because we only need to run several search-space-reduced *Sky-Dijk*s in parallel. When forming the tree nodes, we also follow the degree elimination order. However, we postpone the boundary contraction to the last phase because they are also part of the boundary tree and we can utilise them for faster query answering later on.

Theorem 3.8. *The inner tree construction time complexity is* $O(|C| \cdot |V_i| \cdot (max(|X_j|) \cdot n \cdot c_{max}^{2n-2} \cdot \log^{n-1} c_{max} \cdot (max(|X_j|) + h_i) + \log|V_i|)$*, where* $|V_i|$ *is the vertex set of subgraph* G_i*,* h_i *is the inner tree height, and* X_j *is the tree node belongs to* G_i*.*

Proof. In terms of Theorem 3.2, the time complexity of a single inner tree's contraction is $O(|V_i|(max(|X_j|)^2 \cdot n \cdot c_{max}^{2n-2} \log^{n-1} c_{max} + \log|V_i|)$. In terms of Theorem 3.3, the label assignment phase on the inner tree is $O(h \cdot |V_i|(max(|X_j|) \cdot n \cdot c_{max}^{2n-2} \log^{n-1} c_{max} + \log|V_i|)$. The total time complexity for an inner tree is the sum of the two phases above, which takes $O(|V_i|(max(|X_j|) \cdot n \cdot c_{max}^{2n-2} \log^{n-1} c_{max}(max(|X_j|) + h_i) + \log|V_i|)$. The inner tree construction time complexity is the number of trees $|C|$ times the complexity of one inner tree. □

As for the node number, tree width, tree height, and c_{max} for most of the sub-graphs are much smaller, it is much faster than the original big tree. As the constructions are dependent, they can run in parallel to achieve higher performance.

Theorem 3.9. *The total inner tree label size is* $O(|C| \times |V_i| \cdot |h_i| \cdot c_{max}^{n-1})$.

Proof. In terms of Theorem 3.3, each inner tree takes $|V_i| \cdot |h_i| \cdot c_{max}^{n-1}$ space, and we have $|C|$ inner trees. □

To construct inner trees in Figure 3.2-(a), we first build the index of the partition C_1. We pre-compute the skyline paths between each two boundary vertices, which include $P(v_7, v_8)$, $P(v_7, v_9)$, and $P(v_8, v_9)$. Then we add $P(v_7, v_8)$ and $P(v_7, v_9)$ as shortcuts in C_1, and compare the skyline paths in $P(v_8, v_9)$ with the edge (v_8, v_9) to figure out the final skyline paths between v_8 and v_9. We conduct tree decomposition on the inner partition tree with the shortcuts by the contracting order $\langle v_{13}, v_{11}, v_{12}, v_{10}, v_7, v_8, v_9 \rangle$, which can ensure the top tree nodes are corresponding to the boundary vertices. We can use the same method to construct C_2. Their inner partition trees are shown in Figure 3.2-(b).

3.4.3 Boundary Tree

When we have a query from different partitions, we need inter-partition information to help answer it. One way is precomputing the all-pair boundary results. However, unlike the previous one, this all-pair result is much larger and harder to compute because it contains boundaries of all the partitions and they are far away from each other. To avoid the expensive long-range *Sky-Dijk*, we also resort to the concatenation-based *MCSP-2Hop* to build a boundary tree.

The boundary tree is built on a graph that contains all the boundary edges and all the partition-boundary all-pair edges. Although we can still use the same degree elimination order to contract the vertices, its randomness ignores the property that these boundaries are the cuts of the partitions. As a result, the inner partition trees and the boundary tree have no relation and they can hardly exchange information. Therefore, we contract the boundaries in the unit of partition instead of degree: the boundaries from the same partition are contracted together in the same order of their inner tree. In this way, we can attach the inner partition trees to the boundary tree. Furthermore, only the new shortcuts need skyline concatenation, whilst the existing ones can be inherited directly.

In terms of label assignment, we can populate the labels only to the boundaries and we call them *boundary labels*. It has a smaller index size but requires 6-hop in query answering.

Theorem 3.10. *The boundary tree construction takes* $O($
$|B|(max(|X_j|) \cdot n \cdot c_{max}^{2n-2} \log^{n-1} c_{max}(max(|X_i|) + h_B) + \log|B|)$ *and its label size is* $O(h_B \cdot |B| \cdot c_{max}^{n-1})$*, where* B *is the boundary set and* h_B *is the boundary tree height.*

Proof. We can prove the time complexity of the boundary tree index construction in terms of Theorem 3.8, and the space complexity in terms of Theorem 3.9. □

We can further push down the boundary labels to the inner labels and we call them *extended labels*. Specifically, for each inner partition tree, its ancestors in the boundary tree are also added to its labels. The extended label assignment takes an extra $O(|C||V_i|(max(|X_j| \cdot c_{max}^{2n-2} \log^{n-1} \cdot$
$c_{max}(max(|X_j|) + h_i + h_B)) + \log|V_i|))$ time and $O(h_B \cdot |V - B| \cdot c_{max}^{n-1})$ space.

Figure 3.3-(a) shows the graph that contains all the boundary edges (dashed lines) and all-pair edges (bold arcs) amongst the boundary vertices in each example sub-graph. As the contracting order of the boundary vertices in C_1 and C_2 are $\langle v_7, v_8, v_9 \rangle$ and $\langle v_6, v_5, v_4, v_3 \rangle$, and C_1 is contracted before C_2, the final contracting order for the boundary graph is $\langle v_7, v_8, v_9, v_6, v_5, v_4, v_3 \rangle$. The boundary tree is shown in Figure 3.3-(b). Because all the boundaries from one partition are contracted consecutively, this tree has two portions (shadowed in green and blue).

3.4.4 Query Processing

Depending on the locations of the two query points s and t, we discuss the following situations:

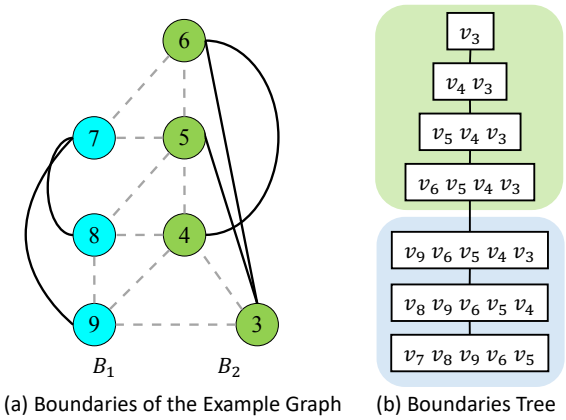


Figure 3.3: Boundary Tree of the Example Graph

1. *s* **and** *t* **are in the same partition**: Because the all-pair boundary information is considered during construction, we can use the local partition tree directly to answer it.

2. *s* **and** *t* **are all boundaries**: It can be answered with the boundary label directly.

3. *s* **is in one partition and** *t* **is another's boundary**: If we only have the boundary label, then we need to first find the results from *s* to its boundaries within the local label, then concatenate the results from these boundaries to *t* using the boundary labels, and it takes 4 hops. If we have the extended labels, we can use it as a 2-hop label directly.

4. *s* **and** *t* **are in different partitions**: If we only have the boundary label, then we need to first find the results from *s* to its boundaries within the local label, then concatenate the results from these boundaries to *t*'s boundaries using the boundary labels, and finally concatenate these results to *t* with *t*'s local labels. Clearly, this is a 6-hop procedure. Similar to the previous case, if we have the extended labels, we can use them as a 2-hop label directly. More advanced query-processing techniques are presented in the next section.

3.5 Skyline Path Concatenation

In this section, we present insights of the skyline path concatenation that speeds up the index construction and query processing. Specifically, we first discuss how to concatenate two skyline path sets via a single hop in Section 3.5.1, which is used in the *tree node contraction* phase of index construction. Then we extend to multiple hops in 3.5.2, which is used in the *label assignment* phase of index construction and query processing. Finally, we explain how to efficiently answer *MCSP* in Section 3.5.3 and how to apply these techniques to *forest labels* in Section 3.5.4.

$$P_1 = p_{<i}\,(s,h) \oplus p'_{<j}(h,t)$$
$$P_2 = p_{>i}\,(s,h) \oplus p'_{>j}(h,t)$$
$$P_3 = p_{<i}\,(s,h) \oplus p'_{>j}(h,t)$$
$$P_4 = p_{>i}\,(s,h) \oplus p'_{<j}(h,t)$$

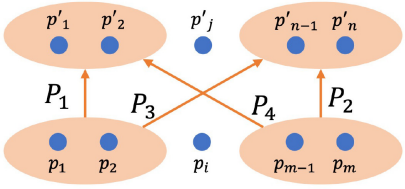


Figure 3.4: Concatenated Path Category for Property 3 and 4

3.5.1 Single Hop Skyline Path Concatenation

Given two sets of skyline paths $P(s,h)$ and $P(h,t)$ with sizes m and n, it takes two operations ($O(mn)$ time *path concatenation* and $O(mn\log mn)$ time *skyline path validation*) to obtain the skyline result. Because the *mn concatenation* is inevitable, we aim to speed up the *validation* by exploring the concatenation properties first. In the following, we first present the observations and algorithms for the 2D skyline path concatenation as it is simpler and has some unique characteristics. Then we generalise to the multi-dimensional case. Even though the computation still bears the same level of complexity, our proposed pruning techniques are efficient and reduce the computation time successfully in practice.

2D Skyline Path Concatenation Observations

The 2D skyline dataset has a unique characteristic compared with the multi-dimensional skyline dataset: when we sort all the data in the weight-increasing order, their costs are naturally sorted in decreasing order. Therefore, we have the following four observations to help improve the concatenation efficiency. We first dig into the simplest *1-n / m-1* skyline concatenation. The following property proves that we do not need validation for this simple case:

Property 3.1. ***(1-n / m-1 Skyline)*** *$\forall p \in P(s,h)$, $p \oplus P(h,t)$ can generate $n = |P(h,t)|$ paths that cannot dominate each other. Similarly, $\forall p' \in P(h,t)$, $P(s,h) \oplus p'$ can generate $m = |P(s,h)|$ paths that cannot dominate each other.*

Proof. We only need to prove the first half. Suppose we choose $p(w,c)$ from $P(s,h)$. Because $P(h,t)$ is the skyline path set, we select any two paths from it: $p_1(w_1,c_1)$ and $p_2(w_2,c_2)$. Suppose $w_1 \leq w_2$ and $c_1 \geq c_2$, then this relation still holds because $w_1 + w \leq w_2 + w$ and $c_1 + c \geq c_2 + c$. □

Next, we explore the *m-n* concatenation case. Suppose all the skyline paths are sorted in the weight-increasing order. We define $p_i(s,h) = (w_i,c_i)$ ($i \in [1,m]$) as the i^{th} skyline path in $P(s,h)$, $p'_j(h,t) = (w'_j,c'_j)$ ($j \in [1,n]$) as the j^{th} skyline path in $P(h,t)$, and p_i^j as the concatenation path of $p_i(s,h)$ and $p'_j(h,t)$. Therefore, p_1^1 is the concatenation of the first skyline path in $P(s,h)$ and the first path in $P(h,t)$, and P_m^n is the concatenation of the last skyline path in $P(s,h)$ and the last one in $P(h,t)$. They have the following property:

Property 3.2. ***(Endpoint Skyline)*** *p_1^1 and p_m^n are two endpoints of the concatenation results and they are skyline paths.*

Proof. $p_1^1 = p_1(s,h) \oplus p'_1(h,t) = p(w_1 + w'_1, c_1 + c'_1)$. Because w_1 is the smallest in $p_1(s,h)$ and w'_1 is the smallest in $p'_1(h,t)$, $w_1 + w'_1$ is still the smallest weight in P_c. Similarly, $p_m^n = p_m(s,h) \oplus p'_n(h,t) = p(w_m + w'_n, c_m + c'_n)$ has the smallest cost in P_c. Therefore, no other path in P_c can dominate p_1^1 and p_m^n. □

Finally, we explore the domination relations amongst the paths between the endpoints. Given any concatenated path p_i^j, we can classify the other paths (except the ones generated by p_i and p'_j) into four categories as shown in Figure 3.4. The following two properties provide the dominance relations between p_i^j and these four path sets.

Property 3.3. ***(Side-Path Irrelevant)*** *The concatenated path p_i^j cannot be dominated by paths in P_1 and P_2.*

Proof. $\forall p_k \in p_{<i}$ and $p'_k \in p'_{<j}$, $c_k > c_i$ and $c'_k > c_j$. Therefore, $c_k + c'_k > c_i + c'_j$, so P_1 cannot dominate p_i^j. Similarly, $\forall p_k \in p_{>i}$ and $p'_k \in p'_{>j}$, $w_k > w_i$ and $w'_k > w_j$. Therefore, $w_k + w'_k > w_i + w'_j$, so P_2 cannot dominate p_i^j. □

Property 3.4. ***(Cross-Path Relevant)*** *p_i^j is a skyline path iff it is not dominated by any path in P_3 and P_4.*

Proof. $\forall p_k \in p_{<i}$ and $p'_k \in p'_{>j}$, we have $c_k > c_i$ and $c'_k < c_j$, $w_k > w_i$ and $w'_k < w_j$, so $c_k + c'_k$ has no relation with $c_i + c'_j$, and $w_k + w'_k$ has no relation with $w_i + w'_j$. Therefore, paths in P_3 have the possibility to dominate p_i^j, and the same applies to P_4. In other words, it is the paths in P_3 and P_4 that determines if p_i^j could be a skyline path or not. □

Efficient 2D Skyline Path Concatenation

The above properties enable us to reduce the validation operations and speed up the 2D concatenation. The following theorem further reduces the comparison number in the validation operation:

Theorem 3.11. *Suppose the concatenated paths are coming in the weight-increasing order. We have k skyline paths and a new path p_{k+1} comes with a weight larger than w_k. If p_k cannot dominate p_{k+1}, then p_{k+1} must be a skyline path.*

Proof. Our theorem is based on the idea proposed in [98] of filtering through sorting in skyline computation. p_{k+1} cannot be dominated by latter paths because $w_{k+1} \leq w_j, \forall j > k+1$. p_k has the smallest cost of the current skyline paths. If $c_k \geq c_{k+1}$, then the other paths' cost is also larger than c_{k+1}. Therefore, if p_k cannot dominate p_{k+1}, then no future path can dominate it. □

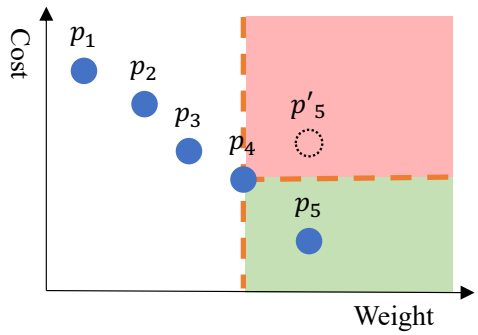


Figure 3.5: Next Skyline Path Validation

For example, in Figure 3.5, we have a set of skyline paths $\{p_1, p_2, p_3, p_4\}$ sorted in the weight-increasing order and p_5 is the next concatenated path with the next smallest weight. Therefore, p_5 can only locate in the right regions of p_4. If p_5 is in the red region, it is dominated by p_4 and discarded; If p_5 is in the green region, it becomes the next skyline path.

Based on Theorem 3.11, we propose an efficient skyline path concatenation as shown in Algorithm 2. As proved in Property 3.2, $p_1^1 = p_1(s,h) \oplus p'_1(h,t)$ is definitely a skyline result and we put it into the result set P_s. Because the next path with the smallest weight is either p_1^2 or p_2^1, we put them in a priority queue Q as candidates. Then we iterate the candidate queue until it is empty. Each time we retrieve the top path p_i^j in Q and compare it with the last path in P_s. If p_i^j is not dominated, we put p_i^j into P_s. Then we generate the next two candidates p_{i+1}^j and p_i^{j+1} if they exist. However, we do not insert them into Q directly as they could already have been dominated by the last path in P_s already, or they could have already been generated by other paths. Therefore, we use *LazyInsert* to identify the next set of candidates that are not dominated (Line 16 and 25) and not generated (Line 12 and 21) recursively. In this way, the pruning power of the current skyline path is maximised, and we could have a smaller Q and eliminate the redundancy.

Correctness: Firstly, because we generate and test both p_{i+1}^j and p_i^{j+1} of all p_i^j even if some of them are not inserted into Q, we have produced all mn candidates. Secondly, because Q is sorted by weight, and both p_{i+1}^j and p_i^{j+1} have larger weight than p_i^j, we visit the p_i^j from Q in the weight-increasing order. Therefore, according to Theorem 3.11, Algorithm 2 returns a set of concatenated skyline paths.

Complexity: The maximum depth of Q is $log(mn)$, so it takes $O(mn\log(mn))$ to pop out all the elements. Since we just check the dominant relation once for each connecting path, the skyline validation time for each path is constant. Although the worst case time complexity of concatenating all the skyline paths via h is still $O(mn\log(mn))$, the *LazyInsert* pruning can keep the size of Q small in practice.

Algorithm 2: Efficient 2D Skyline Path Concatenation

Input: Sorted Skyline Path Set $P(s,h)$ and $P'(h,t)$
Output: Skyline Concatenated Path $P_s \subseteq P(s,h) \oplus P'(h,t)$

$p_1^1 \leftarrow p_1(s,h) \oplus p'_1(h,t)$; $P_s.insert(p_1^1)$;
$p_1^2 \leftarrow p_1(s,h) \oplus p'_2(h,t)$; $Q.insert(p_1^2)$;
$p_2^1 \leftarrow p_2(s,h) \oplus p'_1(h,t)$; $Q.insert(p_2^1)$;
while $!Q.empty()$ **do**
    $p_i^j \leftarrow Q.pop()$; $p_{max} \leftarrow P_s[|P_s|-1]$;
    if $c_i^j \leq c_{max}$ **then**
        $P_s.insert(p_i^j)$;
    $LazyInsert(i,j)$;
return P_s;
Procedure $LazyInsert(i,j)$
    if $i+1 < m$ **then**
        if p_{i+1}^j *is visited* **then**
            $LazyInsert(i+1,j)$;
            break;
        $p_{i+1}^j \leftarrow p_{i+1}(s,h) \oplus p'_j(h,t)$;
        if $c_{i+1}^j > c_{max}$ **then**
            $LazyInsert(i+1,j)$;
        else
            $Q.insert(p_{i+1}^j)$
    if $j+1 < n$ **then**
        if p_i^{j+1} *is visited* **then**
            $LazyInsert(i,j+1)$;
            break;
        $p_i^{j+1} \leftarrow p_i(s,h) \oplus p'_{j+1}(h,t)$;
        if $c_i^{j+1} > c_{max}$ **then**
            $LazyInsert(i,j+1)$;
        else
            $Q.insert(p_i^{j+1})$

Multi-Dimensional Skyline Path Concatenation

Although the previous observations, theorems, and algorithms efficiently reduce the complexity of skyline path validation in 2D road networks, they take advantage of the 2D skyline path cost being natural ranked when sorted in weight-increasing order, which does not hold in the higher dimensional space. For instance, in Algorithm 2, when a new path p_i^j is concatenated, its weight must be no less than all the skyline paths in P_s. Thus, we only need to compare the cost of p_i^j with the cost of the ranked

Table 3.2: An Example of 4D Skyline Paths

P_c	w	c_1	c_2	c_3
p_1	2	4	10	2
p_2	2	4	9	3
p_3	3	7	7	10
p_4	5	3	9	3
p_5	6	6	8	4

last path in P_s, which has the minimum cost. However, this skyline validation method is not feasible when each path has multiple costs, as we can not guarantee the ordering of costs for a weight-sorting multi-skyline path set. For example, Table 3.2 has five 4D skyline paths. Even though we sort them by the weight-increasing order, the remaining three costs have no order at all. To reduce the comparison number of skyline validation in paths with multiple costs, we first investigate the order properties of the multi-dimensional skyline paths:

Definition 3.3 (Cost Rank)**.** *Given a set of skyline paths in a n-dimensional graph, we sort them in each of the cost c_i in increasing order. Then $r_i[p_k]$ denotes the rank of path p_k in cost c_i.*

For example, in Table 3.2, we have a set of skyline paths $\{p_1, p_2, p_3, p_4, p_5\}$. On each dimension, we rank the paths with increasing order, which is shown in Figure 3.6 (ignore p_6 at this stage). Then $r_1[p_4] = 1$ because p_4 has the smallest value in cost c_1. Similarly, $r_2[p_5] = 2$ and $r_3[p_4] = 3$. For each path p, it has a rank in each dimension, and the following *distinct criterion* is the one we are most interested in:

Definition 3.4 (Distinct Criterion)**.** *Among the n rankings of a path p in G, its Distinct Criterion is the one with the highest rank denoted as $c_{dc}[p]$.*

For example, p_3's distance criterion is c_2 as it is the smallest in ranking compared to other criteria: $r_w[p_3] = 3$, $r_1[p_3] = 5$, and $r_3[p_3] = 5$. Similarly, the distinct criterion of p_1 is w, $c_{dc}[p_2]$ is c_3, $c_{dc}[p_4]$ is c_1, and $c_{dc}[p_5]$ is c_2. Then we utilise the distinct criterion to further investigate the dominant relation in the high-dimensions:

Lemma 3.12. *Suppose we already have k skyline paths, and we aim to test a new path p_{k+1} with a weight no smaller than the existing ones. After inserting it into the existing criteria sorting lists, we can obtain the ranks m_i of p_{k+1} in each criteria c_i. Suppose p_{k+1} is ranked last amongst all the paths with the same value. Then all the paths $P_i^{>m}$ that have a ranking lower than m_i have a larger value than p_{k+1} in criteria c_i, and they cannot dominate p_{k+1}.*

Proof. Because the dominance relation requires a smaller value, they cannot dominate p_{k+1}. □

For example in Figure 3.6, we insert a new path $p_6 = (8,5,8,9)$. As p_6 has greater weight than the five existing paths, it must be ranked last on dimension w. Then, we find the ranks of p_6 on other dimensions, which are highlighted in red. p_6 is ranked 4, 3 and 5 in c_1, c_2 and c_3. Then along criteria c_2, paths $P_c^{>3} = \{p_2, p_4, p_1\}$ have lower ranking and larger values than p_4, so they cannot dominate p_6. We can have the following lemma by taking the complementary set of Lemma 3.12.

Lemma 3.13. *Following Lemma 3.12, all the paths $P_i^{\leq m}$ that have ranked higher in criteria c_1 than p_{k+1} can possibly dominate p_{k+1}.*

For example in Figure 3.6, $P_1^{\leq 4} = \{p_4, p_1, p_2\}$ can possibly dominate because they rank higher than p_6 along c_1. Now we have the sufficient and necessary requirement for p_{k+1} being a new skyline path:

Theorem 3.14. *Following Lemma 3.13, p_{k+1} is a skyline path if and only if none of the $m-1$ higher ranking paths $P_i^{\leq m}$ in any criteria c_i can dominate p_{k+1}.*

Proof. Because the paths $P_i^{\leq m}$ have either equal or smaller values than p_{k+1} along criteria c_i, then the other paths $P_i^{>m}$ have larger values and cannot dominate p_{k+1}. Therefore, if none of the paths in $P_i^{\leq m}$ can dominate p_{k+1}, then no existing skyline path can dominate p_{k+1}, so p_{k+1} is a skyline path. □

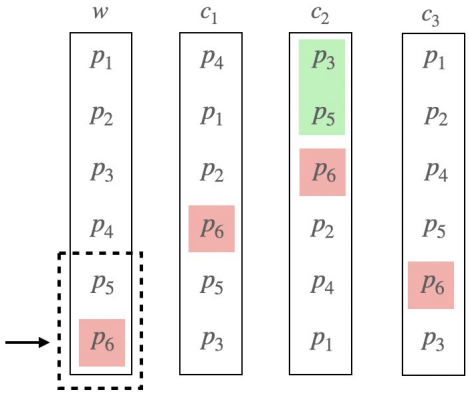


Figure 3.6: Next Skyline Path Validation in Multi-dimensions. New $p_6 = (8,5,8,9)$.

Although any criterion can work for Theorem 3.14, not all of them have the same quality. For instance, along the dimension w, p_{k+1} is ranked last and has k possible dominating paths, so we have to compare p_{k+1} with all k previous paths. Therefore, using the criterion with the least number of possible dominating paths is beneficial, and the distinct criterion $c_{dc}[p]$ is guaranteed to have the smallest $P_i^{\leq m}$ as p_{k+1} ranks highest along it. Then we have the following tighter condition:

Theorem 3.15. *p_{k+1} is a skyline path if and only if none of the $m-1$ higher ranking paths $P_i^{\leq m}$ in the distinct criteria $c_{dc}[p]$ can dominate p_{k+1}.*

The complexity of Theorem 3.15 is $O((|P^{\leq m}_{r_{dc}[p_{k+1}]}| + logk) \times n)$. It requires obtaining all the ranks first and comparing with all the criteria. One way to improve the efficiency is converting the value comparison to the dominating path set intersection:

Theorem 3.16. *Suppose we have obtained all the $P_i^{\leq m_i}$, then p_{k+1} is a skyline path $\Leftrightarrow \bigcap_{i\in[1,n]} P_i^{\leq m_i} = \phi$*

Proof. Suppose $\exists p_i \in \bigcap_{i\in[1,n]} P_i^{\leq m_i}$, then p_i has a higher rank than p_{k+1} in all criteria, so p_i dominates p_{k+1} and p_{k+1} is not a skyline path. □

As the intersection result cannot become larger, we can compute the result in the $|P_i^{\leq m_i}|$ increasing order to keep reducing the intermediate result size. This order can also be used in the value testing method. To further reduce the sorting complexity, we apply the *lazy comparison* only to continue sorting the criteria when needed. We first introduce the hierarchical approximate rank as follows:

Definition 3.5 (Hierarchical Approximate Rank)**.** *Given a set of m sorted numbers S and a query value q, q's rank binary search at level h_0 is $HA_0(q) = m/4$ if $q < S[m/2]$ and $HA_0(q) = m/2$ if $q \geq S[m/2]$. Suppose we are at level h_i and the current HA-Rank is $HA_i(q)$, then for the next level h_{i+1}, we have the following three scenarios: 1) $HA_{i+1}(q) = (2HA_i(q) - 1)/2$ if $q < S[HA_i(q)]$; 2) $HA_{i+1}(q) = HA_i(q)$ if $q \geq S[HA_i(q)]$ and $q < S[HA_{i-1}(q)]$; and 3) $HA_{i+1}(q) = (2HA_i(q) + 1)/2$ if $q \geq S[HA_i(q)]$ and $q \geq S[HA_{i-1}(q)]$. Regardless of the level number, the HA-Ranks are compared with their values.*

The intuition behind it is that we set the *HA-Rank* optimistically. When q is smaller than the value at the current *HA-Rank*, we set it to a smaller *HA-Rank*; when q is not smaller than the value at the current *HA-Rank* but smaller than the previous one, we keep the current *HA-Rank*; when q is not smaller than the values at the current *HA-Rank* and the previous one, we set it to a larger rank.

Now in our case of k n-dimensional skyline paths, we sort them using the *HA-Rank* with query value p_{k+1}. Specifically, each time we update the *HA-Rank* of the criterion that has the highest *HA-Rank* value until one criterion has reached the actual data. Then this criterion is p_{k+1}'s distinct criterion, and we have also got p_{k+1}'s rank on it. We can keep sorting to get the 2^{nd}, 3^{rd} until the last distinct criterion. Algorithm 3 shows the details of the validation using the lazy comparison with *HA-Rank*.

However, the efficiency of Algorithm 3 depends on the number of the paths having higher or equal ranking than the new coming path p_{k+1}. If p_{k+1} ranks very low on distinct criteria, then we may compare a large number of skyline paths, which is still time-consuming. Thus, we provide a pruning technique to identify the non-skyline earlier with the following theorem.

Theorem 3.17. *Suppose we have obtained all the $P_i^{>m_i}$, then p_{k+1} is not a skyline path $\Leftrightarrow |\bigcup_{i\in[1,n]} P_i^{\leq m_i}| < k$.*

Proof. We prove this theorem by contraction. Suppose p_{k+1} is a skyline path and there are $k-1$ skyline paths having a lower ranking than p_{k+1} on at least one criterion. As p_{k+1} has a larger weight

Algorithm 3: Multi-Dimension Skyline Path Intersection Validation

Input: n-Dimensional Skyline Paths $\{p_1, \ldots, p_k\}$, new path p_{k+1}
Output: If p_{k+1} is a skyline path

```
(c, m) ← HA_Sort(p_{k+1});        ▷ c is the distinct criteria and m is its corresponding rank;
P' ← P_c^{≤m};
counter ← 1;
while P' ≠ ϕ and counter < n do
    (c', m') ← HA_Sort(p_{k+1});
    P' ← P' ⋂ P_{c'}^{≤m'};
    counter ← counter + 1;
if P' ≠ ϕ then
    return p_{k+1} is a skyline path
else
    return p_{k+1} is not a skyline path
```

than these k paths, we only need to consider the rankings on the criteria of costs. Except p_{k+1}, k paths are considered in the rankings. Thereby, it must have at least one skyline path p_i, $\forall i \leq k$, having higher or equal ranking than p_{k+1} on all the criteria of costs. Moreover, p_i can dominate p_{k+1}. Since the assumption is false, Theorem 3.17 is true. □

Therefore, when $c_{dc}[p_{k+1}]$ is very low like falling into the last quarter, we can use Theorem 3.17 to do the validation instead of using the intersection. We can further terminate earlier when the distinct criteria are smaller than a threshold according to the following theorem:

Theorem 3.18. *p_{k+1} is not a skyline path* $\Leftrightarrow r_{dc}[p_{k+1}] > k+1-\frac{k}{n-1} = k(1-\frac{1}{n-1})$.

Proof. Because $r_{dc}[p_{k+1}]$ has the highest rank, then p_{k+1}'s ranks on other criteria cannot be higher. In other words, there are at most $k+1-r_{dc}[p_{k+1}]$ paths in each $P_i^{>m_i}$. Therefore, if $(k+1-r_{dc}[p_{k+1}]) \times (n-1) < k$, then $\sum |P_i^{>m_i}| < k \Rightarrow |\bigcup_{i\in[1,n]} P_i^{\leq m_i}| < k$. □

Although Theorem 3.18 seems loose at the first glance, it becomes powerful when k is big and n is small. For instance, when n is 4 and k is 10,000, then when $r_{dc}[p_{k+1}]$ is larger than $10000 \times (1-\frac{1}{4-1} = 6667)$, we can prune p_{k+1} safely.

3.5.2 Multiple Hop Skyline Path Concatenation

In the previous subsections, we have illustrated how to identify all the skyline paths from s to t when h_i is the unique intermediate hop. However, the intersection of s' out-label and t's in-label may contain multiple hops. In this case, the identified skyline paths via h_i could be dominated by the skyline paths via other intermediate hops. Therefore, after we produce all the skyline paths via each intermediate

hop, we need further validation on these path sets. Clearly, it is very time-consuming due to the large concatenation number and time complexity. Therefore, in this section, we introduce some pruning methods to screen the skyline paths via different hops before the actual concatenation to reduce the total concatenation number. We first introduce the *Rectangle Pruning* for *2*-dimensional graph in Section 3.5.2, and then generalise it to *n*-Cube pruning for *n*-dimensional graph in Section 15.

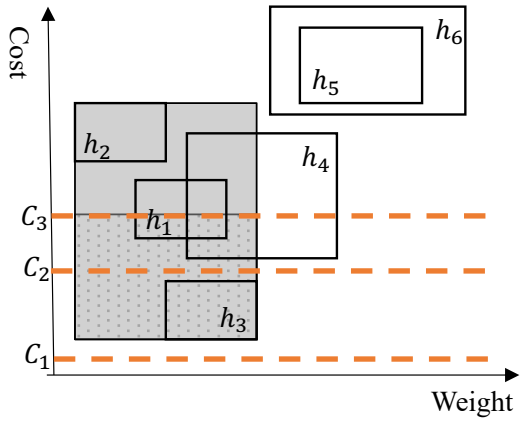


Figure 3.7: Multiple Skyline Rectangles and *CSP* Query Rectangle (Shaded)

Rectangle Pruning for 2D Graph

Given two skyline path sets $P(s,h)$ and $P(h,t)$, according to Property 3.2, p_1^1 and p_m^n are the two endpoints and they can determine a rectangle $[w_1^1, w_m^n] \times [c_1^1, c_m^n]$ (p_1^1 as the top-left and p_m^n as the bottom-right) such that all the other skyline paths must locate in it. Therefore, we can use this rectangle to approximate the skyline path distribution of a hop without computing them. In other words, this rectangle is a *MBR (Minimum Bounding Rectangle)* for all the skyline paths via h. For example, in Figure 3.7, suppose we have the skyline rectangles of six hops and each of them is identified with only two computations for the two endpoints. Amongst these endpoints, we find one with the smallest weight and another with the smallest cost. Then we can obtain a new *hop rectangle* that guarantees that all the skyline results must be inside it (the shaded area formed by the top-left of h_2 and the bottom-right of h_3). We only consider the rectangles of hops that intersect or are inside the *query rectangle* as our final concatenated hops. For instance, as the rectangles h_5 and h_6 are outside the query rectangle, we can prune them directly. For h_4, we further consider the paths with a weight less than $w(p_n.h_3)$. Finally, all the paths in h_1, h_2 and h_3 need actual concatenations. The detailed pruning procedure is described in Algorithm 4. The time complexity is $O(max|X_i|)$ as $max|X_i|$ is the max size of H.

For example, in the label assignment example of Section 19, we first compute three rectangles of the corresponding hops: $h_1 : [6,11] \times [10,5]$; $h_5 : [4,7] \times [6,5]$; $h_8 : (2,3)$. Because $(2,3)$ dominates

the other two rectangles, we prune h_1 and h_5 and return $(2,3)$ directly as the skyline path.

Algorithm 4: Rectangle-based Hop Pruning

Input: The Hop Set H in LCA of s and t
Output: Pruned hop set H'

$TL \leftarrow (\infty,0), BR \leftarrow (0,\infty)$; ▷ Two end-point
foreach $h_i \in H$ **do**
  $P(s,h_i) \leftarrow L(s)[h_i], P(h_i,t) \leftarrow L(t)[h_i]$;
  $p_1^1 \leftarrow p_1(s,h_i) \oplus p_1(h_i,t)$; ▷ Top Left
  $p_m^n \leftarrow p_m(h_i,t) \oplus p_n(h_i,t)$; ▷ Bottom Right
  if $p_1^1.w < TL.w$ **then**
    $TL \leftarrow p_1^1$;
  if $p_m^n).c < BR.c$ **then**
    $BR \leftarrow p_m^n$;
  $Rectangle[h_i] \leftarrow (p_1^1, p_m^n)$;
$R \leftarrow (TL, BR)$;
foreach $h_i \in H$ **do**
  if $R \cap Rectangle[h_i] \neq \phi$ **then**
    $H'.insert(h_i)$
return H';

n-Cube Pruning for n-Graph

Different from the 2D scenario, such rectangles can not be found in the multi-dimensional road networks, as the Property 3.2 cannot hold for the concatenated skyline paths with over two criteria. This is caused by the fact that the path with the largest weight does not necessarily have the largest cost in each dimension. For instance, in Table 3.2, p_5 has the largest w, but it does not have the largest cost in any of the three cost dimensions. Therefore, we can only find a hop rectangle in each 2D subspace, i.e., hop rectangles based on all two criteria combinations. By intersecting the projection of these faces, we obtain a hypercube that encloses all the concatenated skyline paths. For a better description of the pruning in an n-graph, we first map the n-graph to an n-dimensional space: Given an n-dimensional graph $G(V,E)$ and n-dimensional space $D^n = \{d_0, d_1, ..., d_{n-1}\}$, for arbitrary path p, we use dimension d_0 to describe the value of $w(p)$, and d_i to describe the value of $c_i(p)$, where $i \in [1, n-1]$. Now we are ready to describe the n-Cubes.

Definition 3.6 (Skyline Path Set Intervals)**.** *Given a path set P_c and its corresponding space D^n, $\forall d_i \in D^n$, its lower-bound $Min(d_i)$ is the smallest value of all paths $p \in P_c$ on dimension d_i. Similarly, its upper-bound $Max(d_i)$ is the largest value of all paths on d_i. An interval $I_i = [Min(d_i), Max(d_i)]$ covers all the path values in P_c on dimension d_i.*

Definition 3.7 (n-Cube)**.** *Given a path set P_c, an n-cube $\Gamma^n(P_c) = \{I_0, I_1, .., I_{n-1}\}$ covers all the skyline paths in P_c.*

We use the following *supremum* and *infimum* to assist the pruning:

Definition 3.8 (n-Cube Supremum and Infimum)**.** *For an n-cube Γ^n, its supremum $Sup[\Gamma^n(P_c)]$=$\{Max(d_0)$, $..., Max(d_{n-1})\}$ is a set that contains the upper-bounds of P_c on each dimension, and its infimum $Inf[\Gamma^n(P_c)] = \{Min(d_0), ..., Min(d_{n-1})\}$ is a set that contains the lower-bounds of P_c.*

For instance in Table 3.2, there are five skyline paths in P_c. We use d_0 to denote skyline path values on criterion w, and d_1 to d_3 to denote the values on the criteria from c_1 to c_3. As the smallest value on d_0 is 2 and the largest on is 6, we can find a set $I_0 = [2,6]$ to contain the five skyline path values on d_0. By the same token, we obtain $I_1 = [3,7]$, $I_2 = [7,10]$, and $I_3 = [2,10]$. Therefore, we can identify a 4-cube $\Gamma^4(P_c) = \{[2,6],[3,7],[7,10],[2,10]\}$ for the example skyline paths. Note that this 4-cube can cover all skyline path values in Table 3.2. In addition, $Inf[\Gamma^4(P_c)] = \{2,3,7,2\}$ and $Sup[\Gamma^4(P_c)] = \{6,7,10,10\}$. Naturally, we have the following property:

Property 3.5. *Given a set of n-dimensional skyline paths P_c, its n-cube $\Gamma^n(P_c)$ contains all the paths in P_c.*

Similar to the 2D scenario, we first create the n-cube for each hop to approximate the concatenation result space. According to Definition 3.7, to identify an n-cube of hop h_k, we need to compute both the lowest bound and the highest bound on each dimension. Specifically, we compute the concatenated skyline path $p_{Min(d_i)}$ with the smallest value on dimension d_i. Suppose p has the smallest value on d_i in $P(s,h_k)$ and p' has the smallest value on d_i in $P(t,h_k)$. Then $Min(d_i)$ can be obtained by concatenating p and p'. If $p_{Min(d_i)}$ is a unique path that has the smallest value on d_i, we can easily prove that $p_{Min(d_i)}$ must be a concatenated skyline path. Thus, the d_i value of $p_{Min(d_i)}$ must be the lower-bound of d_i for all the concatenated skyline paths. We can use the same method to compute the upper-bound by finding $Max(d_i)$ in both $P(s,h_k)$ and $P(t,h_k)$. However, different to the 2D case, we can not guarantee that the concatenated path with the largest value on d_i is an n-dimensional skyline path. Therefore, the obtained upper-bounds are actually looser than the actual bounds. For ease of presentation, we refer to n-cube of the concatenated skyline path set via h_k as $\Gamma(h_k)$, and $\Gamma(h_k) \supseteq \Gamma(P(s,h_k) \oplus P(h_k,t))$. Also, we refer to $Sup[\Gamma^n(h_k)]$ and $Inf[\Gamma^n(h_k)]$ as $Sup(h_k)$ and $Inf(h_k)$, if no special instructions on dimensions are needed.

Because the n-cube is a hypercube, it is difficult to clearly demonstrate a range of skyline paths in an example plot as seen in the hop rectangle in 2D. Nevertheless, if we treat the dimensions as x-coordinates and supremum/infimum values on each dimension as y-coordinates, we are able to map an n-cube to a 2D plot in the form of two polylines. For instance, Figure 3.8 (left) shows four 4-cubes,

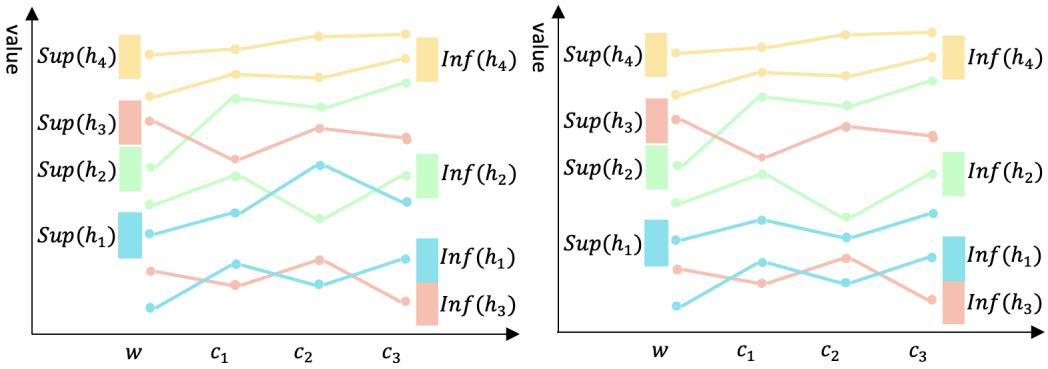


Figure 3.8: Supremum and Infimum for Hops (left) and Supremum Shrinking(right)

each represented by two polylines with the same colour. In the following, we discuss the n-cube pruning with the help of Figure 3.8.

Property 3.6. ***(n-Cube Dominance)*** *Given any two n-cube Γ_i and Γ_j, if $Sup[\Gamma_i]$ dominates $Inf[\Gamma_j]$, then Γ_i dominates Γ_j and all the paths in Γ_j are also dominated by the paths in Γ_i .*

For example in Figure 3.8, the polyline $Sup(h_4)$ is over the supremum of other three 4-cube. Therefore, all the paths in $\Gamma(h_4)$ have greater value on each dimension than the paths in other 4-cubes. However, as the polylines of other three 4-cubes are intersected with each other, they need further pruning.

As discussed previously, the infimum of each estimated hop's n-cube $\Gamma(h_i)$ is fixed, but the supremum is loose. Therefore, it is promising to further prune more hops by shrinking the supremums, which could be achieved by concatenating the skyline paths on some hops earlier. For instance in Figure 3.8, if we first compute all the concatenated skyline paths on h_1, we can obtain a fixed $Sup(h_1)$, which is shrunken to the position in Figure 3.8 (right). Thus, $\Gamma(h_2)$ can be pruned directly by $\Gamma(h_1)$. Thereby, it is important to determine an efficient concatenating order for the hops, which is defined as follows.

Definition 3.9 (n-Cube Comparison)**.** *Given two of n-Cubes Γ_i and Γ_j that cannot dominate each other, we use $|Inf^{<}[\Gamma_i]|$ to denote the number of dimensions where $Inf[\Gamma_i]$ is smaller, and $|Inf^{<}[\Gamma_j]|$ to denote Γ_j's. Then we say $\Gamma_i < \Gamma_j$ if $|Inf^{<}[\Gamma_i]| > |Inf^{<}[\Gamma_j]|$. The draw of comparison can be settled by comparing the supremum. The further draw is settled by comparing infimum dimensions in decreasing order, and the one with the first smaller dimension lower-bound is smaller.*

The intuition behind this order is that if a hop has a lower infimum, it has a higher chance to contain skyline paths. Especially, the hop with the lowest infimum must contain skyline paths. Moreover, an n-cube with a lower infimum has a higher chance to prune more n-cubes after it shrinks. Based on this observation, we have the following order of n-cubes.

Definition 3.10 (n-Cube Concatenation Priority)**.** *Given a set of n-Cubes that cannot dominate each other, their concatenation order is the same as their increasing infimum order.*

For instance, because all the lowest bounds of $Inf(h_1)$ are lower than $Inf(h_2)$, we need to concatenate skyline paths on h_1 before h_2. $\Gamma_1 < \Gamma_3$ because $|Inf^{<}[\Gamma_1]| = |Inf^{<}[\Gamma_3]|$ but $|Sup^{<}[\Gamma_1]| > |Sup^{<}[\Gamma_3]|$. Finally, $\Gamma_3 < \Gamma_2$ because $|Inf^{<}[\Gamma_3]| < |Inf^{<}[\Gamma_2]|$. Therefore, the three 4-Cubes should be concatenated in the order of Γ_1, Γ_3, and Γ_2. It should be noted that this order can be changed dynamically when any n-cube is pruned or when any supremum is fixed. The detailed procedure is shown in Algorithm 5.

Algorithm 5: n-Cube Hop Pruning

Input: The Hop Set H in LCA of s and t
Output: Skyline Paths Set $P_{s,t}$

$P_{s,t} \leftarrow \phi$;
foreach $h_i \in H$ **do**
  foreach $d_j \in D^n$ **do**
    $Inf[\Gamma_i][j] \leftarrow p_1(s,h_i).d_j + p'_1(h_i,t).d_j$;
    $Sup[\Gamma_i][j] \leftarrow p_n(s,h_i).d_j + p'_m(h_i,t).d_j$;
  $\Gamma_i \leftarrow (Inf[\Gamma_i], Sup[\Gamma_i])$;
$\hat{\Gamma} \leftarrow$n-Cube-Sort($\{\Gamma_i\}$);
$Sup[\hat{\Gamma}] \leftarrow [-1] \times n$;
foreach $\Gamma_i \in \hat{\Gamma}$ **do**
  if $Sup[\hat{\Gamma}]$ *dominates* $Inf[\Gamma_i]$ **then**
    Continue;
  $P_{s,t} \leftarrow Skyline(P_{s,t}, P(s,h_i) \oplus P(h_i,t))$;
  foreach $d_j \in D^n$ **do**
    if $Sup[\hat{\Gamma} < Max(P_{s,t}.d_j)$ **then**
      $Sup[\hat{\Gamma} \leftarrow Max(P_{s,t}.d_j)$;
return $P_{s,t}$;

Finally, the rectangle pruning in the 2D scenario can be viewed as a special case of the n-Cube pruning with $n = 2$. Although its supremum is fixed initially, its performance can also be improved with the 2-Cube concatenation priority.

3.5.3 Constraint Skyline Path Concatenation

This section further digs into the concatenation when the query constraints $\overline{C}$ are provided. Similar to the previous sections, we also first discuss the easier 2D case and then generalise it to the n-dimensional case.

CSP Concatenation

We first deal with the *CSP* query $q(s,t,C)$. As illustrated in Figure 3.7, the query rectangle is further reduced to a smaller one by applying an upper-bound of C (like the dotted area created by C_3). If C is smaller than the lowest cost (like C_1), then the result is empty. If there exists one rectangle whose upper-left point is the only left most point in the query rectangle (like C_2 and h_3), we return this point directly. Otherwise, there must be some partial rectangles. We first find the most upper-left point in the query region, denote it as p^* and use it for the current best point (w^* as its weight). Then for all the partial rectangles whose left boundary is larger than w^*, we can prune them directly (e.g., h_4 is pruned by h_3). After that, we enumerate the candidates in the remaining partial rectangles (like h_1) in a similar way like Algorithm 2. The difference is we only start testing when the sub-path's cost is smaller than C, and we stop if either the current minimum weight is larger than w^* or we obtain the first one whose cost is no larger than C. Traversal from multiple partial hops can run in parallel as they have the same termination condition and do not interfere each other.

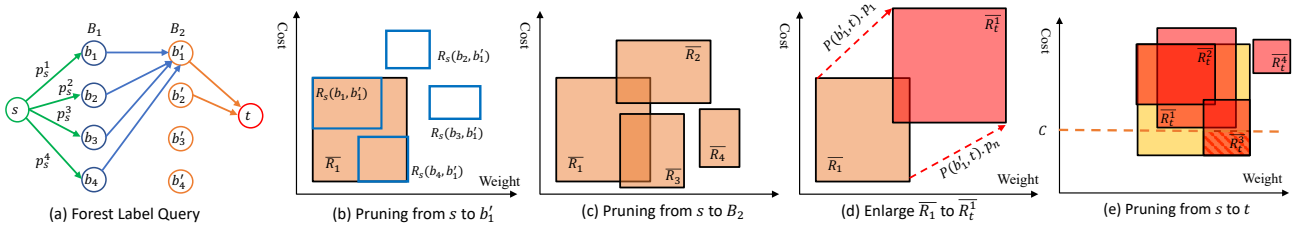


Figure 3.9: Constraint Shortest Path Query with Forest Labels

MCSP Concatenation

Different from the n-Cube in Section 15, the supremums that are larger than the constraint $\overline{C}$ can be replaced by $\overline{C}$ during query answering. Besides, an n-Cube can be further pruned by the following property.

Property 3.7. *Given an n-Cube and a set of query constraints $\overline{C}$, the n-Cube can be pruned if any of its infimums is larger than the corresponding cost.*

Then for the concatenation of each n-Cube, because the paths are concatenated in the distance-increasing order, we only need to concatenate the first one that satisfies $\overline{C}$ for each hop. Within each hop concatenation, the paths in the labels are also pruned when any of its cost does not satisfy $\overline{C}$. When we have the first candidate, its distance can also be used to prune the remaining concatenations for other hops.

3.5.4 Forest Constraint Skyline Path Concatenation

Rectangle Pruning for 2D Graph

This section presents how to apply the above techniques to answer *MCSP* with forest labels. The extended labels can use the methods in Section 3.5.3 directly. As for the forest labels, we break it into three parts as shown in Figure 3.9-(a): 1) from s to its boundary B_1, 2) from B_1 to B_2, and 3) from B_2 to t. However, different from the previous query that needs concatenation, the skyline results of 1) and 3) can be retrieved from their sub-trees directly because their boundaries are all their ancestors. As for the boundary results, we first obtain their rectangles, then obtain the approximate results from s to B_2 by viewing each boundary in B_1 as a hop. For example in Figure 3.9-(b), we get a candidate rectangle $\bar{R_1}$ from s to b'_1 by pruning the multi-hop of B_1. By applying the same procedure to the boundaries B_2 in parallel, we can have a set of candidate rectangles as shown in Figure 3.9-(c). After that, these rectangles are "enlarged and moved" to the new locations by viewing the first and last skyline paths of each B_2 to t as a vector as shown in Figure 3.9-(d). Then we have a final set of rectangles $\bar{R}_t$ that approximates the skyline path space from s to t. Finally, we can apply the constraint C to shrink the search to a small area (shaded area in Figure 3.9-(e)) and conduct the actual concatenation similarly as in Section 3.5.3.

n-Cube Pruning for n-Graph

Similar to the 2D scenario, the query processing is the same as the steps in Figure 3.9-(a). However, the rectangle pruning is displaced by n-cube pruning. When pruning from s to b'_1, we construct an n-cube for each hop in B_1. According to Property 3.6, we can prune the dominated n-cubes. Then, we approximate the concatenation result space from s to b'_1 by updating an infimum and a supremum depending on the undominated n-cubes in B_1. If it is the case, we compare their infimums to find the lowest value on each dimension and compare their supremums to find the highest values. For example in Figure 3.9-(a), assume we are considering 4-cube. For the four hops in B_1, we have $\Gamma_1 = \{[4,7],[5,7],[5,8],[1,9]\}, \Gamma_2 = \{[3,6],[4,6],[4,7],[3,10]\}, \Gamma_3 = \{[8,12],[8,10],[9,12],[10,13]\}$, and $\Gamma_4 = \{[7,12],$
$[8,10],[9,14],[12,14]\}$, Γ_3 and Γ_4 can be pruned in terms of Property 3.6. By viewing the infimums and supremums of Γ_1 and Γ_2, a new infimum from s to b'_1 is $\{3,4,4,1\}$ and a new supremum is $\{7,7,8,10\}$. Afterwards, we enlarge the n-cube from s to b'_1 by adding on the infimum and the supremum of the n-cube from B_2 to t. For instance, assume the n-cube from b'_1 to t is $\Gamma'_1 = \{[3,5],[2,6],[5,9],[1,4]\}$, we can get the concatenation result space from s to b'_1 to t as n-cube, which has the infimum $\{6,6,9,2\}$ and the supremum $\{12,13,17,14\}$. We can use the same way to compute the n-cubes from s to t via other hops in B_2. We prune the dominated n-cubes from s to t and update

Table 3.3: Partition Method Comparison (The best values are shown in bold). The number after each network is the partition number and the average partition size. $|G_i^B|$ is the average boundary number of each partition, $|B|$ is the total boundary size, and σ is the partition boundary's standard deviation.

Network	NY(75/3523)			BAY(74/4341)			COL(83/5249)			FLA(124/8632)		
	$\|G_i^B\|$	$\|B\|$	σ	$\|G_i^B\|$	$\|B\|$	σ	$\|G_i^B\|$	$\|B\|$	σ	$\|G_i^B\|$	$\|B\|$	σ
HiTi	96.27	9627	67.03412	596.53	59653	595.3152	615.87	61587	872.4065	842.43	84243	1307.488
METIS	55.50667	4163	25.87045	44.17568	3269	**18.74585**	46.93976	3896	25.44068	48.46774	6010	20.41259
SCOTCH	54.42667	4082	22.21831	42.94595	3178	18.51315	41.45783	3441	24.69487	49.37903	6123	23.93813
Natural Cut	**32.89333**	**2467**	**20.60213**	**36.28378**	**2685**	20.54432	**23.50602**	**1951**	**19.03038**	**24.15323**	**2995**	**13.30141**

a infimum and a supremum as the concatenation result space from s to t by viewing the remaining n-cubes. We can apply the constraints $\overline{C}$ as the final supremum to shrink the n-cube from s to t.

Block Maintenance

Given two sets of m and n skyline paths, the concatenation paths number mn could be large and deteriorates the cube concatenation efficiency. To reduce mn to a smaller value, we first introduce the *path entropy* [99].

Definition 3.11 (Path Entropy ε). *Given an n-D path $p \in P_s$, its entropy $\varepsilon(p) = lnw(p) + \sum_{i=1}^{n-1} \ln c_i(p)$. If $\varepsilon(p)$ is small, then p has a higher probability to dominate paths in P_s.*

Specifically, we maintain a *Block* of k smallest entropy concatenated paths. For each $p_i \in P_s$, we can discard it if dominated by any path in the *Block*. Otherwise, we compute its entropy $\varepsilon(p_i)$ and replace p_j in *Block* if $\varepsilon(p_j) > \varepsilon(p_i)$. In addition, the *Block* maintenance can be further optimised in the *tree node contraction* phase where the skyline paths are concatenated when we compute skyline shortcuts between the neighbour pairs of each vertex w. If an edge $e = (u, v)$ exists between w's two neighbours u and v, then it already includes a skyline path set $P_{u,v}$, and we need to merge the concatenated paths $P_{u,w,v}$ with $P_{u,v}$. As $|P_{u,w,v}|$ can be much larger than $|P_{u,v}|$, we use the k smallest entropy paths in $P_{u,v}$ as the initial *Block*. In practice, this method is effective, especially in the boundary tree.

3.6 Experiment

3.6.1 Experimental Settings

We implement all the algorithms in C++ with full optimisations. The experiments are conducted in a 64-bit Ubuntu 18.04.3 LTS with two 8-cores Intel Xeon CPU E5-2690 2.9GHz and 186GB RAM.

Datasets. All the tests are conducted in three real road networks obtained from the 9^{th} DIMACS Implementation Challenge [4]. Specifically, *NY* is a dense grid-like urban area with 264,246 vertices, 733,846 edges, and several partitions connected by bridges. *BAY* has a shape of a doughnut around the

[4] http://users.diag.uniroma1.it/challenge9/download.shtml

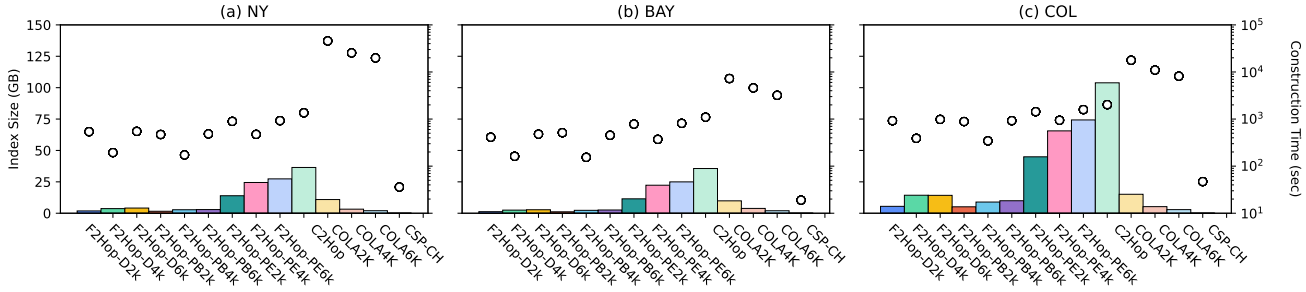


Figure 3.10: Index Size (Bar) and Construction Time (Ball) of Different Partition Sizes, Boundary Contraction Orders, and Algorithms in 2-Graph

San Francisco Bay with 321,270 vertices, 800,172 edges, and several bridges connecting the banks. *COL* has an uneven vertex distribution (dense around Denver but sparse elsewhere) with 435,666 vertices and 1,057,066 edges. *FLA* is the network Florida with 1,070,376 vertices and 2,712,798 edges. We use *FLA* in the scalability test in Section 3.6.4 because it is much larger than the other three. We use the road length as the weight w and travel time as the first cost to test the performance of *CSP*. We also generate two cost sets that have random and negative correlations with the distance for each network, and we generate three more positive correlation costs for *MCSP* tests.

To test our algorithms on the road network with real-world criteria, we collect real-world data *NY-REAL* with five criteria: distance, travel time, toll charge, the number of traffic lights, and attractiveness. The data on distance and travel time is the same as *NY*. For the toll charge, we collect New York City data from *OpenStreetMap* and identify the road types for the edges in *NY* based on their coordinates. We assign the real toll fees for the edges identified as bridges and tunnels. For computing the number of traffic lights on each edge, we collect all the coordinates of traffic lights in New York City from *OpenStreetMap* and map them carefully on the edges in *NY*. Finally, referring to the data collection method in [100], we calculate the attractiveness for each edge based on the geo-tagged *Flickr* photos in New York. We use the number of *Flickr* photos associated with the edges to evaluate their attractiveness. For each edge, we count the photos taken within 1 kilometre from either start or end of the edge and fetch the minimum one as its attractiveness value. We discuss the performance on *NY-REAL* in Section 3.6.4.

Algorithms. We extend the following state-of-the-art exact *CSP* algorithms to their *MCSP* versions [101]: *Sky-Dij* [20], *cKSP* [68], *eKSP* [24], *ARSC* [17], *SHARC* [102], *CSP-CH* [30], and *COLA* [38] with α=1. We denote our methods as *C2Hop* and *F2Hop*. The straightforward version (as previously implemented in [39]) is denoted as *F2Hop-S*.

Query Set. For each dataset, we randomly generate five sets of OD pairs Q_1 to Q_5 and each has 1000 queries. Specifically, we first estimate the diameter d_{max} of each network [103]. Then each Q_i represents a category of OD pairs falling into the distance range $[d_{max}/2^{6-i}, d_{max}/2^{5-i}]$. For example,

Q_1 stores the OD pairs with distances in the range $[d_{max}/32, d_{max}/16]$ and Q_5 are the OD pairs within $[d_{max}/2, d_{max}]$. Then we assign the query constraints C to these OD pairs. Firstly, we compute the cost range $[C_{min}, C_{max}]$ for each OD. This is because if $C<C_{min}$, the optimal result can not be found, thus invalidating the query; and if $C>C_{max}$, the optimal result is the path with the shortest distance. Furthermore, in order to test the constraint influence on the query performance, we generate five constraints for each OD pair: $C = r \times C_{max} + (1-r) \times C_{min}$, with $r = 0.1, 0.3, 0.5, 0.7$ and 0.9. When r is small, C is closer to the minimum cost; and when r is big, C is closer to the maximum cost. This constraint generation method is applied on all the cost dimensions.

Partition Method. We test the following three widely used road network partition methods: *HiTi* [81, 104], *METIS* [105] that is used in *ParDiSP* [106] and *G-Tree* [107], *SCOTCH* [108], and *Natural Cut* [79] used in the *Customizable Routing* [109]. For each network, we assign the same sub-graph number to these algorithms as the partition parameter, and we compare the following result properties: 1) *Average Boundary Number in Partition* $|G_i^B|$ that is the smaller the better, 2) *Boundary Vertices Number* $|B|$ that is smaller the better because it would lead to smaller boundary tree, and 3) *Partition Boundary Size Standard Deviation* σ that is also the smaller the better because it has a more balanced inner tree construction (the partition vertices sizes are similar, but each partition has $|G_i^B|^2$ new edges to form the complete boundary graph to guarantee the correctness. Therefore, it is the boundary size's variation that actually determines construction balance when the vertex numbers are similar). The partitioning results are shown in Table 3.3. Firstly, all these methods have the same average partition size because they are all edge-cut methods and have no repeated vertices. Secondly, Natural Cut always has the smallest boundary vertices and average boundary number among all the methods, especially when the network covers a larger area. This is because while the other methods mainly focus on cutting the regions in balance, the *Natural Cut* mainly focuses on finding those cuts like bridges and highways to reduce the boundary number. Finally, *Natural Cut* also has the smallest partition boundary deviation. Therefore, we use *Natural Cut* as our underlying partitioning method to achieve more balanced inner tree constructions and a smaller boundary tree. Furthermore, the influence of the partition sizes is tested in Section 3.6.2.

3.6.2 Index Size and Construction Time

In this section, we first test the influence of the partition sizes and boundary contraction orders on index size and construction time. We use the 2-Graph for the test and select the best ones for the higher dimensional tests. Specifically, we test the *F2Hop*s and *COLA* with three different partition sizes for each graph ($2k, 4k$, and $6k$), *C2Hop* and *CSP-CH* with no partition. In terms of contraction order, *F2Hop-D* uses the degree elimination order with boundary label, *F2Hop-PB* uses the partition order with boundary label, and *F2Hop-PE* uses the partition order with the extended label. As shown

in Figure 3.10, *F2Hop* all have smaller index sizes and shorter construction time than *C2Hop* because of the smaller tree sizes and parallel construction. Specifically, among the *F2Hop*s, *F2Hop-PB* is the fastest and smallest *F2Hop* because it has the smallest number of labels to construct, and *F2Hop-PE* is the largest and slowest because it has the largest number of labels. It also shows that the partition order performs better than the degree elimination order in the boundary tree. Among the baseline methods, *COLA* also has a small index but takes the longest to construct because it uses *Sky-Dijk* search. *CSP-CH* takes the shortest time to construct the smallest index. The partition's influences are also reflected by the index size and index construction time. The smaller partition size would increase the partition number and the boundary number. Therefore, although the smaller partition size has faster individual inner tree construction, it has more inner trees to construct, and what's more, it has a bigger boundary tree to construct. On the other hand, the larger partition size would decrease the partition number and boundary number slightly, but each inner tree construction takes longer time while the boundary tree does not decrease dramatically. The partitioning quality standards on boundary vertices are also used by [102, 109] to select the partition method. In In the following tests, we use $4k$ as the default partition and *PB* as the boundary order and boundary label.

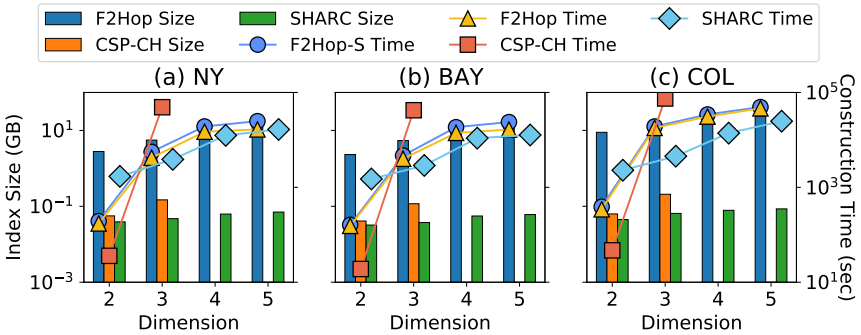


Figure 3.11: Index Size (Bar) and Construction Time (Ball) of n-Graphs

Next, we test the index size and construction time in the higher dimensional graphs and the results are shown in Figure 3.11. Because *COLA* already takes more than 10^4 seconds in the 2-dimensional graph, we do not test it in the higher-dimensional graphs. As for the *F2Hop* and *F2Hop-S*, they construct the same index, so their index sizes are the same and we show them together as *F2Hop Size*. As for the construction time, we distinguish them because they use different strategies. Specifically, *F2Hop* is faster than *F2Hop-S* by less than $10\times$. *CSP-CH* is faster in 2-dimensional but the construction time soars up in 3-dimensional, and takes longer than 10^5 in higher dimensions so we do not show *CSP-CH*'s performance on 4 and 5-graphs. This is caused by the explosion of the skyline path numbers on the entire graph, while our *F2Hop* methods reduce it with forest. As for the construction time *F2Hop*s, 3-graph takes an order of magnitude longer time than 2-graph, and 4-graph takes another order of magnitude longer time than 3-graph. This phenomenon demonstrates the influence of dimension

number of costs on the *MCSP* problem. However, compared with the 2 to 3 and 3 to 4, 5-graph's construction time is longer but is still in the same order of 4-graph. This is because the increase of skyline path slows down in higher dimensions. Finally, *SHARC* is the fastest to construct in higher dimensions with the smallest index size.

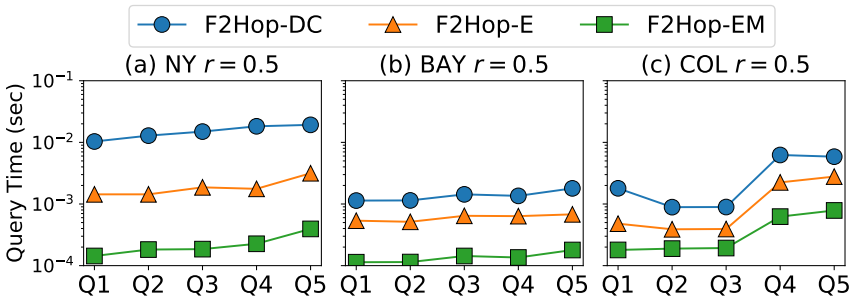


Figure 3.12: Effectiveness of Skyline Concatenation of CSP

3.6.3 Effectiveness of Skyline Concatenation

In this section, we study the effectiveness of our proposed skyline concatenation methods: *F2Hop-DC* uses the classic *Divide-and-Conquer*, *F2Hop-E* uses our *Efficient concatenation* for *CSP*, and *F2Hop-EM* further applies the *Multi-hop pruning* based on *F2Hop-E*. As shown in Figure 3.12, *F2Hop-E* is faster than *F2Hop-DC*, while *F2Hop-EM* is the fastest. This validates the effectiveness of our concatenation methods. Therefore, we only use *F2Hop-EM* as the concatenation algorithm in the latter experiments. Among the networks, although *NY* is relatively small, it is denser than the other two so the query performance is worse. *BAY* is always faster because it has smaller label size thanks to its ring-shape topology. *COL*'s Q_1 is worse than its Q_2 and Q_3 because the shorter queries have higher chance to fall into the dense Denver city and the mid-range query have higher chance to fall into the mountains and canyons.

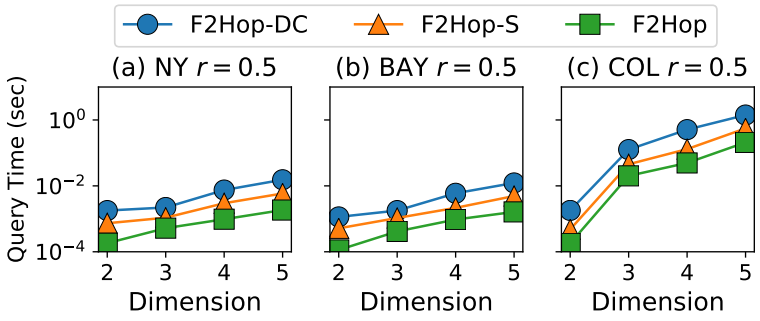


Figure 3.13: Effectiveness of Skyline Concatenation in MCSP

For the higher dimensional cases, we further test our *MCSP* concatenation techniques and compare with the techniques for *CSP*. The query sets are Q_3 with $r = 0.5$. As shown in Figure 3.13, we can further achieve a nearly one order of magnitude faster concatenation speed.

3.6.4 Query Performance

Query Distance

Figure 4.9 (a)-(c) show the query processing time of different algorithms in distance-increasing order with $r = 0.5$. Firstly, all *CSP-2Hop* methods outperform the others by several orders of magnitude. Among them, *C2Hop* is the fastest that runs in 10^{-5} seconds because it has the largest label size and only needs one multi-hop concatenation. *F2Hop-PE* is only slightly slower because it also has a large index size. *F2Hop-PB* is slower running in 10^{-3} seconds but it has the smallest index size and is the fastest to construct. Secondly, all the algorithms' running time increase as the distance increases. Compared to the other methods, the *CSP-2Hop* methods are more robust as their running time only increases slightly. Among the comparing methods, when the ODs are near to each other, like Q_1 and Q_2, all of them can finish in 10 seconds. When the distance further increases, some of them soar up to thousands of seconds. Specifically, *cKSP* is always the slowest one because it uses the pruned *Dijkstra* search to generate and test the k shortest paths in the distance-increasing order. Therefore, when the OD is far away from each other, there could enumerate a huge number of possible paths. The second slowest is *Sky-Dij*, which has a huge amount of intermediate skyline paths to expand before reaching the destination. *eKSP* is slightly faster than them because it avoids the expensive *Dijkstra* search and uses path concatenation instead. *CSP-CH* is faster as expected, but its performance also deteriorates dramatically in Q_4 and Q_5. Finally, *COLA* also tends to be stable because its query time is bounded by the inner-partition search, and inter-boundary results are precomputed regardless of the actual distance. In summary, our *CSP-2Hop* is orders of magnitude faster and also robust in terms of different distances.

Constraint Ratio

Figure 4.9 (d)-(f) show the influence of the constraint ratio r, there are several interesting trends. Firstly, *CSP-2Hop* is still robust and fastest under different r. Secondly, the baseline methods fall into two trends: *Sky-Dij* and *CSP-CH* that grows as the r increases, while *cKSP* and *eKSP* have a hump shape. It should be noted that because the time is shown in the logarithm, a very small change over 10^2 in the plot actually has a very large difference. The reason for the increasing trend is that *Sky-Dij* and *CSP-CH* expand the search space incrementally until they reach the destination, during which they accumulate a set of partial skyline paths. Therefore, when the r is small, the partial skyline path set is

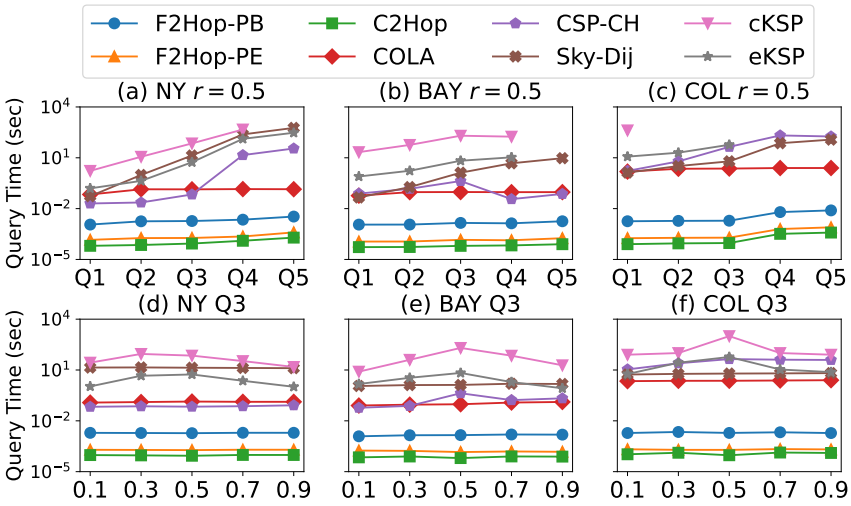


Figure 3.14: CSP Query Performance of Distance and Ratio

also small because most of the infeasible ones are dominated. When r is big, the partial skyline path set becomes larger so it runs slower. As for the hump trend, the two *KSP* algorithms test the complete paths iteratively in the distance-increasing order. Therefore, when r is big, more paths would satisfy this constraint, so there is a higher chance to enumerate only a few paths to find the result. In fact, they are even faster than *CSP-CH* in Q_4 and Q_5 when $r = 0.9$ (not shown here). On the other hand, when r is small, large amounts of infeasible paths are pruned during the enumeration so the running time is also short. But when r is not big or small, the pruning power becomes weak and the actual result is much longer than the shortest path, they have to enumerate tens of thousands of paths to find it, so query performance deteriorates.

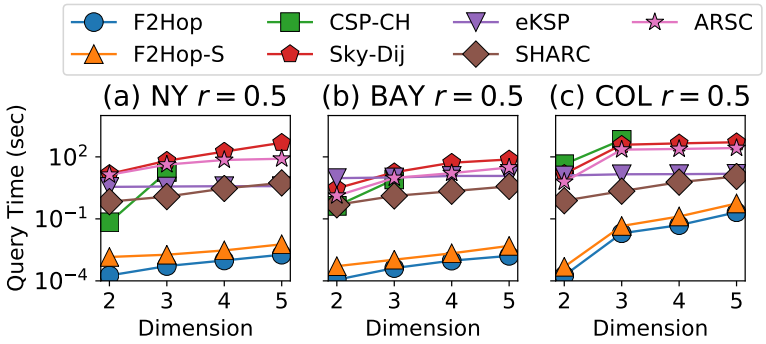


Figure 3.15: MCSP Query Performance

MCSP Query

We test the *MCSP* queries with Q_3 and $r = 0.5$. As shown in Figure 4.8, *F2Hop* are still orders of magnitude faster than the baseline. The performance of *eKSP* looks stable because it is an enumeration-

based method and we terminate the enumeration when more than $10k$ paths have been tested. Its current performance is at the cost of failing to answer some queries and is bounded by $10k$ enumeration. *ARSC* is faster than *Sky-Dij* because it has a smaller search space but it is still orders of magnitudes slower than our approach. With the number of dimensions increasing, *CSP-CH* has high complexity on skyline domination, and the number of skyline paths increases rapidly in the contraction phase. In this case, It is impractical on the index construction when the number of dimensions exceeds 3. Hence, we only show the query performance of *CSP-CH* on networks with less than four dimensions. Moreover, the increasing number of skyline paths in shortcuts seriously reduces the query performance of *CSP-CH*. Finally, *SHARC* is generally faster than the other baselines but it is still nearly 100× slower than *F2Hop*.

Cost Correlation

We use the 2-dimensional index size and construction time to demonstrate the influence of the correlation. As shown in Figure 3.16, the positive correlation has the smallest index size and smallest construction time, and the negative correlation has the largest index size and longest construction time. This phenomenon is also caused by the *skyline* itself instead of our index structure. When the criteria have negative correlations, the skyline number tends to increase to the worst square case. As a result, the query time increases dramatically for the baselines, while ours only increases slightly.

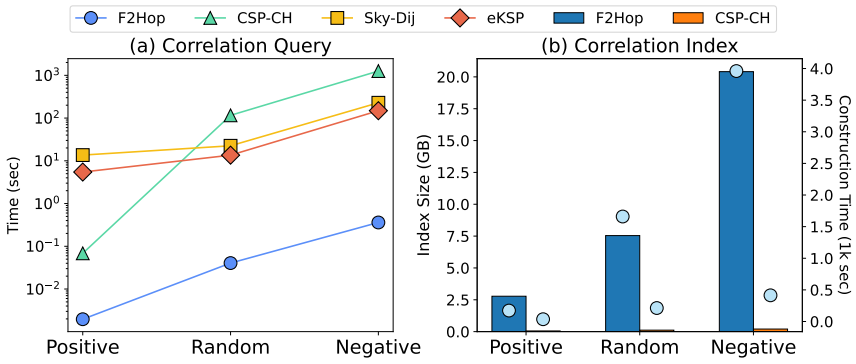


Figure 3.16: Performance under Different Correlations (Bar: Size; Ball: Time)

Real World Criteria

We test the *MCSP* queries with Q_3 and $r = 0.5$ on *NY-REAL*. We construct a 2-graph index including distance and travel time. Then, we add toll charges to build a 3-graph index. The index size of *F2Hop* slightly increases from the 2-graph to the 3-graph as only a few edges are assigned with toll fee, which results in a small number of skyline paths produced subsequently. We add the number of traffic lights as the fourth criterion to build a 4-graph index. As only thousands of vertices are identified as traffic

lights, this criterion also has less impact on the skyline index size of *F2Hop*. However, when we add attractiveness value for constructing a 5-graph, the index size and construction time of *F2Hop* soar up. It is because the assignment of attractive value tends to be random on edges, which results in plenty of skyline paths generated. In addition, the index construction of *CSP-CH* becomes impractical on the 5-graph. As for *SHARC*, it is the fastest to construct with the smallest index, and it is faster to answer queries than the other baselines. On the query performance, *F2Hop* is still efficient and stable on the road networks without considering attractiveness, but its query efficiency reduces very quickly on the 5-graph. We can see that this situation also happens to all the other competitors.

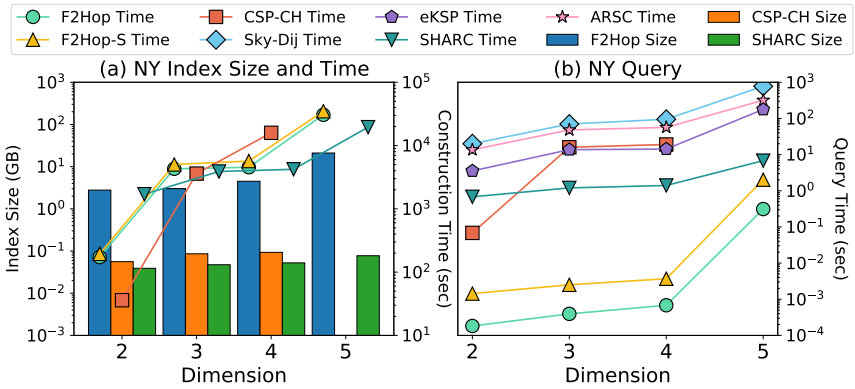


Figure 3.17: Real World Criteria Test (Bar: Size; Ball: Time)

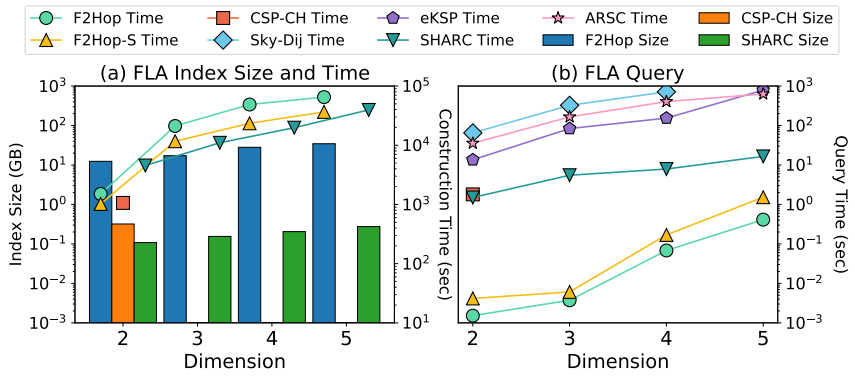


Figure 3.18: Scalability Test (Bar: Size; Ball: Time)

Scalability Evaluation

As shown in Figure 3.18, our *F2Hop* can scale well up to 5 dimensions on the large *FLA* network with index size under 30 GB. As for *CH*, it can construct on two dimensions but cannot scale to higher dimensions. As for *SHARC*, it is still the fastest to construct with the smallest index, and it has the fastest query time among the baselines. In terms of query processing, our *F2Hop* can still answer the

worst case under one second, which is still at two orders of magnitude faster than all the baseline and acceptable in real life.

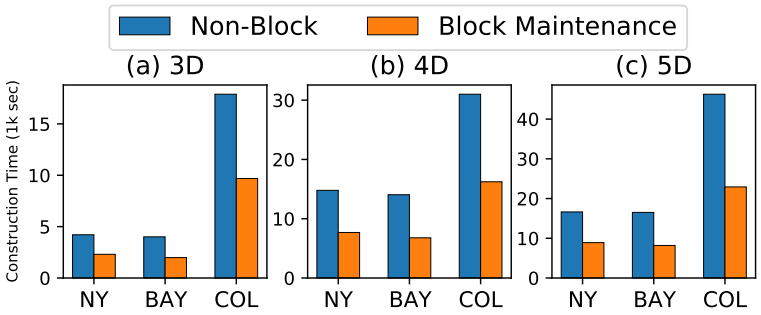


Figure 3.19: Effectiveness of Block Maintenance

Effectiveness of Block Maintenance

In this section, we show the effectiveness of *Block* on $3D$ to $5D$ graphs. The pruning method significantly reduces the time cost of index construction. It is because the *Block* reduces many concatenated paths before our domination operations. Thus, it can save time on domination operation, as each concatenated path may compare with fewer other candidates. On the other hand, we do not adopt *Block* on $2D$ graph because we have a more efficient method to ensure that each concatenated path only compares with another path once in our domination operation.

3.7 Conclusion

In this paper, we have proposed a forest-based skyline path index structure to answer the *MCSP* query efficiently. Moreover, we have proposed several skyline path concatenation methods to support index construction and query answering. The new path concatenation paradigm enables indexing and storing the complete exact skyline paths. Several pruning techniques have been developed for single-pair concatenation, multiple-hop concatenation, *MCSP* query answering, and forest query answering. Extensive evaluations on real-life road networks have demonstrated that our new methods can outperform the state-of-the-art *MCSP* approaches by several orders of magnitude. Moreover, our pruning techniques can provide a further 10x speedup.

The following publication has been incorporated as Chapter 4.

1.**Ziyi Liu**, Lei Li, Mengxuan Zhang, Wen Hua, and Xiaofang Zhou, FHL-Cube: Multi-Constraint Shortest Path Qerying with Flexible Combination of Constraints, *Proceedings of the VLDB Endowment*, Issue 11, pp 3112–3125, 2022

Contributor	Statement of contribution	%
Ziyi Liu	Conception and design	70
	Analysis and interpretation	60
	Drafting and production	65
	proof-reading	55
Lei Li	Conception and design	10
	Analysis and interpretation	10
	Drafting and production	15
	supervision, guidance	50
	proof-reading	15
Mengxuan Zhang	Analysis and interpretation	10
	Drafting and production	10
	proof-reading	15
Wen Hua	Conception and design	10
	Analysis and interpretation	10
	Drafting and production	10
	supervision, guidance	50
	proof-reading	15
Xiaofang Zhou	Conception and design	10
	Analysis and interpretation	10

Chapter 4

Flexible Multi-Constraint Shortest Path Querying

Multi-Constraint Shortest Path (*MCSP*) generalises the classic shortest path from single to multiple criteria to satisfy more personalised needs. However, *MCSP* query is essentially a high-dimensional skyline problem and thus time-consuming to answer. Although the current *Forest Hop Labelling (FHL)* index can answer *MCSP* efficiently, it takes a long time to construct and lacks the flexibility to handle arbitrary criteria combinations. In this paper, we propose a skyline-cube-based *FHL* index that can handle the flexible *MCSP* efficiently. Firstly, we theoretically analyse the relation between low and high-dimensional skyline paths and use a cube to organise them hierarchically. After that, we propose methods to derive the high-dimensional path from the lower ones, which can naturally adapt to the flexible scenario and reduce the expensive high-dimensional path concatenation. Then we introduce efficient methods for both single and multi-hop cube concatenations and propose pruning methods to further alleviate the computation. Finally, we improve the *FHL* structure with a lower height for faster construction and query. Experiments on real-life road networks demonstrate the superiority of our method over the state-of-the-art.

4.1 Introduction

Multi-criteria decision-making on path-finding is important in our daily life due to the increasingly informative road networks that can satisfy various user demands. For example, a user may have a limited budget to pay the toll charge of highways, bridges, tunnels, or congestion. Some mega-cities require drivers to reduce the number of big turns (e.g., left turns in the right driving case [110] as they have a higher chance to cause accidents). There are other side-criteria such as the minimum

height of tunnels, the maximum capacity of roads, the maximum gradient of slopes, the total elevation increase, the length of tourist drives, the number of transportation changes, etc. However, most of the existing path-finding algorithms only optimise one objective and ignore the other side-criteria, such as minimum distance [1, 2, 32], travel time of driving [4, 5] or public transportation [95, 96], fuel consumption [6, 7], battery usage [8], etc. In fact, it is these criteria and their combinations that endow different routes with diversified meaning and satisfy users' flexible needs. For example, users' requirements on path-finding may change according to their current events or the means of transport used: A commuter usually likes the most fuel-efficient path within a limited time budget; a cyclist or walker may prefer the shortest whilst labor-saving (gentle slope) way; a traveler prefers the most attractive path within limited tolls. Hence, this flexible *multi-criteria* route planning is more generalised whilst the *shortest path* is its special case. Moreover, it is also an important research problem in the fields of both transportation [9, 10] and communication (Quality-of-Service constraint routing) [11–14].

Motivations. Typically, *skyline path* [17–19] is applied to find the result which is optimal in every criterion when there are multiple optimisation goals. It provides a set of paths that cannot dominate each other in all criteria. However, skyline path computation is very time-consuming with time complexity $O(c_{max}^{n-1} \times |V| \times (|V| \log |V| + |E|))$, where $|V|$ is the vertex number, $|E|$ is the edge number, c_{max} is the largest criteria value, and n is the number of criteria [20] (c_{max}^{n-1} is the worst-case skyline number). In addition, it is impractical to provide users with a large set of potential paths for them to choose from. Therefore, the *Multi-Constraint Shortest Path (MCSP)* is widely applied and studied. Specifically, it finds the best path based on one objective whilst requiring other criteria satisfying some predefined constraints. For example, suppose each road has a distance w and two costs c_1, c_2, and we set the maximum constraints C_1 and C_2 on the costs. Then this *MCSP* problem finds the shortest path p whose cost $c_1(p) \leq C_1$ and $c_2(p) \leq C_2$.

Nevertheless, it is non-trivial to solve the flexible *MCSP* problem where users could specify arbitrary constraint combinations based on their current requirements. Firstly, it inherits all the challenges from the skyline path search as proved in [39]. In fact, its simplest version *Constraint Shortest Path (CSP)* (with only one constraint) is already an NP-H problem [25, 26], and this is why most of the existing works only focus on *CSP* queries and many of them trade the result optimality for efficiency [13, 20, 26–28, 38]. Currently, *Forest Hop Labelling (FHL)* [39] is the only *CSP* algorithm that can achieve both accurate and efficient results. However, its *MCSP* version [41] takes orders of magnitude longer time to construct the path index because both the skyline path number and the skyline validation cost soar up as the dimension increases. Moreover, *FHL* is only efficient for the *MCSP* queries with the same constraint number, but its performance deteriorates when the query's constraint is a subset due to its huge label redundancy and limited pruning power. Another way to adapt to the flexibility is building indexes for each constraint combination (Figure 4.1-(a) *FHL-Multi*),

but it would take a longer time to construct and result in a much larger index space. Instead, in this work, we aim to design a single index (Figure 4.1-(b) *FHL-Cube*) to support flexible *MCSP* queries. Detailed analysis of these approaches is provided in Section 4.3.

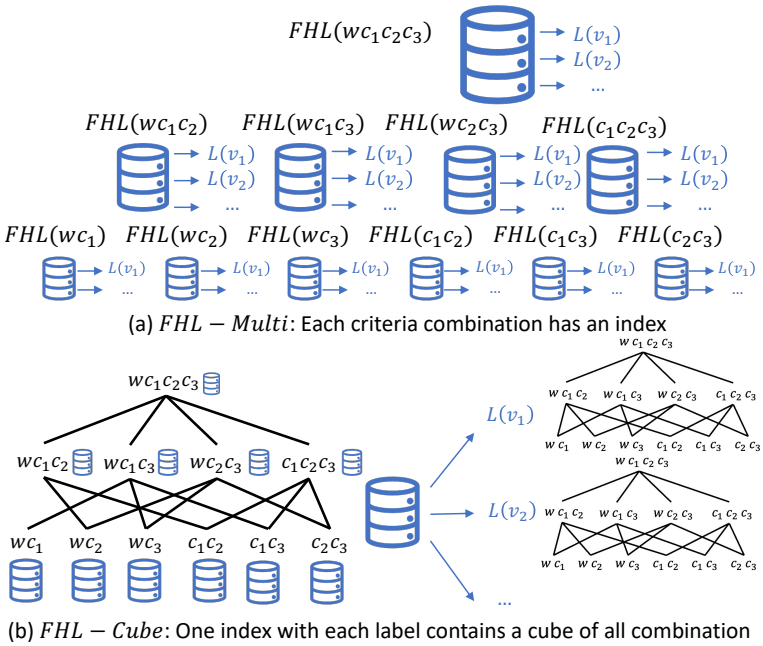


Figure 4.1: FHL and FHL-Cube Comparison

Challenges. The flexible *MCSP* requires answering queries of arbitrary criteria combinations, so the first challenge lies in how to build an index to support the query flexibility. Even though the higher dimensional *MCSP* queries take larger index space with longer construction and query time than the lower ones, it is observed that the query results of the low dimension also belong to those of the high dimension. This insight provides us with a chance to derive the high-dimension skyline paths by accumulating those from the low dimension. To this end, it is critical to distinguish between low-dimensional skyline paths and high-dimensional ones and establish their relations. Specifically, we first propose the *exclusive skyline path* to determine the lowest dimension of a skyline path, such that the skyline paths can be organised hierarchically into a *cube* by categorising them into different subspaces. Then, we extend *FHL* to *FHL-Cube* by changing the index values from paths of only one space to a cube that organises all the spaces as shown in Figure 4.1-(b). Next, we establish their relations and propose theorems to derive high-dimensional results from lower ones. Finally, the higher dimensional cuboid (the basic component of cube) is smaller than the lower ones (reverse to the *FHL* as indicated by the database sizes), and the flexible *MCSP* can be handled naturally.

In addition, the cuberisation of skyline paths involves cube concatenation in both index construction and query processing, which would cause numerous intermediate skyline results with heavy computation. The difficulty here is to improve the cube concatenation efficiency such that the index

construction and query processing can be accelerated as well. Naively, we concatenate the skylines of the corresponding cuboids from two-dimension to higher dimensions, since the subspaces are organised hierarchically. However, there are multiple two-dimension subspaces waiting for concatenation. Hence, we dedicate ourselves to pruning the unnecessary computation in cube concatenation through subspace pruning and hop pruning, respectively. To be specific, we determine the path range of each subspace through calculating the lower and upper bounds of each dimension, such that the subspaces whose path range is dominated by others could be safely pruned without affecting the correctness. In terms of the hop pruning, we precompute the lower bound and the upper bound of the path results concatenated by each hop and those hops with their lower bound surpassing others' upper bound can be safely pruned. Moreover, it is proved that those pruned subspaces or pruned hops of one specific dimension also apply to all its higher dimensions. Finally, both the index construction and query processing of *FHL-cube* are highly improved owing to the proposed pruning techniques of cube concatenation.

Finally, it is observed that the tree height of the *tree decomposition* structure during the index construction is usually large, which seriously deteriorates the index performance since a larger tree height indicates more cube concatenation in both index construction and query processing. To alleviate this problem, we analyse the forest structure and propose several principles to optimise the index structure. As for the query processing, our index is flexible for any combination of criteria because of the skyline path relation established between low and high dimensions.

Contributions. 1) We introduce a novel flexible *MCSP* problem and propose the *FHL-Cube* index for its query processing; 2) We achieve the query flexibility by organising and deriving paths with different dimensions hierarchically as a cube, and further propose pruning techniques for faster cube concatenation; 3) We optimise the boundary label structure and concatenation method for faster index construction and querying; 4) Lastly, we conduct extensive evaluations on real-world road networks to verify the superiority of our approach compared with the state-of-the-art methods.

4.2 Problem Definition

A multi-criteria road network is an n-dimensional graph $G(V,E)$, where V is a vertex set and $E \subseteq V \times V$ is an edge set. Each $e \in E$ has n criteria falling into two categories: a *weight* $w(e)$ and a set of *costs* $\{c_i(e)|i \in [1,n-1]\}$. A path p from $s \in V$ to $t \in V$ is a sequence of consecutive vertices $p = \langle s = v_0, v_1, \ldots, v_k = t\rangle = \langle e_0, e_1, \ldots, e_{k-1}\rangle$, where $e_i = (v_i, v_{i+1}) \in E, \forall i \in [0,k-1]$. Each path p has a weight $w(p) = \sum_{e\in p} w(e)$ and a set of costs $\{c_i(p) = \sum_{e\in p} c_i(e)\}$. We assume the graph is undirected and will extend to directed in Section 4.6.4. We denote an n-dimensional space as $D^n = \{w, c_1, ..., c_{n-1}\}$ to map each criterion in G. In addition, we use β to represent a combination of criteria for short. We define the *MCSP* queries as follows:

Definition 4.1 (**constraint group**). *A constraint group $\mathcal{C}^{\beta}$ is a set of values $\{C_i\}$, where $C_i \in \mathbb{R}^+$, $i \in [1, n-1]$. The size of the group is $|\beta|$, $|\beta| \in [1, n-1]$, and $\mathcal{C}^{|\beta|}$ denotes all the groups with size $|\beta|$.*

For example, given a 4-dimensional graph, constraint group $\mathcal{C}^{\{C_1,C_3\}}$ belongs to $\mathcal{C}^2$, and $\mathcal{C}^{\{C_1,C_3,C_4\}}$ is another one that belongs to $\mathcal{C}^3$. For ease of explanation, the notation of constraint group is simplified,i.e., $\mathcal{C}^{\{C_1,C_3,C_4\}}$ is written as $\mathcal{C}^{C_1C_3C_4}$

Definition 4.2 (**Flexible *MCSP* Query**). *Given an n-dimensional graph $G(V,E)$, an MCSP query $q(s,t,\mathcal{C}^{\beta})$ returns an optimal path with the minimum $w(p)$ whilst $c_i(p) \leq C_i, \forall C_i \in C^{\beta}$.*

Note that β can be varying and it involves $\sum_{i=1}^{n-1} \binom{n-1}{i}$ different constraint groups. Thus, our solution will support queries from all these groups, which highlights the flavour of the "flexibility" of *MCSP* query answering. In the following, we also define *fixed MCSP* query as a reference:

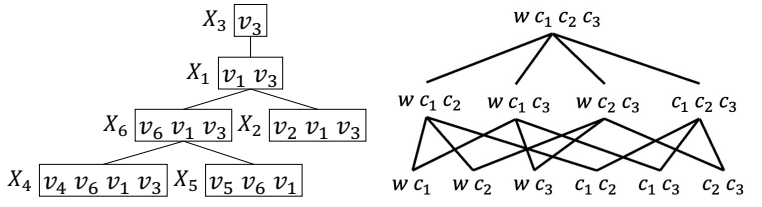
Definition 4.3 (**Fixed *MCSP* Query**). *Given an n-dimensional graph $G(V,E)$ and a fixed constraint group $\overline{\mathcal{C}}$, a fixed MCSP query $q(s,t,\overline{\mathcal{C}})$ returns an optimal path with the minimum $w(p)$ whilst $c_i(p) \leq C_i, \forall C_i \in \overline{\mathcal{C}}$.*

For brevity, the term "*MCSP* query" discussed in the remaining paper refers to *flexible MCSP* query. As analyzed in Section 5.1, it is impractical to answer a *MCSP* query by online graph search. Therefore, in this paper, we resort to index-based method and study the following problem:

Definition 4.4 (**Index-based MCSP Query Processing**). *Given an n-dimensional graph $G(V,E)$, we aim to construct a label-based path index L such that any flexible MCSP query $q(s,t,\mathcal{C}^{\beta})$ in G can be answered efficiently only with L.*

We assign each vertex $v \in V$ a label $L(v) \in L$. Specifically, $L(v)$ contains the set of all the path index starting from v and we use $L(v,u) \in L(v)$ to denote the set of path index from v to u.

4.3 Analysis of FHL on Flexible MCSP



(a) Tree Decomposition of G_1 (b) 4D Bottom-Up Enumeration Tree

Figure 4.2: Tree Decomposition and Bottom-Up Tree

For an n-D space, there could be $\sum_{i=2}^{n}\binom{n}{i}$ subspaces. Figure 4.2-(b) shows a bottom-up enumeration tree for a 4D space $wc_1c_2c_3$, and each k-D ($2 \leq k \leq n$) space can be divided into C_k^2 $(k-1)$-D subspaces. For instance, the 3D space wc_1c_2 can be divided into three 2D subspaces: wc_1, wc_2, and c_1c_2. In this section, we analyse the two *FHL* approaches for the flexible *MCSP*. We call the one with index only on the highest dimension *FHL* [41], and the one with index on every subspace *FHL-Multi*. Specifically, because *FHL-Multi* has the fastest query performance, we analyze its index construction; because *FHL* has the smallest index, we analyze its query performance.

1) Hardness of the High-Dimensional Skyline Path Concatenation. Given two skyline path sets $P(v,h_i)$ and $P(h_i,u)$ with sizes of m and n, where $\{h_i\}$ is the set of v and u's common hops (i.e., cut set), we can compute the d-D skyline paths $P(v,h_i,u)$ from v to u via each h_i, which is the skyline results from $P(v,h_i) \oplus P(h_i,u)$, with the time complexity $O(Nlog^{max(1,d-2)}N)$, where d is the dimension number and $N = mn$ is the cardinality of generated data. It should be noted that this time complexity does not just have a higher order of log as it seems, the skyline paths sizes m and n also increase exponentially as the dimension number grows. Moreover, as u and v have multiple hops, it takes $O(\sum_{i=1}^{k}|P(v,h_i,u)|\log\sum_{i=1}^{k}|P(v,h_i,u)|)$ time in total to obtain the final skylines $P(v,u)$.

2) FHL Query Processing. To answer a query $q(s,t,C^{\beta})$, we first compute the *LCA* X of $X(s)$ and $X(t)$. We can compute the subspace skyline paths $P(s,t)$ of $D^{\beta\cup\{w\}}$ through the concatenation: $P(s,h_i) \oplus P(h_i,t), \forall h_i \in X$ in $O(Nlog^{max(1,|\beta|-1)}N)$ time. After that, we find the one with minimum w under the constraints of C^{β} in constant time. If the query points are from different partitions, similar process would occur three times (in the small trees and boundary tree respectively). However, N here comes from the highest dimensional space so it is much larger than the ones actually needed. To make things worse, the pruning power among multiple hops [41] also decreases due to high-overlapping in the high dimensional space. So the efficiency using *FHL* for subspace *MCSP* is incomparable with an *FHL* built speficially for that subspace.

3) FHL-Multi Index Construction. To adapt *FHL* to *FHL-Multi*, its n-D skyline paths are replaced with all their subspace skyline paths based on the enumeration tree as shown in Figure 4.1-(a). During the index construction, to compute a label from v to each u, we consider all the vertices in $X_v \setminus \{v\}$ as their common hops. The skyline paths are concatenated in each D^{β} instead of one fixed d-D, so time complexity of the concatenation on each hop becomes $O(\sum_{d=2}^{n}\binom{N}{d}\cdot Nlog^{max(1,d-2)}N)$, and the total complexity on k hops takes $O(\sum_{d=2}^{n}\binom{n}{d}\cdot\sum_{i=1}^{k}|P(v,h_i,u)|\log\sum_{i=1}^{k}|P(v,h_i,u)|)$ time. Besides, the index sizes also grows exponentially, which makes this approach impractical for real-life applications.

In summary, fast query processing requires a huge index (*FHL-Multi*), whilst a reasonably sized index (*FHL*) is insufficient for query processing. Therefore, we describe our *FHL-Cube* that is query efficient whilst having a reasonable index size.

4.4 FHL-Cube Construction

FHL-Cube consists of two levels of index: a set of inner skyline-cube trees for each partition and a boundary skyline-cube tree to organise the inner trees. Because these two structures follow the similar construction procedure, we describe how to construct the skyline-cube tree first. Then we present how to further optimise the boundary tree construction based on the problems in *FHL*.

4.4.1 Skyline Cube Tree Construction

Before constructing the index, we first compute the all-pair skyline results of each partition's boundaries on the original graph such that the path information detouring outside each partition is also captured. After that, we add this boundary all-pair information to each partition and start the index construction. The construction is made up of three steps as shown in Algorithm 7. The first step (lines 2-6) forms the tree nodes by contracting the vertices in order (we use the *Minimum Degree Elimination* [18, 63]), with the boundary vertices the last to contract. We denote the vertex set including v and its neighbour set $N(v)$ at the time when v is contracted as X_v. The contraction utilizes the single hop skyline cube concatenation. Because the edges between the boundaries are already computed, we can use the edge information directly without actual concatenation. Secondly, a tree is formed by setting the parent of each X_v as X_u with $u \in X_v$ the lowest-ranking vertex in X_v (lines 8-9). Finally, the labels are populated from the tree root in a depth-first fashion by reusing the labeling of its ancestors' (lines 11-15). Specifically, we assign labels to v from X_v to all its ancestors X_u, because the vertices in X_v naturally form a cut between v and all its ancestors. Then for any vertex pair (s,t) with X_v as the lowest common ancestor of X_s and X_t, we can view the vertices in X_v as their hops. We use multi-hop cube concatenation to obtain the label cubes. Details about the skyline cubes and cube concatenation are introduced in Sections 4.5 and 4.6 respectively.

4.4.2 Boundary Tree Construction

The boundary graph is made up of all partition boundaries. In the same vein as *FHL*, we can contract the boundaries in the unit of partition with the same order used in the inner-partition index to keep the tree structure of the boundaries from the same partition stable. Moreover, all the concatenations used between the boundaries from the same partition can also be skipped as we have already obtained their cubes. However, this contraction method still takes a very long time to propagate on the boundary tree, which includes a series of skyline path cube concatenations amongst the tree nodes. This is because *FHL* constructs the boundary tree following the contracting order of the small trees without considering the hierarchical structure among the partitions. This makes the boundary tree become very unbalanced and the tree height increases dramatically. Moreover, as discussed in Section 5.2.2, when we create a

Algorithm 6: FHL Cube Construction

Input: Graph $G(V,E)$
Output: FHL-Cube Labeling L

// Tree Node Contraction
$v \in V$ has the minimum degree and $V \neq \phi$ $X_v = N(v)$, $r(v) \leftarrow$ Iteration Number
for $(x,y) \leftarrow N(v) \times N(v)$ **do**
    $Cube_{x,y} \leftarrow Cube(Cube_{x,v} \oplus Cube_{v,y} \cup Cube_{x,y})$;
    $E \leftarrow E \cup (P(x,y))$, $V \leftarrow V - v$, $E \leftarrow E - (x,v) - (v,y)$;
// Tree Formation
for v *in* $r(v)$ *increasing order* **do**
    $u \leftarrow min\{r(u)|u \in X(v)\}, X_v.Parent \leftarrow X_u$
// Label Assignment
$X_v \leftarrow$ Tree Root, $Q.insert(X_v)$
$X_v \leftarrow Q.pop()$ **for** $u \in \{u|X_u$ *is ancestor of* $X_v\}$ **do**
    $P(v,u) \leftarrow Cube(\bigcup Cube_{v,h} \oplus Cube_{h,u})|\forall h \in X_v)$;
    $L(v) \leftarrow (u, Cube_{v,u})$;
$Q.insert(X_u.children)$, $L \leftarrow L \cup L(v)$;

boundary graph, we first build a complete graph for each partition and then connect them together by all cutting edges. Therefore, it results in a very dense graph with a large tree height.

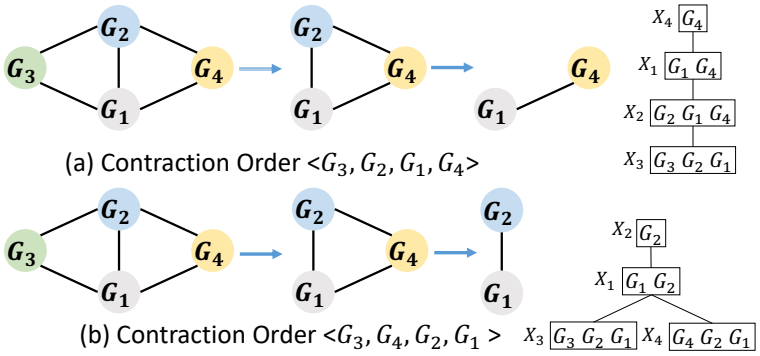


Figure 4.3: Contracting Orders with Different Principles

Hence, we propose a contraction method on the boundary graph to achieve the balance of the boundary tree and then reduce the computation on the propagation phase. To ensure the hierarchical structure of the partitions, we first create a connected graph that presents their connective relationships. Specifically, we treat each partition as a node and add an edge between two nodes if their corresponding partitions are connected in the original boundary graph. For example, Figure 4.3 is the connected graph of four partitions. It displays the procedure of contracting in two different orders. Even though both Figure 4.3-(a) and Figure 4.3-(b) contract the graph by the order of vertex degree, the contraction in Figure 4.3-(b) is better than Figure 4.3-(a), because it can produce lower tree height. As a consequence,

we propose our observation as follows:

Lemma 4.1 (**Single Child Tree Decomposition (SC-TD)**). *If the vertices in a contracting order are consecutive and form a path, then each node in its tree decomposition only has a single child.*

Proof. When contracting each vertex v, it will form a tree node, including v and its current neighbours. After contracting all vertices, we check the vertices of each node X_v, except v, to find the vertex u contracted prior to others, and select its corresponding node X_u as the parent of X_v. As the vertices in the contracting order are consecutive, the next contracting vertex must be a neighbour of the current contracting vertex. Therefore, the corresponding node of the next contracting vertex will always be the child of the current contracting one. This property degenerates the tree into a stick. □

Note that the *SC-TD* or the extremely unbalanced tree is very common in dense graphs, such as the connected graph of New York City, where most of its partitions connect to many other partitions. Moreover, when the graph is complete, its tree decomposition must be an *SC-TD*. Therefore, it is critical to solving this contracting problem in the boundary graph. Based on Lemma 4.1, we aim to break the consecutive of the contraction order: if several vertices following the consecutive order can form a path on the graph, we reduce its length as much as possible. Therefore, we iteratively choose a vertex to contract with the following principles: 1) smallest degree; 2) if v is the neighbour of the last contracted vertex, we find the next one to extract; 3) the number of times each vertex v appears as a neighbour is recorded as $Count(v)$; 4) if 2) cannot be held, we contract the vertex v with the smallest $Count(v)$.

4.5 Skyline Path Relations and Cube

Intuitively, concatenation in lower dimensions is much faster, so we resort to "pushing it down" from higher to lower dimensions. Firstly, we discuss the relationship between the lower-dimensional and higher-dimensional skyline paths. Then we propose several observations to derive the higher dimensional skyline path from the lower ones. Finally, we present the *skyline path cube* that organises the skyline paths into different categories, which will further improve the concatenation efficiency and support flexibility.

4.5.1 Skyline Path Categorisation

Because two paths with the same value in one subspace may have different values in other dimensions, we first distinguish the basic types of non-dominated paths based on their values:

Definition 4.5 (**Non-dominated Path Type**). *Given two non-dominated paths p and q, they are **Indistinct** if they have the same values on all dimensions; otherwise they are **Incomparable**.*

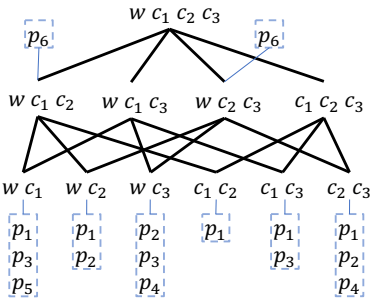
For example, $p = (1,1,2,3)$ and $q = (1,1,3,2)$ are *incomparable*, whilst $p = (1,1,2,3)$ and $q = (1,1,2,3)$ are *indistinct*. Although two paths are indistinct in 4D, they could have different values on other dimensions. Therefore, we further discuss the path relations within a specific subspace as follows:

Definition 4.6 (**Skyline Subspace**). *Given a set P_s of skyline paths in subspace D^β ($\beta \in [2,n]$), we say D^β is: an **Incomparable Subspace** $D^\beta_{\perp}\{p\}$ of p if $\beta(p)$ is incomparable with the projections of other skyline paths in D^β; an **Indistinct Subspace** $D^\beta_{=}\{p\}$ of p if $\exists q \in P_s$, $\beta(p)$ and $\beta(q)$ are indistinct in D^β; a **Dominated Subspace** $D^\beta_{\prec}\{p\}$ of p if $\exists q \in P_s$ dominates $\beta(p)$ in D^β.*

For example in Figure 4.4-(a), $P_s = \{p_1, p_3, p_5\}$ is a skyline path set in subspace wc_1, and D^{wc_1} is 1) $D^\beta_{\perp}\{p_3\}$ an incomparable subspace for p_3 because it has different values with p_1 and p_5; 2) $D^\beta_{=}\{p_1, p_5\}$ an indistinct subspace as the their path values are the same in wc_1; 3) $D^\beta_{\prec}\{p_2, p_6\}$ dominated, because their projection $(3,7)$ and (3,5) in wc_1 are dominated by $wc_1(p_1) = (2,4)$.

P	w	c_1	c_2	c_3
p_1	2	4	10	2
p_2	3	7	7	10
p_3	5	3	9	3
p_4	6	6	8	3
p_5	2	4	10	3
p_6	3	5	9	4

(a) Example 4D Path Set



(b) Skyline Cube Example

Figure 4.4: 4D Path Example

Depending on the subspace types, we divide the n-D paths into the following four types: **Case 1 Path** has at least one incomparable subspace; **Case 2 Path** only has indistinct subspaces; **Case 3 Path** has indistinct and dominated subspaces; **Case 4 Path** only has dominated subspaces.

Next, we present the skyline path derivation using the categorisation above.

4.5.2 Skyline Path Derivation

To establish the relations between the lower dimensions and the higher dimension, we discuss the properties of these four path categories. We first have the following theorem for the *Case I Path*:

Theorem 4.2 (**Incomparable Lower Dominates Higher**). *p is a skyline path in D^n if it has an incomparable subspace $D^\beta \subseteq D^n$.*

Proof. Suppose D^β is p's incomparable subspace. If p is not a skyline path in D^n, then $\exists p'$ dominates p in D^n, which dominates p in D^β. This contradicts with the incomparable definition. □

For example, given a 4D path $p(wc_1c_2c_3)$, if it is a skyline path in wc_1 and $D^{\{wc_1\}}$ is its 2D incomparable subspace, then p must be a skyline path in all the superspaces of $D^{\{wc_1\}}$, including $D^{\{wc_1c_2\}}$, $D^{\{wc_1c_3\}}$, and $D^{\{wc_1c_2c_3\}}$. When we generate the skyline paths to higher dimensions, we can obtain the *Case I Paths* on the fly. Moreover, the number of *Case I Paths* keeps monotonically increasing when the dimension increases, because the emerging subspaces could be new incomparable subspaces, which can produce more skyline paths. For *Case 2 Paths*, we have the following theorem.

Theorem 4.3 (**Full Indistinct Dominates Higher**). *p is a skyline path in D^n if any D^2 is an indistinct subspace of p.*

Proof. We also prove it by contradiction. Suppose p is indistinct in all D^2 but not a skyline path in D^n, then $\exists p'$ dominates p in D^n. Therefore, there exists at least one dimension c_i such that $p'(c_i) < p(c_i)$ whilst others are smaller or equal, so p could not be indistinct in those subspace D^2 containing c_i. □

For *Case 3 Paths*, we have the following theorem.

Theorem 4.4 (**Partial Indistinct and Incomparable**). *$\forall q$ that is indistinct skyline path of p in some D^2 subspaces, p is a skyline path if it cannot be dominated by all these q in other D^2 subspaces.*

Proof. If any indistinct q can dominate p in all the other spaces, then q can dominate p in D^n, so p is not a skyline. □

As for the *Case 4*, it is a little tricky because it only has dominated subspaces and thus seemingly cannot be a skyline path. However, this is incorrect. For example in Figure 4.4-(a), $\overline{p_6}$ is dominated in all the D^2 subspaces, but it cannot be dominated in the D^4 space. Therefore, a path can be a skyline path even though it is dominated in every D^2. This is caused by the fact that the skyline paths within different dimensions cannot hold monotonicity [92]. The following theorems help to rule out some non-skyline paths:

Theorem 4.5 (**Non-Skyline Case IV**). *A Case 4 Path p cannot be a skyline path in $D^{\beta\cup\beta'}$ if p is dominated by the same path q in D^{β} and $D^{\beta'}$*

Proof. Because q can dominate p in both D^{β} and $D^{\beta'}$, q can also dominate p in $D^{\beta\cup\beta'}$, then p is not a skyline path in $D^{\beta\cup\beta'}$. □

Theorem 4.6 (**Skyline Case IV**). *Given a Case 4 Path p, suppose S_i is a set of skyline paths that can dominate p in D_i^2, where D_i^2 denotes the i^{th} D^2 subspace. Then p is a skyline path in $\bigcup_{i\in[0,k]} D_i^2$ iff $\bigcap_{i\in[0,k]} S_i = \Phi$, where $k \leq \binom{n}{2}$.*

Proof. This is the opposite of Theorem 4.5. Suppose p is not a skyline path, then there exists a path that can dominate p in D^n and every D^2, which contradicts to $\bigcap_{i\in[0,k]} S_i = \Phi$. □

Theorem 4.2 to 4.4 can derive the skyline path results from lower to higher dimensions, which satisfies our aim of "pushing down" the skyline concatenations: with several fast lower-dimensional skyline concatenations, we are able to obtain a large set of the lower dimensional results, which can be used to answer the higher dimensional queries directly. As for the results missed by the lower dimensional results, Theorem 4.5 and 4.6 help to reduce the concatenation space for the higher dimensional skyline paths.

4.5.3 Skyline Path Cube and Cuberisation

Now we are ready to describe the structure of the skyline path cube. Firstly, a skyline cube acts like a subspace that organises the criteria hierarchically. Apart from the subspaces, the skyline path cube also allocates the skyline paths into different cuboids of the lattice. The following definition categorises the skyline paths according to the dimension numbers, which can be further used to allocate the paths into different levels of the cuboid:

Definition 4.7 (**k-Exclusive Skyline Path**). *A path p is a k-exclusive skyline path if it can be derived from the D^k but not from D^{k-1}. In other words, k is the lowest dimension number to validate p as a skyline.*

Definition 4.8 (**Skyline Path Cube**). *Given a set of skyline paths from u to v in D^n, a skyline cube $Cube^n_{uv}$ is a mapping from the k-exclusive paths to their corresponding D^k cuboids.*

Theorem 4.7 (**Skyline Cube Disjoint and Completeness**). *All the skylines are stored in one and only one level of the cuboids.*

Proof. Evidently, a skyline path can only appear in one level of the cuboid. As for the completeness, suppose p is a skyline path but does not exist in the cube. If p does not exist in level k, then it must be either dominated by the skyline paths in D^k, or it exists in the higher levels at least as a $k+1$-exclusive skyline path. If p does not exist in level $k+1$, then it is at least a $k+2$-exclusive skyline path. Therefore, when this procedure reaches $k = n$, then p is dominated by the paths in the cube, so such p does not exist. □

Figure 4.4-(b) shows an example of the skyline cube of the paths in (a). It should be noted that the skyline cube is another layer to organise the skyline paths, so the original skyline paths are still stored in a table like (a), and the cuboids only store the address of the path in the table. Firstly, the cuboid $P^{\{wc_1\}}$ has the path set $\{p_1, p_3, p_5\}$, as the projections of them on $D^{\{wc_1\}}$ are skyline paths. Secondly, we can also derive that p_1, p_2, p_3 and p_4 are skyline paths in the superspaces of their incomparable D^2 subspaces. Specifically, p_3 has three incomparable D^2 subspaces $D^{\{wc_1\}}$,$D^{\{c_1c_2\}}$ and $D^{\{c_1c_3\}}$. Therefore, p_3 is also the skyline path in the superspaces that contain any one of these

incomparable subspaces, and it is a skyline path in all the higher spaces. As for p_5, it is indistinct in $D^{\{wc_1\}}$ with p_1 but it is not a skyline in its superspace $D^{\{wc_1c_2c_3\}}$ as it is dominated by p_1. As for p_6, it can only be found in $D^{\{wc_1c_2\}}$ and $D^{\{wc_2c_3\}}$, so it is a 3-exclusive skyline path.

We call the process to construct a skyline cube from a set of paths a *Cuberisation* operation $Cube(p)$. Firstly, we validate the paths from lower to higher dimensional cuboids. For the higher dimensional cubes, we also need the results from their corresponding sub-cuboids for path validation. Each time we finish a level of cuboids, we can remove these paths from P^β. To further reduce the index size, we propose the following skyline cube bounds to prune the impossible paths from the existing result:

Lemma 4.8 (**Skyline Path Boundary**). *Given any $D^{c_ic_j}$, its skyline paths must exist within the boundary of $[c_i.min, c_i.max] \times [c_j.min, c_j.max]$, where the values obtained from its skyline path set.*

We emphasise that if a subspace path p in a $D^{c_ic_j}$ has the minimum value on c_i and on other paths have the same value with p on c_i, p must be a subspace skyline path in $D^{c_ic_j}$, and it determines the maximum value of skyline paths on c_j. Hence, we sort the paths in P on both c_i and c_j in increasing order. The top one path on each criterion can determine the skyline path boundary $[c_i.min, c_i.max] \times [c_j.min, c_j.max]$ if $D^{c_ic_j}$ is an incomparable subspace of them. For example in Figure 4.4-(a), as p_2 has $c_2.min = 7$, it determines that $c_3.max = 10$. p_1 has $c_3.min = 2$. Then we locate $c_2(p_1) = 10$ as $c_2.max$. Thus, the skyline boundary of $D^{c_2c_3}$ is $[c_2(p_2), c_2(p_1)] \times [c_3(p_1), c_3(p_2)] = [7, 10] \times [2, 10]$.

Definition 4.9 (**Skyline Cube Bound**). *Given a set of skyline paths, its skyline cube bound is a hyper-cube $B = \Pi_{\forall c_i \in \beta}[c_i.min, c_i.max]$ such that all the skyline path values must exist in the corresponding ranges.*

To obtain a skyline cube bound of a set of skyline paths, we sort them on each criterion, and locate two skyline results of each $D^{\{c_ic_j\}}$ space: the first one on c_i that has the $c_i.min$ and $c_j.max$, and the first one c_j that has the $c_j.min$ and $c_i.max$. After that, we go through all the D^2 boundaries and take the minimum/maximum of each dimension as the final bounds. With such a bound derived from the D^2, we can use it to prune the paths whose values are all larger than each upper bound. The time complexity of Cuberisation can be loosely bounded by $O(\sum_{i=2}^{|\beta|} \binom{|\beta|}{i} \times n \log^{|\beta|-1} n)$, whilst the actual running time is much faster due to the above pruning on the higher dimensional exclusive path size.

4.6 Skyline Cube Concatenation

Different from *FHL*, whose label stores the skyline paths of a specific D^β, *FHL-Cube*'s label stores the skyline path cube. During the index construction and query processing, *FHL-Cube* needs to concatenate two skyline path cubes in all non-empty subspaces instead of from only one D^β. In the

following, we present how to concatenate the skyline cubes efficiently in the single (Section 4.6.1) and multiple (Section 4.6.2) hop scenarios, and how to utilise them during query processing (Section 4.6.3).

4.6.1 Single Hop Cube Concatenation

Definition 4.10 (Skyline Path Cube Concatenation).
Given two skyline path cubes $Cube^n_{uh}$ and $Cube^n_{hv}$, the concatenation result $Cube^n_{uhv} = Cube^n_{uh} \oplus Cube^n_{hv} = \{Cube^\beta_{uh} \oplus Cube^\beta_{hv}, |\forall D^\beta \subseteq D^n\}$ contains all the skyline paths from u to v via h, and these skyline paths are organised in a skyline path cube $Cube^n_{uhv}$.

The concatenation operation includes two phases: 1) Add up the path values to generate a concatenated path set P_s; 2) Find the skyline paths from P_s for computing skyline cubes. As we can not guarantee that all concatenated paths are still skyline paths [39, 41], it increases the complexity of computing skyline cubes. For example, Table 4.1 and 4.1 are two D^4 skyline path sets P_{uh} and P_{hv}. The $D^{\{c_1c_3\}}$ skyline paths in P_{uh} are p_1 and p_3, whilst P_{hv}'s are p'_2 and p'_3. When concatenating $Cube^{c_1c_3}_{uh} \oplus Cube^{c_1c_3}_{hv}$, we can generate four candidates $p^2_1 = (8,10,18,5), p^3_1 = (5,9,19,6), p^2_3 = (7,7,19,6), p^3_3 = (8,8,18,7)$. p^3_1 and p^3_3 are dominated by p^2_3 in $D^{\{c_1c_3\}}$. Therefore, the final result of $Cube^{\{c_1c_3\}}_{uhv}$ is $P^{\{c_1c_3\}}_{uh} \oplus P^{\{c_1c_3\}}_{hv} = \{p^2_1, p^2_3\}$.

Table 4.1: Skyline Path Concatenation of P_{uw} and P_{wv}

(a) Skyline Path Set P_{uh}

P_{uh}	w	c_1	c_2	c_3
p_1	2	4	10	2
p_2	3	7	7	10
p_3	5	3	9	3

(b) Skyline Path Set P_{hv}

P_{hv}	w	c_1	c_2	c_3
p'_1	6	6	8	3
p'_2	2	4	10	3
p'_3	3	5	9	4

We improve the concatenation efficiency from two aspects: 1) *Block Maintenance* when concatenating two underlying skyline paths; 2) *Skyline Path Derivation* when identifying all subspace skyline paths in each *Cube*. In the following, we first introduce 1) for reducing the size of the concatenated paths, which can further reduce the computation time of *Cubes* in 2).

Block Maintenance. Given two sets of m and n skyline paths, the concatenation paths number mn could be large and deteriorates the cube concatenation efficiency. To reduce mn to a smaller value, we first introduce the *path entropy* [99].

Definition 4.11 (Path Entropy ε). *Given an n-D path $p \in P_s$, its entropy $\varepsilon(p) = lnw(p) + \sum_{i=1}^{n-1} \ln c_i(p)$. If $\varepsilon(p)$ is small, then p has higher probability to dominate paths in P_s.*

Specifically, we maintain a *Block* of k smallest entropy concatenated paths. For each $p_i \in P_s$, we can discard it if dominated by any path in the *Block*. Otherwise, we compute its entropy $\varepsilon(p_i)$ and replace p_j in *Block* if $\varepsilon(p_j) > \varepsilon(p_i)$. In addition, the *Block* maintenance can be further optimised in the *tree node contraction* phase where the cubes are concatenated when we compute skyline shortcuts between the neighbour pairs of each vertex w. If an edge $e = (u, v)$ exists between w's two neighbours u and v, then it already includes a skyline path set $P_{u,v}$, and we need to merge the concatenated paths $P_{u,w,v}$ with $P_{u,v}$. As $|P_{u,w,v}|$ can be much larger than $|P_{u,v}|$, we use the k smallest entropy paths in $P_{u,v}$ as the initial *Block*. In practice, this method is effective, especially in the boundary tree.

Cuberisation with Skyline Path Derivation. We propose to break down the expensive high-dimensional cube concatenation into several cheap low-dimensional ones using the previously discussed skyline path derivation and *Cube* prunings.

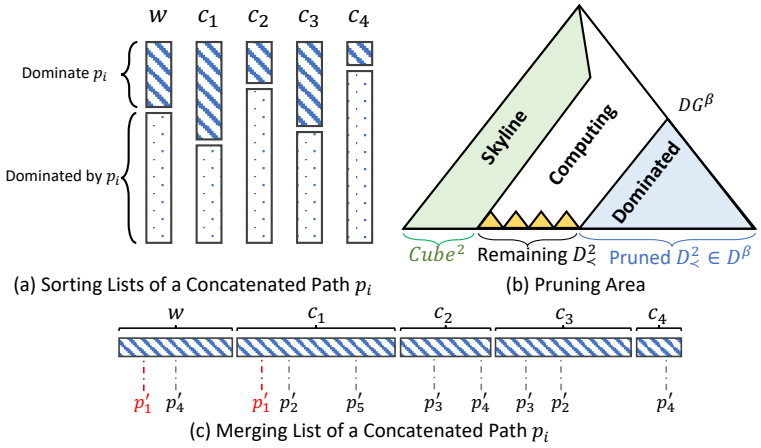


Figure 4.5: Sorting List and Pruning Area Examples

Given two skyline path sets P_1 and P_2, we first add their path values to obtain an all-pair concatenated path set P_s. Then we sort the paths in P_s in ascending order on each criterion. As shown in Figure 4.5-(a), $\forall p_i \in P_S$, the paths ranking before p_i can dominate p_i. On the contrary, the paths after p_i are dominated by p_i. We denote the set of paths dominating p_i on a subspace D^β as $\mathcal{DG}^\beta(p_i)$. Then, we pair p_i to the correct $Cube^\beta$ by checking whether there is a common path p'_i ranking before p_i on each criterion in D^β. For example, if $\mathcal{DG}^w(p_i) \bigcap \mathcal{DG}^{c_1}(p_i) = p'_1$, $\mathcal{DG}^w(p_i) \bigcap \mathcal{DG}^{c_3}(p_i) = p'_2$, we can determine that D^{wc_1} and D^{wc_3} are two dominated subspace $D^{wc_1}_\prec(p_1)$ and $D^{wc_3}_\prec(p_1)$. As $\mathcal{DG}^w(p_i) \bigcap \mathcal{DG}^{c_1}(p_i) \bigcap \mathcal{DG}^{c_3}(p_i) = \phi$, no path can dominate p_i in $D^{wc_1c_3}$. Thus, $wc_1c_3(p_1)$ is a 3-exclusive skyline path and is recorded in a $Cube^{wc_1c_3}$

Next, we find these common paths from lower to higher dimensions. Firstly, we merge all the $\mathcal{DG}^1(p_i)$ and form a new list $\mathcal{L}(p_i)$ as shown in Figure 4.5-(c). Note that the paths p'_i in any $\mathcal{DG}^1(p_i)$ can be ordered by the record location on the concatenated path set P_s. The order range of p'_i is in $[0, mn-1]$. Therefore, we can create a hash table based on the paths' subscripts and the locations of

P_s. It reduces the space complexity and avoids redundant access on $\mathcal{L}$. Specifically, we scan $\mathcal{L}(p_i)$ from the start. When we visit each path p'_i, we mark the i^{th} location of P_s and record the current $\mathcal{DG}^1$. During the scanning, once we visit p'_i, the i^{th} location on P_s will be marked again and another $\mathcal{DG}^1(p_i)$ is recorded. After finishing the scan, the locations of P_s marked more than twice are extracted and we can also get their $\mathcal{DG}^1(p_i)$. For example in Figure 4.5-(c), when we scan $\mathcal{DG}^{c_1}(p_i)$ and encounter p'_1, we mark 1^{st} location on P_s and record c_1 at the same time. As we have scanned $\mathcal{DG}^w(p_i)$ already, the 1^{st} location now is marked twice and recorded wc_1. We can determine D^{wc_1} as $D^{wc_1}_{\prec}(p_i)$. In this way, we can find all the $D^2_{\prec}(p_i)$ and the remaining D^2s are $Cube^2(p_i)$s.

The *Cube*s of p_i on the upper level of $Cube^2(p_i)$ can be directly obtained by the intersection operation on determined $D^2_{\prec}$ and the corresponding subspaces of $Cube^2(p_i)$. However, we still have room to improve the computing efficiency owing to the theorem below:

Theorem 4.9 (**Skyline Path Cube Pruning**). *Given an n-D path p_1, once we determine a dominated subspace $D^{\beta}_{\prec}$, $D^{\beta} \subseteq D^n$, we can prune D^{β} and all its subspaces to compute Cubes of p_1.*

Proof. p_1 exists in $Cube^{\beta}_{uv}$ only if $\beta(p_1)$ is a subspace skyline path. Then D^{β} can be pruned. For the subspaces, the only case that p_1 is a skyline path in the subspace of D^{β} is when $\beta(p_1)$ has at least one indistinct subspace $S \subset \beta$ and one dominated subspace. Therefore, it is a *Case 3* path and must have another path $S(p'_1)$ including the same path value on S with $S(p_1)$. As p_1 is dominated in the superspace of S, there always exists a path p'_1, which is better than p_1. Therefore, pruning p_1 in all the subspaces of D^{β} can not affect the correctness of the query result. □

As shown in Figure 4.5-(b), we can prune a part of the subspaces by computing *Cube*s of any n-D $p_i \in P_s$ based on the results of $D^2_{\prec}(p_i)$ and $Cube^2(p_i)$. Firstly, we can prune all superspaces of each determined $Cube^2(p_i)$ (green shaded area) by referring to Definition 4.7. Suppose we compute T $Cube^2$ of p_i, each $Cube^2(p_i)$ can prune $\sum_{i=0}^{n-2}\binom{n-2}{i}$ subspaces of p_i. Therefore, we can totally prune $T \cdot \sum_{i=0}^{n-2}\binom{n-2}{i} - \sum_{i=1}^{k}\binom{k}{i}$ subspaces, where $\sum_{i=1}^{k}\binom{k}{i}$ is the number of the common subspaces, and the value of $k \in [T+1, 2T]$.

Note that our algorithm can discover all the $DG^{max(\beta)}(p_i)$ after scanning on $\mathcal{L}(p_i)$, where $max(\beta)$ refers to the maximum dominated subspace of p_i. Then we prune all the subspaces of $max(\beta)$, as shown in the blue shaded area. For example in Figure 4.5-(c), p'_4 is marked the most times, the corresponding criteria form the $DG^{wc_2c_4}(p_i)$. Thus, all subspaces of wc_2c_4 can be pruned.

In this step, it will cover one or more pruned $D^2_{\prec}(p_i)$. Suppose the total number of $D^2_{\prec}(p_i)$ is $M < \binom{n}{2} - T$, and the number of pruned ones is R, which results in $R \cdot \sum_{i=1}^{|max(\beta)|} - \sum_{i=1}^{k}\binom{k}{i}$ pruned subspaces, where $k \in [|max(\beta)| + R - 1, |max(\beta)| \cdot R]$. Consider the *Case 4 path*, p_i can be a subspace skyline path in a set of subspaces S, including all the remaining subspace except for the D^2 ones. Note that all subspaces in S are the skyline subspaces of p_i. However, we only need to find the lowest dimensional subspaces in S as the *Cubes* of p_i in terms of Definition 4.8.

4.6.2 Multiple Cube Concatenation Pruning

During the label construction and query answering, we need the skyline concatenation result $Cube^{\beta}_{uh_iv}$ on several hops with $h_i \in H = L(u) \cap L(v)$. The straightforward way is computing the cubes of these h_i, merging the results, and constructing the new cube $Cube^{\beta}_{uv}$. However, because the cube concatenation has a high complexity, and a pair of vertices could have hundreds of such hops, which further creates a large number of paths to check, it is very slow (several seconds) to finish one concatenation. What is worse, the current hop-first concatenation paradigm, which finishes all cube concatenations as a whole, makes it hard to prune useless concatenations beforehand. In query processing, we change the concatenation order from the hop-first perspective to the subspace-first perspective according to the number of constraints and propose a rectangle-based subspace pruning technique to reduce the number of hops and the corresponding cuboids.

Firstly, for each $Cube^2$, we find the bounds of each hop h_i by concatenating their first and last skyline paths. In this way, we obtain a virtual rectangle r_i for each hop such that their concatenation results all fall into it. Then for any two rectangles r_i and r_j, if r_i's lower-left point is dominated by r_j's upper-right, we can safely prune all the points in r_i. In other words, we can avoid concatenating h_i in this subspace. Therefore, a linear scan of the rectangles is enough to obtain a candidate hop set for each D^2. For example in Figure 4.6-(a), h_5 and h_6 are dominated by h_1, h_2, and h_3, so we can avoid concatenating h_5 and h_6 safely. As for h_4, although the right part of it is dominated, we still have to keep it as the left part could still be a skyline result. After that, for the higher dimensional cuboids, we only need to take the union results of its corresponding hop candidates, which is proved in the following theorem:

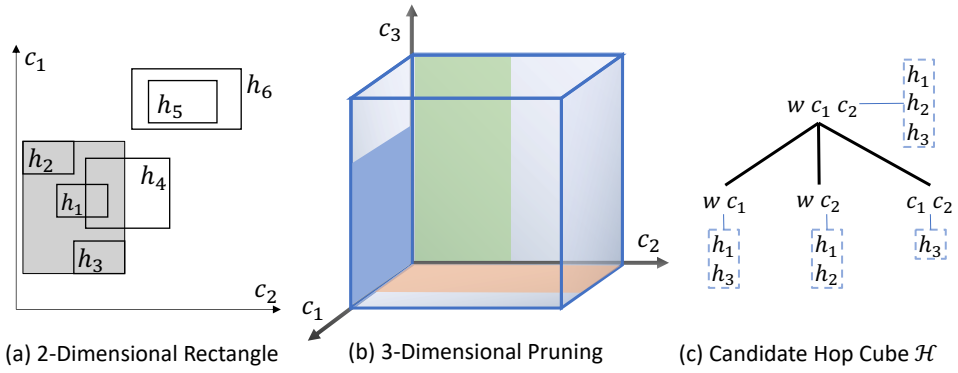


Figure 4.6: Rectangle-based Hop Pruning

Theorem 4.10. *A hop h is a candidate in D^{β} if it exists in one of its subspaces.*

Proof. Firstly, it is trivial to prove that the union of the subspace's rectangles is equivalent to taking the skyline cube's bounds. Then the rectangles that are out of a cube's bounds are dominated in

this subspace. Therefore, those hops associated with the dominated rectangles can be pruned safely. Because we did not prune any correct hop, the remaining ones are the correct candidates. □

By organising the candidate hops in different subspaces, we can obtain the candidate hop cube $\mathcal{H}$. We first obtain the candidate hops for each D^2 subspace. After that, the candidates propagate upward to the candidate set of their superspaces. After all the superspaces have obtained their candidates, we organise them into a hop cube to guide the actual hop concatenations in each subspace, as shown in Figure 4.6-(c). Figure 4.6-(b) illustrates an example of deriving the D^3 candidates from its corresponding D^2 candidates. The hops that are covered by the new cube are also potential results, so we only need to take the union of the three D^2 candidate sets (blue, green, and orange regions). The complexity of the pruning phase is linear to $O(|H|\log|H|)$, which is spent on the rectangle sorting. As $|H|$ is much smaller than the skyline path number $|P|$, its complexity is dominated by the cube concatenation. Therefore, the pruning phase reduces the concatenation number at a very small cost.

4.6.3 Flexible MCSP Query Answering

The following theorem first proves the information we need to answer a flexible *MCSP* query correctly:

Theorem 4.11. *Given a MCSP query $Q(s,t,\mathcal{C}^\beta)$, its result could only exist in the $Cube^\beta$*

Proof. Because the *MCSP* results can only be one of the skyline paths as proved in Theorem 5.2, we only need to guarantee that the paths not in $Cube^\beta$ are dominated in D^β, which is ensured by the definition of the skyline cube. □

Therefore, we only need to obtain the $Cube^\beta_{st}$ to answer $Q(s,t,\mathcal{C}^\beta)$. To further reduce the computational cost, we can apply the constraints such that only the paths satisfying the constraints are needed. In this way, only a smaller cube $Cube^\beta_{st}(\mathcal{C})$ is needed. In the following, we describe how to obtain the $Cube^\beta_{st}(\mathcal{C})$ when s and t are in the same or different partitions.

Inner Partition Query. When s and t are in the same partition, we only need to use one inner skyline cube tree. Firstly, we find the *LCA* X_u of X_s and X_t, then all the vertices $\{h_i\}$ in X_u are the hops from s to t. Secondly, we apply multiple cube concatenations based on these hops $\{Cube^\beta_{sh_i} \oplus Cube^\beta_{h_it}\}$. To further reduce the concatenated hop number, we replace the upper-bounds of dimension with the constraints in $\mathcal{C}$. In this way, smaller rectangles are obtained whilst more hops are pruned. Besides, during the hop concatenation, we also apply the constraints on the cubes to be concatenated. In other words, $Cube^\beta_{sh_i} \oplus Cube^\beta_{h_it}$ is replaced with $Cube^\beta_{sh_i}(\mathcal{C}) \oplus Cube^\beta_{h_it}(\mathcal{C})$. Finally, as *MCSP* only needs one constraint-satisfying path with the smallest weight, early termination can be applied to further improve the efficiency. Specifically, we only concatenate the paths that satisfy the constraints in the weight-increasing order. Hence, when the first concatenated path satisfies the constraints appears, we can stop concatenating and return it as the final result.

External Partition Query. Suppose s and t are in different partitions, and their corresponding boundary sets are denoted as B_s and B_t. The inter partition query can be decomposed into three sub-queries: i) from s to B_s, from ii) B_s to B_t, and iii) from B_t to t. The results of the i) and iii) can be obtained directly without concatenation because the boundaries must be the ancestors of the inner vertices. The results of ii) can be obtained as a query on the boundary tree. As for the concatenation results of i) and ii), we can view B_1 as the hops from s to each $b_2 \in B_2$. Therefore, multi-hop pruning can also be applied here. Subsequently, we get the cubes from s to B_2, and can also view each $b_2 \in B_2$ as hops from s to t. Besides, the aforementioned cube pruning using the constraints can also be applied here to further reduce the concatenation computation.

4.6.4 Extension For Directed Graph

We denote the directed version as *FHL-Cube+*.

Indexing. For both *Inner Tree* and *Boundary Tree*, *FHL-Cube+* is similar to *FHL-Cube* with two differences: 1) The neighbours of any vertex v are classified into *in*-neighbours $N_i(v)$ and *out*-neighbours $N_o(v)$. During contraction, we iteratively concatenate one $u \in N_i(v)$ and one $w \in N_o(v)$ to form a directed pair (u,w). 2) Instead of saving skyline cubes between v and v's neighbours as labels in each node $X(v)$, we store $Cube_{u,v}$ as *in*-label and $Cube_{v,w}$ as *out*-label separately to distinguish their directions towards v. Hence, when we assign labels between v and all its ancestors in $X(v)$, we consider the vertices in $N_i(v)$ as *in*-hops and the vertices in $N_o(v)$ as *out*-hops to compute the skyline cubes from its ancestors to v as *in*-labels and v to its ancestors as *out*-labels separately.

Query Processing. Given a query $q(s,t,\mathcal{C}^\beta)$, suppose s and t are in the same partition, we first compute the $LCAX_u$ of X_s and X_t. In X_u, we find the vertices, which have the *out*-label with s and the *in*-label with t, as the hops. After that, we compute the cubes from s to the selected vertices and from the vertices to t under the constraints $\mathcal{C}^\beta$. We can use the same way to obtain the results on cube trees and the boundary tree if s and t are in different partitions.

4.7 Experiment

4.7.1 Experimental Settings

We implement all the algorithms in C++ with full optimisation. The experiments are conducted in a 64-bit Ubuntu 18.04.3 LTS with two 8-cores Intel Xeon CPU E5-2690 2.9GHz and 186GB RAM.

Datasets. We conduct experiments on the following four real-world networks [111]: 1) *NY* is a dense grid-like urban area with 264,246 vertices, 733,846 edge, and several partitions connected by bridges; 2) *BAY* has a shape of a doughnut around the *San Francisco Bay* with 321,270 vertices,

800,172 edges; 3) *COL* has an uneven vertex distribution (dense around Denver but sparse elsewhere) with 435,666 vertices and 1,057,066 edges; 4) *FLA* is a large network Florida with 1,070,376 vertices and 2,712,798 edges. We use the road length as the weight w and travel time t as the first cost. And three more positive correlation costs are generated randomly. We also obtain three additional real-world criteria for *NY-REAL*: 1) *Toll Charge* from NY City Council, 2) *Traffic Light* from OpenStreetMap, and 3) *Attractiveness* from geo-tagged *Flickr* photos [100].

Algorithms. We compare the proposed *FHL-Cube* with the existing exact *MCSP* solutions: 1) *FHL* [41]: index for full space; 2) *FHL-Multi* [41]: index for each subspace; 3) *Sky-Dij* [20]; 4) *eKSP* [24]; 5) *CSP-CH* [30]. We do not compare with *COLA* [38] here since its query results are approximate. Among them, *Sky-Dij* and *eKSP* are index-free approaches and the index size of *CSP-CH* is out of memory when the dimension is larger than three.

Query Set. For each dataset, we randomly generate three sets of OD pairs Q_1 to Q_3 and each has 1000 queries. Specifically, we first estimate the diameter d_{max} of each network [103]. Then each Q_i represents a category of OD pairs falling into the distance range $[d_{max}/2^{4-i}, d_{max}/2^{3-i}]$. Then we assign the query constraints C to these OD pairs. Firstly, we compute the cost range $[C_{min}, C_{max}]$ for each OD. This is because if $C<C_{min}$, the optimal result can not be found then the query is invalid; and if $C>C_{max}$, the optimal result is the path with the shortest distance. Furthermore, to test the constraint influence on the query performance, we generate five constraints for each OD pair: $C = r \times C_{max} + (1-r) \times C_{min}$, with $r = 0.1, 0.3, 0.5, 0.7$ and 0.9. When r is small, C is closer to the minimum cost; and when r is big, C is closer to the maximum cost.

4.7.2 Experiment Results

Index Size and Construction Time. The results are shown in Figure 4.7. Since *Sky-Dij* and *eKSP* are index-free, they are not compared here. As for the *CSP-CH*, its construction time soars up to five orders of magnitude from 2-D to 3-D, which is caused by the explosion of the skylines on the entire graph as the dimension increases. Consequently, it fails to construct in the 4-D and 5-D graphs, so we do not show its results. *FHL-Multi* takes the longest time with the largest index since it constructs labels for each subspace. *FHL* constructs the index only for the full space, so it has the smallest index size. However, its construction time is still very long. For *FHL*-based algorithms, the most time-consuming part is on boundary tree construction, as the boundary graph are extremely dense and most edges stores skylines. Compared with *FHL* and *FHL-Multi* on indexing, *FHL-Cube* has the significant reduction on time cost and performs the best (*FHL* spends 15631 seconds on 5-dimensional indexing of *NY*, whilst *FHL-Cube* takes 7861 seconds). Its efficiency mainly embodies in two aspects: first, our principles of boundary tree construction reduce the tree height obviously, and then decrease the computation from a tree node to its ancestors corresponding to the vertices in other partitions; second, in each contraction

of tree construction, *Block Maintenance* reduces many concatenated paths before the cuberisation operation. In addition, its index size is much smaller than the *FHL-Multi*, but has a larger index than *FHL*. It is because our skyline cube is essentially another layer of path management index, so it may store the address of the same path multiple times at different cuboids. Nevertheless, it is still faster to construct even with larger index size.

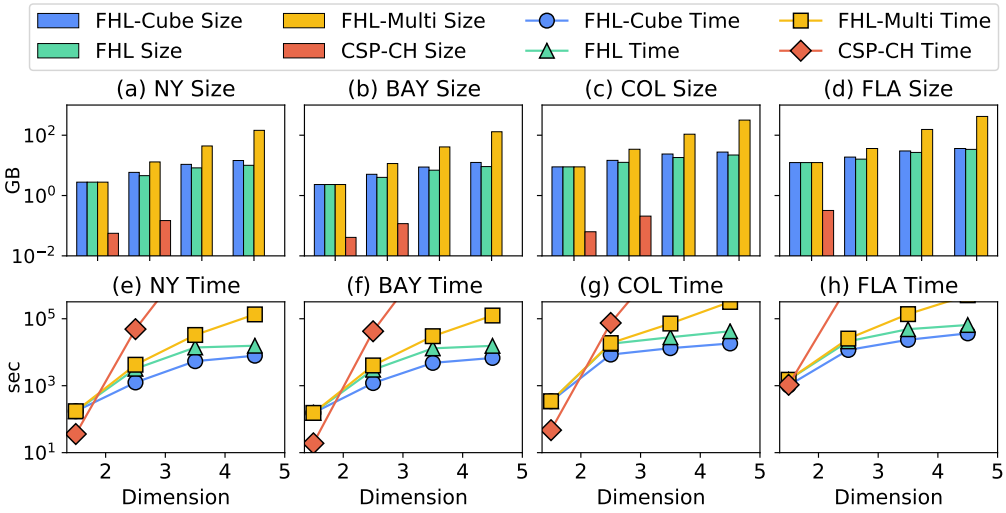


Figure 4.7: Index Size and Construction Time

Constraint Number. As shown in Figure 4.8, the query time of *eKSP*, *Sky-Dij* and *CSP-CH* are orders of magnitude higher than that of the *FHL*s'. This is because *eKSP* has to enumerate a huge amount of paths before one can satisfy the constraints, whilst the graph-search methods have accumulated a huge amount of intermediate skyline results and have to traverse the same edge multiple times. The performance of *FHL* with different constraint numbers is almost in the same order of magnitude because they use the same whole-space index. *FHL-Multi* is always faster than *FHL* in lower dimensions since it has an independent index in each subspace. The performance of all comparable algorithms deteriorates as the dimension grows. When we reach D^5, they present the same performance because they use exactly the same 5-D skyline index. *FHL-Cube* has the similar performance with *FHL-Multi* because the index size they use for concatenation is the same. However, it is faster than *FHL* in the other dimensions, because *FHL* answers the queries by the full-dimensional index, which results in more concatenated paths. Hence it will spend more time on identifying an optimal skyline path under the constraints. On the other hand, *FHL-Cube* can locate the subspace skyline index according to the constrained criteria. It also provides more powerful hop pruning on the corresponding subspace and produces fewer concatenated results.

Query Distance and Constraint Ratio Since the query processing efficiency of *eKSP*, *Sky-Dij* and *CSP-CH* are far behind the *FHL*s as shown in Figure 4.8, we do not include their performance in this section. As shown in Figure 4.9, the query time increases slightly as the query distance becomes

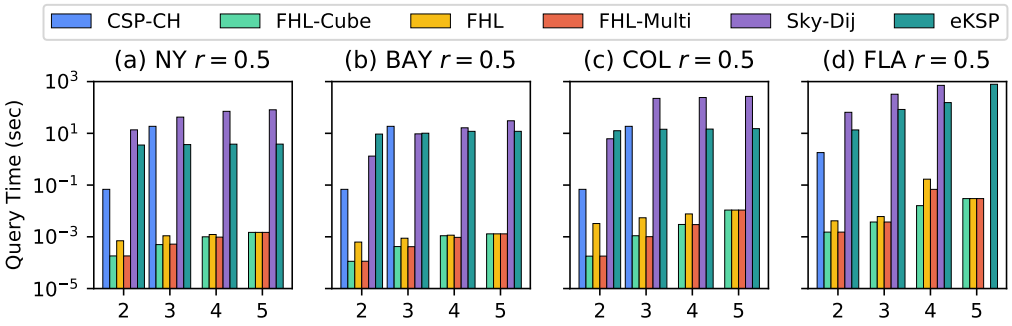


Figure 4.8: Comparison with MCSP Baselines ($r = 0.5$, Q_3)

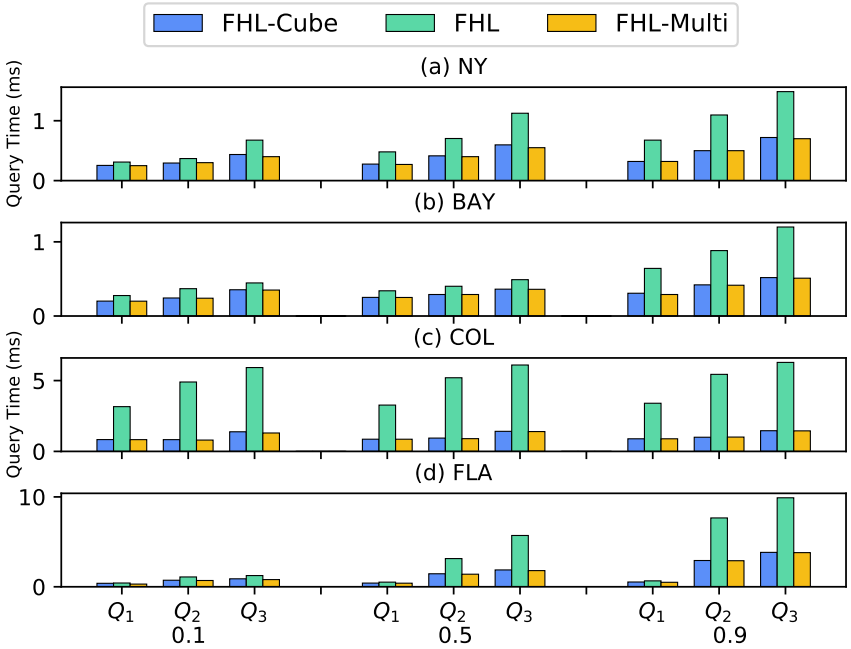


Figure 4.9: Query Time (3-Dimension)

longer. This is because that the longer path tends to pass through more partitions, which could cause more path concatenation. Nevertheless, the query performance is stable regardless of the distance changes. With the increase of constraint ratio, the query performance of the *FHLs* also keeps stable whilst other methods are very sensitive to the constraint ratio [39, 101]. Moreover, all of them perform the best when the constraint ratio is set to 0.1. In the case, we can prune more hops, because we can give a smaller concatenation range ahead of time. In summary, our method is robust to different query distances and constraint ratios.

Flexible MCSP. To test the performance of the flexible *MCSP* query, we generate the criteria combinations randomly. The number of constraints is set from 1 to 4 in Q_3 with $r = 0.5$, and the results are shown in Figure 4.10. *FHL-Multi* and *FHL-Cube* have the similar performance due to the similar index sizes used on any combinations. The performance of *FHL-Cube* is better than *FHL*, and the

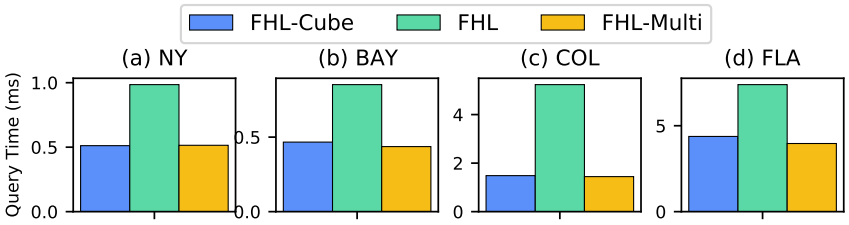


Figure 4.10: Flexible MCSP Query Time (Q_3, $r = 0.5$)

advantage is more obvious in larger graph like *COL*, because *FHL* always uses the full-dimensional index and produces more concatenated results. Conversely, *FHL-Cube* can find a smaller index size due to the hierarchical structure of *Cube*, and provide a smaller bound to prune calculative hops as much as possible.

Real Road Network. We test both directed and undirected on *NY Real*, with directed verions are labeled with "+". Moreover, each pair of adjacent vertices has two-way edges with the same attributes. As shown in Figure 4.11, the directed ones take double the construction and query time of the undirected ones with double index size. Both directed and undirected versions of *FHL-Cube* have the similar performance with *FHL-Multi* but better than *FHL*, because the random criterion *Attractiveness* produces a large number of skylines with *distance*. The number of skyline paths in the 5-D graph is far greater than the 2 to 4-D graphs'. Therefore, *FHL* has to consider several times more skylines than both *FHL-Cube* and *FHL-Multi*.

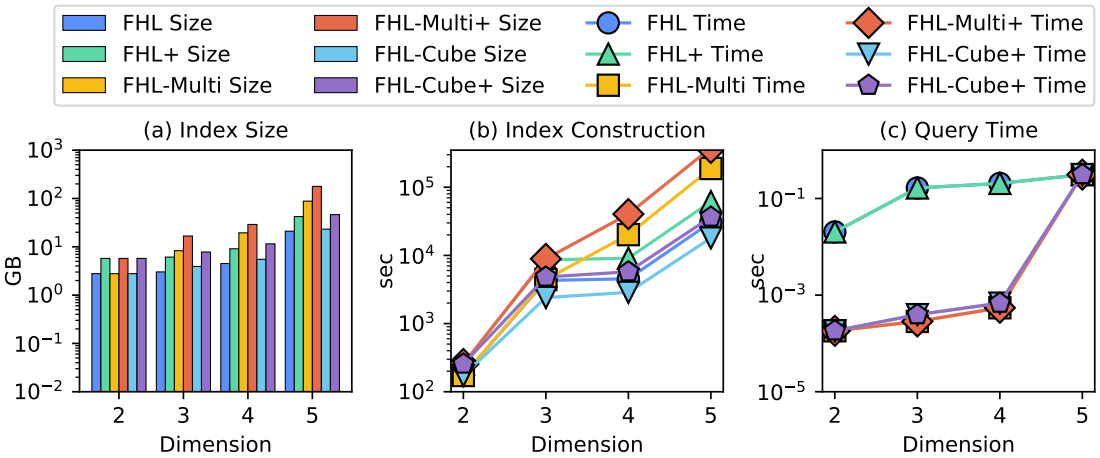


Figure 4.11: NY Real Criteria and Directed Evaluation

Remarks. Overall, our *FHL-Cube* is more efficient and practical to answer flexible *MCSP* queries than the baselines. Although the query performance of *FHL-Cube* is similar with *FHL-Multi*, its index size is much smaller. On the other hand, *FHL-Cube* achieves better query performance than *FHL* on both flexible and fixed *MCSP* query processing. As a result, even though its index size is slightly larger than

that of *FHL*, it takes shorter time to construct thanks to the efficient cube concatenation and optimized index structure. Given these points, *FHL-Cube* is more suited for flexible *MCSP* querying.

4.8 Conclusion

In this paper, we studied the flexible *MCSP* problem, which is the most generalised and practical multi-objective/constraint routing problem, and proposed the *FHL-Cube* index to handle it efficiently. By establishing the skyline path relation theories between the lower and higher dimensions, we introduced cubes to organise the skyline paths in different subspaces, derived higher dimensional skyline paths from lower ones, and improved the single- / multi-hop skyline cube concatenation for both index construction and query processing. We reduce the *FHL-Cube* construction time by optimising the structure and concatenation strategies. Extensive evaluations on real-life road networks show the priority of *FHL-Cube* compared with the state-of-the-art approaches in terms of construction time, index size, query processing efficiency and flexibility. Finally, to cope with the dynamic criteria like travel time, we could either resort to the speed profiles and create indexes for each time slot, or resort to index maintenance techniques [33, 34, 112] to update the skyline path labels accordingly.

The following publication has been incorporated as Chapter 5.

1.**Ziyi Liu**, Lei Li, Mengxuan Zhang, Wen Hua, and Xiaofang Zhou, Approximate FHL Index for Fast Constrained Shortest Path Query, submitted to *SIGMOD* on 15th January 2023.

Contributor	Statement of contribution	%
Ziyi Liu	Conception and design	70
	Analysis and interpretation	60
	Drafting and production	65
	proof-reading	55
Lei Li	Conception and design	10
	Analysis and interpretation	10
	Drafting and production	15
	supervision, guidance	50
	proof-reading	15
Mengxuan Zhang	Analysis and interpretation	10
	Drafting and production	10
	proof-reading	15
Wen Hua	Conception and design	10
	Analysis and interpretation	10
	Drafting and production	10
	supervision, guidance	50
	proof-reading	15
Xiaofang Zhou	Conception and design	10
	Analysis and interpretation	10

Chapter 5

Approximate Constrained Shortest Path Querying

The *Constrained Shortest Path (CSP)* problem aims to identify the shortest path between two vertices in a road network subject to a given constraint on another criterion. The solution to the *CSP* problem often follows the skyline path problem of the two criteria, resulting in an incredibly expensive computing cost for large road networks. The primary technical challenge is to deal with a huge number of partial skyline paths, which prevents existing index-based solutions from obtaining the exact skyline paths. In this paper, we propose α-*FHL*, a practical approximate method to avoid the expensive skyline path search and accelerate computation on skyline path indexing. α-*FHL* adapts the structure of *Tree Decomposition* to hierarchically allocate approximate ratios, leading to effective pruning, in the labeling index. Furthermore, we design different strategies to allocate the approximate ratios, and the efficient approximate concatenation method to answer the approximate *CSP* queries with the α-*FHL* index. Our approach ultimately achieves efficient query answering and fast index construction. Extensive experiments in real-life road networks demonstrate the superiority of our method compared to state-of-the-art solutions.

5.1 Introduction

Route planning is integral to our daily routines, and various path optimization challenges have been explored in recent years. Existing solutions primarily focus on a single objective, such as minimizing distance [1,2,32], travel time [4,5,95,96,113], fuel consumption [6,7], battery usage [8], or diversifying routes [114–117], among others. However, real-world route planning often requires accounting for various auxiliary criteria to meet user preferences, such as tunnel height, road capacity, slope gradient,

total elevation gain, duration of tourist drives, or the number of transportation changes. For instance, a user may need to limit the budget for tolls on highways, bridges, or tunnels; in populous cities, drivers often aim to make fewer left turns (assuming right-side driving) to avoid potential accidents. Therefore, traditional shortest path algorithms may not provide optimal solutions when these auxiliary criteria are considered. As a response, the *constrained shortest path (CSP)* method was developed. It identifies the optimal path based on a primary objective while adhering to a set of predefined constraints on other criteria. For example, with two objectives (cost c and distance d) and a maximum constraint C on c, the *CSP* solution selects the shortest path p with cost $c(p)$ less than C. Despite the absence of an upper limit on the number of possible criteria [21, 40, 41], our study concentrates on the single-criterion *CSP* problem. We chose this focus due to the following reasons: 1)It lays the groundwork for multi-criteria *CSP*; 2)Addressing the single-criterion *CSP* query itself poses substantial challenges; 3)Excessively combined criteria may render the routing system less usable, as the algorithm might not return any viable path.

Existing Solutions. Contemporary approaches to the queries broadly belong to two categories: exact solutions and approximate solutions. The exact *CSP* algorithms provide precise answers that fully meet the user's requirements. However, their computation is notably time-intensive, given their status as NP-Hard problems [14]. Moreover, they inherit the complexities of the skyline path search [17, 18, 20], which generates a set of paths, none of which dominate others across all criteria.

In the context of index-free *CSP* algorithms, every vertex visited retains a set of skyline paths originating from the source. The challenge lies in determining which paths should be retained to compose the final skyline path for subsequent searches. This process results in significant memory usage, which is why an exact index-free solution struggles to process *CSP* queries efficiently on real-world road networks. While exact index-based methodologies have been proposed, they present greater challenges, as balancing query efficiency with index size is nontrivial. For example, *CSP-CH* [30] preserves only a few skyline results on its shortcuts to maintain construction and index size, but this approach negatively impacts its query time. *Forest hop labeling (FHL)*-based methods [39–41] offer the quickest query processing by eliminating online searches with tree decomposition. However, its index is considerably larger than other index-based methods due to the need to store a set of exact skyline indices as 2-hop labels for each vertex.

Conversely, approximate *CSP* algorithms compromise on path length optimality to conserve computational time and indexing space. In particular, these solutions predefine an approximation ratio α, ensuring the length of the final path returned does not exceed α times the length of the optimal path. Most index-free approximate methods offer limited speedup compared to exact solutions because they still contend with high computational costs in large road networks. Currently, no approximate solution has been directly applied to the *CSP* index. *COLA* [38] represents the cutting-edge in approximate solutions. However, its performance enhancement is constrained as the approximation is applied to

search-based query processing rather than the index.

Non-Necessity Cases of Exact Methods. In real-world scenarios, the necessity for the exact *CSP* is limited. Firstly, the user's specific requirements can often be met with a good approximation rather than the exact shortest path. Consider a driver who wishes to minimize the travel distance while reducing toll charges. An approximate path that slightly extends the travel distance while effectively avoiding more toll roads can still satisfy the user's needs. Secondly, the exactness of a route can be compromised by the unpredictable nature of traffic, road conditions, and other dynamic factors. These uncertainties can transform an optimal route into a non-optimal one after the plan has been made, indicating that a near-optimal solution may suffice in many practical cases. Thirdly, the computational resources required to calculate the exact CSP can be prohibitive, especially when the constraints are complex or the network is extensive. Hence, an approximate CSP solution, which can provide near-optimal results with significantly less computational cost, is not only practical but can be more efficient for many applications.

Challenges and Contributions. In our quest for an efficient, fast-to-construct approximate *CSP* index, we propose the α-*FHL* index. This work builds upon the foundations set by [39], [40], and [41], which effectively integrate 2-hop-based skyline path concatenation for query processing and tree decomposition-based indexing. [39] investigates *CSP* query responses under two criteria, with a focus on techniques for constructing the 2-dimensional skyline index. The study capitalizes on the inherent properties of 2-dimensional skyline paths to facilitate their concatenation. Simultaneously, [41] expands on [39] to include multi-constraints shortest path queries. The focus is mainly on optimizing the efficiency of the multi-dimensional skyline path index. This optimization becomes imperative as the time complexity of skyline computation increases significantly with the introduction of multiple constraints. Moreover, the natural properties utilized in [39] offer limited optimization benefits in multi-constraints scenarios. On another note, [40] focuses on providing a more user-friendly querying approach that supports any combination of constraints in queries. The resulting index, termed skyline path cubes, organizes skyline paths across different dimensions. In this paper, we aim to build upon these previous efforts by approximating the skyline path index through the exploration of approximation ratio allocation on tree decomposition. Our goal is to boost the performance of indexing, streamline query processing, and minimize memory consumption. However, actualizing these goals presents substantial challenges, which we detail in subsequent sections.

Approximate Framework. Let F represent the solution for constructing the *FHL* index and Λ° symbolize the index result of graph G. Given the approximation ratio $\alpha \geq 1$ as a parameter of F, the optimal approximation result of $F(\alpha, G)$ would prune Λ° to $\alpha \otimes \Lambda^\circ$ such that $F(\alpha, G)$=$\alpha \otimes \Lambda^\circ \subseteq \Lambda^\circ$ ($\otimes$ indicates Λ° approximated by α to achieve the minimum approximated index). In $\alpha \otimes \Lambda^\circ$, no two skyline paths can α-dominate each other. A special case arises when α=1, such that $F(1, G)$=$1 \otimes \Lambda^\circ$=Λ°, which signifies the exact *FHL* [39] index. However, $\alpha \otimes \Lambda^\circ$ means that α is acting on Λ°,

thus suggesting the exact index is obtained first and then pruned by α. This process is expensive and more time-consuming than the exact *FHL*. As such, we propose an approximate *FHL* framework that prunes the index at different indexing phases. Specifically, the *FHL* indexing consists of two phases: *skyline tree decomposition* and *skyline label assignment*. In our framework, we design a bottom-up approximate tree decomposition to form trees with approximate skyline shortcuts, and a top-down approximate label assignment strategy. Notably, our approximation techniques revolve around the structure of tree decomposition, a subject yet to be thoroughly investigated in the existing literature.

Allocation of α. Allocating α for pruning during *FHL* index computation is challenging for several reasons: 1)Directly using α to calculate the approximate index leads to an over-approximation ratio in the results, yielding far from optimal *CSP* query answers; 2)A small approximation ratio applied at each iteration results in insufficient pruning power, generating a less effective and redundant index, thus impacting query processing efficiency; 3)Maintaining the same pruning power is difficult as each label may experience different approximation times; 4)A fixed allocation plan is inadequate to solve the problem. If we compute a label with high approximation, subsequent labels will be approximated insufficiently. Hence, we propose efficient methods to allocate α and recycle overused approximate power. Moreover, we theoretically analyze the relationship between α allocation and the tree decomposition structure, which can provide support for the approximate scheme of tree decomposition-based indices.

Skyline Operator. Skyline path concatenation, a crucial operator in both index construction and query processing, is time-consuming, particularly in large-scale graphs. To compute a label $L(v,u)$, skyline paths in $L(v,w_i)$ and $L(w_i,u)$, where w_i is a hop between v and u, must be concatenated. For multiple intermediate hops, the concatenated paths through each must be computed. Assume that we generate a set P_s including all concatenated results and select the final approximate skyline paths of P_s. Given that the size of hops is h, we will concatenate around mnh times. Although α-*FHL* can reduce concatenation times by approximating each label, dominance verification is still required in both single-hop and multiple-hop concatenations. This is because, we cannot guarantee the concatenated results from v to u remain skyline paths, requiring $O(mn\log(mn))$ time to verify their approximate dominance relations. To expedite this process, we designed an efficient approximate algorithm α-SPC_1 for single-hop concatenation and a pruning technique α-SPC_n to reduce the number of intermediate hops. Our contributions are summarized as follows:

- We offer an approximate framework for *FHL* with a theoretical guarantee of our approximation's correctness.

- We suggest a threshold to selectively approximate the indexing labels and a recycling technique to reuse redundant approximate power on labels.

- We introduce an efficient operator to concatenate skyline paths obtained from different approximation ratios, enabling quick discovery of concatenated skyline results approximated by a specific ratio.
- We provide a comprehensive evaluation of our methods using extensive real-life road networks, showcasing superior performance over current methods in terms of query time, index size, and construction time.

5.2 Problem Statement

A road network is an undirected graph $G(V,E)$, where V denotes a set of vertices, and $E \subseteq V \times V$ signifies a set of edges. Each edge $e \in E$ is characterized by two metrics: weight $w(e)$ and cost $c(e)$. A path p extending from the source $s \in V$ to the target $t \in V$ is a sequence of consecutive vertices. Each path p has an associated weight $w(p)$ and a cost $c(p)$. Our work utilizes an undirected road network to simplify descriptions and lay emphasis on relations between approximation ratio and tree structure. Additionally, it can be easily extended to a directed version.

Next, we introduce the concept of an α-*CSP query*, which serves as an approximation to the *CSP* query.

Definition 5.1 (α**-CSP Query**). *Given a graph G, a source s, a destination t, a maximum cost bound* $C \in \mathbb{R}^+$*, and an approximation ratio* $\alpha \in [1,2]$*, an* α*-CSP query* $q(s,t,C,\alpha)$ *returns a path p with the weight* $w(p) \leq \alpha \cdot w(\hat{p})$ *while guaranteeing that* $c(p) \leq C$*, where* $\hat{p}$ *is the result of the exact CSP query* $q(s,t,C)$*.*

Problem Definition. Given a graph G, our goal is to construct an index L capable of efficiently processing any α-*CSP* query $q(s,t,C,\alpha)$.

Unlike the index for the shortest path queries, which merely need to store the shortest distance from one vertex to another, the α-*CSP* index stores skyline paths for answering queries with any constraint C. In addition, the *CSP* paths are essentially a subset of the *skyline* paths, and the α-*CSP* paths are a subset of the *CSP* paths. We will discuss it in Section 5.2.1.

5.2.1 Skyline Path

Initially, we establish the *dominance* relation between any two paths with identical sources and destinations:

Definition 5.2 (α**-Path Dominance**). *Given two paths* p_1 *and* p_2 *with the same s and t,* p_1 α*-dominates* p_2 *iff* $w(p_1) < \alpha \cdot w(p_2)$ *and* $c(p_1) \leq c(p_2)$*, or* $w(p_1) \leq \alpha \cdot w(p_2)$ *and* $c(p_1) < c(p_2)$*.*

Definition 5.3 (**Skyline Path**). *Given any paths p_1 and p_2 with the same s and t, p_1 and p_2 are skyline paths if they cannot α-dominate each other.*

Hereafter, we employ α_i-skyline path to emphasize the skyline paths that cannot α_i-dominate each other.

Definition 5.4 (**Skyline Path Set**). *In a set $P(s,t)$ of paths from s to t, we can obtain a set $P_s(s,t) \subseteq P(s,t)$ of skyline paths based on the definition 5.2. P_s is maximal when α=1.*

Consider that we can set α=1 to generate the complete skyline index for *CSP* query processing.

Example 5.1. *In Figure 5.1-(a), each edge contains two criteria (w,c). When α=1, we find three skyline paths from v_3 to v_7: p_1=$\langle v_3,v_2,v_7\rangle$=$(2,8)$; p_2=$\langle v_3,v_1,v_2,v_7\rangle$=$(3,7)$; p_3=$\langle v_3,v_1,v_7\rangle$=$(6,3)$. The other paths from v_3 to v_7 are dominated, e.g., p_4=$\langle v_3,v_2,v_1,v_7\rangle$=$(7,6)$ is dominated by p_3. Once we increase α to 1.5, p_1 is dominated by p_2, because $w(p_2)$=$\alpha \cdot w(p_1)$=3 and $c(p_2) < c(p_1)$. Thus, the skyline paths from v_3 to v_7 remain p_2 and p_3.*

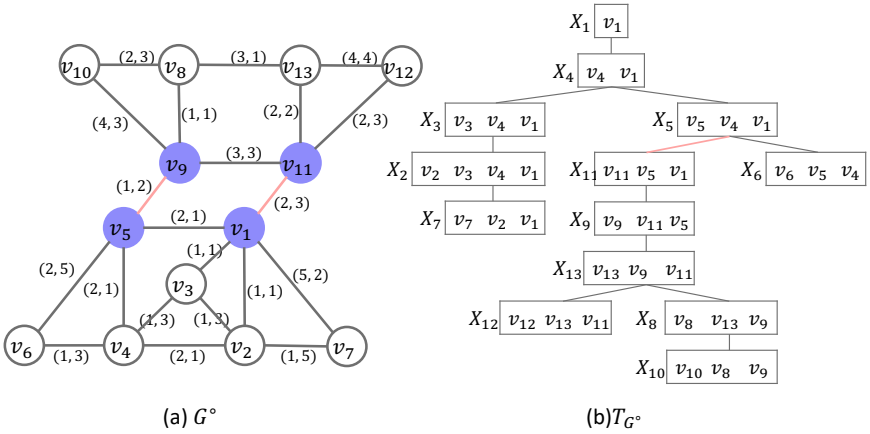


Figure 5.1: Example network G° with its tree decomposition T_{G°

Theorem 5.1 (**Non-empty skyline path set**). *For any $\alpha \geq 1$, if two vertices s and t are reachable in graph G, the cardinality of the set $P_s(s,t)$ is at least one, i.e., $|P_s(s,t)| \geq 1$.*

Proof. When vertices s and t are reachable, $|P(s,t)| \geq 1$. If $|P(s,t)|$=1, the sole path in $P(s,t)$ becomes a skyline path in $P_s(s,t)$. If $|P(s,t)| > 1$, any path p with the minimum cost in $P(s,t)$ cannot be α-dominated by other paths, given that α applies to $w(p)$. So p belongs to $P_s(s,t)$ for any $\alpha \geq 1$. □

Theorem 5.2 (**Completeness of Skyline Index**). *The skyline paths between any pair of vertices are both complete and minimal for every conceivable α-CSP query.*

Proof. The completeness of the skyline index can be established in a manner similar to that presented in [39]. □

Therefore, constructing an index for an α-*CSP* query is equivalent to creating the index for a skyline path query.

5.2.2 Forest Hop Labeling (FHL) Index

The *FHL* data structure is derived from the underlying tree decomposition.

For the sake of brevity, we use $\mathcal{W}_v$ and $\mathcal{H}_v$ to denote the tree width and tree height of X_v in T_G,

Example 5.2. *In T_{G° of Figure 5.1-(b), $\mathcal{W}_{G^\circ}$=3, $\mathcal{H}_{G^\circ}$=8, $\mathcal{W}_{v_2}$=3, and $\mathcal{H}_{v_2}$=4.*

The *FHL* index is specifically designed to answer *CSP* queries. Thus, α is set to one by default. To mitigate the large index size introduced by lengthy skyline paths, *FHL* initially partitions G into a set $\{G_1, \ldots, G_k\}$ of vertex-disjoint subgraphs such that $\bigcup_{i\in[1,k]} V(G_i)$=$V(G)$, and $V(G_i) \cap V(G_j)$=$\emptyset$ for all $i \neq j, i, j \in [1,k]$. An edge connecting two subgraphs is referred to as a *boundary edge*, like the edges (v_5, v_9) and (v_1, v_{11}) in Figure 5.1-(a). These edges divide the example graph into two partitions: G_1° comprising $\{v_1, v_2, v_3, v_4, v_5, v_6, v_7\}$ and G_2° containing $\{v_8, v_9, v_{10}, v_{11}, v_{12}, v_{13}\}$. The terminal vertices of a boundary edge are termed *boundary vertices*.

Moreover, *FHL* extracts all boundary edges and their associated vertices to form a *boundary graph*. Importantly, *FHL* precomputes the skylines between every pair of boundary vertices in each partition to ensure correct index construction, adding shortcuts for them if they are not connected in the original graph. Subsequently, it maps each subgraph onto a small tree structure via tree decomposition. Analogously, the boundaries of the small trees form a boundary tree. The small trees and the boundary tree together constitute a forest.

Example 5.3. *When decomposing along G_1° and G_2°, the original tree T_{G° is essentially bifurcated into two distinct subtrees. As depicted in Figure 5.1-(b), upon severing at the indicated red line, the upper segment forms $T_{G_1^\circ}$, while the lower segment constitutes $T_{G_2^\circ}$. The boundary graph G_b°, consisting of the edges depicted in red and solid vertices as seen in Figure 5.1-(a), subsequently produces a boundary tree $T_{G_b^\circ}$ (not shown).*

Based on the tree decomposition of each subgraph and a boundary graph, *FHL* computes the labels as defined below.

Definition 5.5 (Forest Hop Labeling (FHL)). *For each subgraph and a boundary graph $G_i(V_i, E_i) \subseteq G$, FHL computes the skyline label $L(v)$ for every $v \in V_i$. The label $L(v)$ stores the skyline paths from v to each of its ancestors on T_{G_i}, and $L(v, a_i)$ denotes the skyline paths from v to its ancestor a_i; that is, $L(v, a_i)$=$P_s(v, a_i)$.*

FHL Indexing

The construction of the index on each subgraph and a boundary graph, as detailed in Algorithm 1, comprises two main phases: a bottom-up Skyline Tree Decomposition, which collects the skyline shortcuts, generates the tree nodes, and builds the tree, and a top-down Skyline Label Assignment, which creates the labels necessary for query answering.

We briefly introduce the two phases shown in Algorithm 7.

Phase1: Skyline Tree Decomposition. The construction of the corresponding tree nodes for each vertex takes place in this phase, involving the contraction of vertices in a particular sequence (lines 1-5). During each iteration, we employ *minimum degree elimination* [18, 63] to dictate the contraction order. Once a vertex v is contracted, its vertex set X_v includes v and its neighbour set $N(v)$. The contraction of vertex v leads to the generation of skyline shortcuts $P_s(u,w)$ between each pair of neighbours (u,w) in $N(v)$, accomplished by concatenating skyline paths from v to u and those from u to w. After creating all the tree nodes, we form a tree by assigning the parent of each X_v as X_u, where $u \in X_v$ is the vertex with the lowest rank in X_v (lines 6-7).

Phase2: Skyline Label Assignment. This phase involves the population of labels in a depth-first approach from the tree root, employing the labelling of its ancestors (lines 8-13). Specifically, we assign labels from X_v to all its ancestors X_u to v since the vertices in X_v naturally establish the *cut property* between v and all its ancestors.

Algorithm 7: *FHL* Index Construction

Input: Graph $G(V,E)$
Output: CSP-2Hop Labeling L

1 **while** $v \in V$ *has the minimum degree and* $V \neq \phi$ **do**
2 $\quad X_v$=$N(v)$, $r(v) \leftarrow$ Iteration Number ▷ Tree Node Contraction
3 $\quad$ **for** $(u,w) \leftarrow N(v) \times N(v)$ **do**
4 $\quad\quad P_s(u,w) \leftarrow Skyline(P_s(u,v) \oplus P_s(v,u),(u,w))$;
5 $\quad\quad E \leftarrow E \cup (P_s(u,w)), V \leftarrow V - v, E \leftarrow E - (u,v) - (v,w)$;
6 **for** v *in* $r(v)$ *increasing order* **do**
7 $\quad u \leftarrow min\{r(u) | u \in X(v)\}, X_v.Parent \leftarrow X_u$ ▷ Tree Formation
8 $X_v \leftarrow$ Tree Root, $Q.insert(X_v)$ ▷ Label Assignment
9 **while** $X_v \leftarrow Q.pop()$ **do**
10 $\quad$ **for** $u \in \{u | X_u$ *is ancestor of* $X_v\}$ **do**
11 $\quad\quad P_s(v,u)) \leftarrow Skyline(P_s(v,w) \oplus P_s(w,u)), \forall w \in X_v$;
12 $\quad\quad L(v) \leftarrow (u, P_s(v,u))$;
13 $\quad Q.insert(X_u.children), L \leftarrow L \cup L(v)$;

FHL Querying

The structure of *FHL* comprises a *cut property*, which is utilized to address queries. For $\forall s,t \in V$, where X_s and X_t represent their corresponding tree nodes without an ancestor/descendant relation, their cuts are found in the *lowest common ancestor (LCA)* node [62].

For queries involving s and t located on different small trees in the forest, the *CSP* results are derived with the assistance of the boundary tree.

Considering a *CSP* query q=(s,t,C), in the case where s and t belong to the same tree, the first step is to determine the common intermediate hop set H=$\{h|h \in X(LCA(s,t)\}$ by extracting the vertices in the *LCA* of s and t. The term h is specifically employed to denote the vertices in the *LCA*, so that $P_c(s,h,t)$ symbolizes the skyline path candidates extending from s to t via h. Following this, $\forall h_i \in H$, the computation of skyline path candidates $P_c(s,h_i,t)$ is performed by executing $P_s(s,h_i) \oplus P_s(h_i,t)$. The set of candidates $P_c(s,t)$ is obtained by consolidating all $P_c(s,h_i,t)$ with the use of the following formula:

$$\begin{aligned} P_c(s,t) &= \{P_c(s,h_i,t) | \{P_s(s,h_i) \oplus P_s(h_i,t)\}\}, \forall h_i \in H \\ &= \{\{p_j(w_j(s,h_i)+w_k(h_i,t), c_j(s,h_i)+c_k(h_i,t))\}\} \\ &\forall p_j \in P_s(s,h_i) \wedge p_k \in P_s(h_i,t) \end{aligned} \tag{5.1}$$

Here, each $P_c(s,h_i,t)$ encompasses all the pairwise skyline concatenation results.

If $s \in T_{G_1}$ and $t \in T_{G_2}$, let H_1 and H_2 be the boundary vertices in $T_{G_1}$and T_{G_2}, $P_c(s,t)$ is calculated in three steps:

Step 1: For each $h_1^i \in H_1$, we compute $P_s(s,h_1^i)$. The computation aligns with Equation 5.1 because both s and h_i^1 belong to the same tree.

Step 2: For each $h_2 \in H_2$, we compute $P_c(s,h_2)$ including all skyline candidates via $\forall h_1^i \in H_1$ by the equation:

$$P_c(s,h_2) = \{P_c(s,h_1^i,h_2) | \{P_s(s,h_1^i) \oplus P_s(h_1^i,h_2)\}\}, \tag{5.2}$$

where $\forall h_1^i \in H_1$, $P_s(s,h_1^i)$=$L(s,h_1^i)$ is stored in T_{G_1}, and $P_s(h_1^i,h_2)$ is computed using the labels in T_{G_b} since both h_1^i and h_2 are boundary vertices

Step 3: The calculation of $P_c(s,t)$ involves the union of all $P_c(s,h_2^i,t)$ as follows:

$$P_c(s,t) = \{P_c(s,h_2^i,t) | \{P_c(s,h_2^i) \oplus P_s(h_2^i,t)\}\}, \tag{5.3}$$

where $\forall h_2^i \in H_2$, $P_c(s,h_2^i) \in P_c(s,h_2)$ is already computed in Step 1, and $P_s(h_2^i,t)$=$L(h_2^i,t)$ is recorded in T_{G_2}.

Ultimately, the optimal path that meets the constraint C of the *CSP* query q=(s,t,C) is found in $P_c(s,t)$.

Remark. The *FHL* index for $\forall v \in V$ includes a set $P_s(v,a_i)$ of skyline paths from v to a_i, with a_i denoting each ancestor of v in T_G. $P_s(v,a_i)$ is considered a label $L(v,a_i)$ stored in $L(v)$. The comprehensiveness of the skyline path index restricts the scalability of *FHL*, as the information about the skyline path can be extensive in dense or large-scale networks. It's inevitable that *FHL* generates a larger volume of labels, particularly for vertices situated far from tree root.

5.3 α-*FHL* Concatenation

The α-*FHL* utilizes a hop-based indexing method, which designates approximate labels to each vertex within G. In contrast with the *FHL*, α-*FHL* prunes each label that has an approximation ratio exceeding 1. This section introduces α-*skyline path concatenation (α-SPC)*, a fundamental operation to build an α-*FHL* index and to address α-*CSP* queries.

Given P_α, which symbolizes a set of α-skyline paths, $P_{\alpha_1}(s,t)$ denotes a set of α_1-skyline paths that originates at s and terminates at t. Here, we first establish a complete P_α.

Definition 5.6 (Complete α-Skyline Path Set). *Assuming that $P_s(u,v)$ comprises all exact skyline paths from s to t, $P_\alpha(u,v)$ is complete if it encompasses the maximum count of α-skyline paths derived from $P_s(u,v)$. We denote the complete $P_\alpha(u,v)$ as $\mathcal{P}_\alpha(u,v)$.*

Definition 5.7 (α-Skyline Path Concatenation (α-SPC)). *The concatenation result $\mathcal{P}_\alpha(u,v)$ is calculated by two operators: 1) α-SPC_1 computes $\mathcal{P}_{\alpha_0}(u,h_i,v)$=$\mathcal{P}_{\alpha_1}(u,h_i) \oplus \mathcal{P}_{\alpha_2}(h_i,v)$, $\forall h_i \in H$', where H' signifies a set of screened hops between u and v. 2) α-SPC_n initiates by pruning the hops in H, which encompass all intermediary hops and forms $H' \subseteq H$ to direct the concatenation of α-SPC_1. It then forms the union $P_\alpha(u,v)$=$\bigcup_{h_i \in H'} \mathcal{P}_\alpha(u,h_i,v)$, and retrieves $\mathcal{P}_\alpha(u,v)$ from $P_\alpha(u,v)$.*

Observe that α-SPC_1 operates on a single hop, while α-SPC_n emphasizes pruning across multiple hops. Henceforth, we notate $\mathcal{P}_\alpha(u,v)$ as $\mathcal{P}_\alpha$ and $\mathcal{P}_\alpha(u,h_i,v)$ as $\mathcal{P}_\alpha(h_i)$, given the context clarity.

5.3.1 Operator α-SPC_1

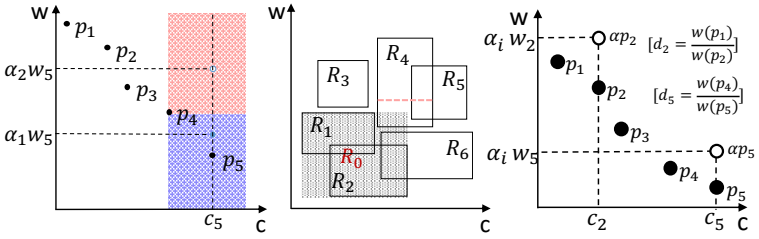
In α-SPC_1, the operator $\oplus$ is similar to skyline path concatenation of *FHL*, we need to first compute the weight and cost for a set P_c of concatenated paths and then verify their α-dominance. However, it is a little tricky to verify $\mathcal{P}_\alpha$ from P_c, because a correct $\mathcal{P}_\alpha$ is highly dependent on the verification order in P_s. For the exact skyline paths, we can directly drop a path p in P_c once we find another path p' dominating p, but this may result in errors when computing $\mathcal{P}_\alpha$. We use an example to explain this.

Within α-SPC_1, the operator $\oplus$ is similar to the skyline path concatenation of *FHL*. We first compute the weight and cost of a set P_c of concatenated paths and then confirm their α-dominance. However, verifying $\mathcal{P}_\alpha$ from P_c is differ significantly, as a correct $\mathcal{P}_\alpha$ heavily depends on the order of verification in P_s. For exact skyline paths, we can promptly discard a path p in P_c once another path p' dominates p. However, this method potentially leads to inaccuracies when deriving $\mathcal{P}_\alpha$. The following example further elaborates on this concept.

Example 5.4. *In the set P_c=$\{(6,14),(7,12),(8,12),(9,10)\}$, the exact skyline path P_s can be obtained by comparing each pair by weight-increasing order and discarding dominated ones, resulting in*

P_s=$\{(6,14),(7,12),(9,10)\}$. *However, with* $\alpha = 1.35$, $(6,14)$ *is* α*-dominated by* $(7,12)$, *and* $(8,12)$ *is* α*-dominated by* $(9,10)$, *leaving only* $(9,10)$ *in* $\mathcal{P}_\alpha$. *But, the correct* $\mathcal{P}_\alpha$ *is* $\{(6,14),(9,10)\}$.

Therefore, α-SPC_1 utilizes a greedy concatenation method to generate and verify concatenated paths in a cost-increasing order (i.e., the criterion without α). This guarantees the correctness of $\mathcal{P}_\alpha$ due to two reasons: 1)It is ascertained that the first concatenated path with the minimum cost is an α-skyline path; 2)For the current concatenated path p_i (the subscript indicates the order of produced paths), it is solely required to verify if the current concatenated path can be α-dominated by the preceding α-concatenated skyline path. For instance, if p_{i-1} is verified as an α-skyline path, if p_{i-1} is confirmed as an α-skyline path, the dominance of p_i is verified by comparing $\alpha \times w(p_{i-1})$ with $w(p_i)$. If p_{i-1} fails to dominate p_i, all the skyline paths formulated before p_i are incapable of dominating p_i. Additionally, p_i cannot be dominated by subsequent concatenated paths due to their costs being higher than $c(p_i)$.



(a) Approximate Greedy Concatenation on the Current Concatenated Path (p_5)

(b) Example of Approximate Rectangle-Pruning

(c) Example of F_2-R ($d_5 < d_2 < \alpha_i$)

Figure 5.2: α-*FHL* Pruning Techniques

Example 5.5 (α-SPC_1)**.** *As illustrated in Figure 5.2-(a), the current concatenated path is* p_5, *and paths from* p_1 *to* p_4 *have already been verified as* α*-skyline paths. These are plotted on a cost-weight axis. The domination areas of* p_4 *are demonstrated, where all points in the red area are dominated by* p_4. *When* α>1, *the domination of* p_5 *falls into two cases: 1) if* α=α_1, *it generates a point* $(\alpha_1 w_5, c_5)$ *located in the blue area, such that* p_5 *is an* α_1*-skyline path; 2) if* α=α_2, *where* α_2>α_1, *the point* $(\alpha_2 w_5, c_5)$ *is located in the dominating area (red) of* p_4, *such that* p_5 *is* α_2*-dominated by* p_4.

5.3.2 Operator α-SPC_n

In G, there are likely many intermediate hops for an *origin-destination (OD)* pair (u,v). To enhance the computation of α-SPC, α-SPC_n incorporates an approximate rectangle-pruning method aimed at reducing the number of h_i to be considered. Essentially, this method identifies a range R_i of $\mathcal{P}_\alpha(h_i)$, and subsequently determines a range R_0 of $\mathcal{P}_\alpha$ before concatenating at each h_i. If any R_i is outside of R_0, the concatenation at h_i can be skipped.

The range of concatenated skyline paths $P_s(h_i)$ is determined by the following theorem:

Theorem 5.3 (**Endpoint Skyline**). *Given two labels $L(u,h_i)$ with n skylines and $L(h_i,v)$ with m skylines, both sorted in increasing weight order. Their two endpoint skyline paths are p_1, p_n and p'_1, p'_m, respectively. We denote $p_1^1 = p_1 \oplus p'_1$ and $p_n^m = p_n \oplus p'_m$. Consequently, p_1^1 and p_n^m must be the skyline paths in the concatenated results of $L(u,h_i)$ and $L(h_i,v)$.*

For each hop h_i, p_1^1 is the first concatenated path and p_n^m is the last concatenated path. Among the paths via h_i, p_1^1 has the smallest weight with the largest cost, and P_n^m has the smallest cost with the largest weight. Thus, the mapping points of p_1^1 and p_n^m form a rectangular range R_i of concatenated results $P_s(h_i)$. Comparing all the endpoint skylines of the rectangles, we select the point with the smallest weight and the other with the smallest cost to form a range R_0, which can cover all the concatenated skyline paths in $L(u,v)$. Thus, we can prune all hops if their corresponding rectangles are outside R_0. Nevertheless, We can further narrow down the rectangles by applying an α. For a hop h_i, suppose that the weight of the last concatenated path through h_i is w_{min}, which provides the lower-bound of R_i, then we create a new lower-bound of dominance using $\alpha \times w_{min}$. Thus, the shrunken range has a higher chance of being further pruned by others.

Example 5.6 (α-*SPC*$_n$). *Figure 5.2-(b) illustrates the computation of $L(u,v)$ considering six hop vertices (h_1 to h_6) in the LCA of X_u and X_v. We generate six rectangles like R_3, representing the range of skyline paths from u to v via hop h_3. The path with the lowest cost maps to the upper-left endpoint and the one with the lowest weight to the bottom-right. Hops h_3 and h_5 are pruned as R_3 and R_5 lie outside R_0. R_1-R_4 are further scrutinized using α. The red line inside R_4 depicts its dominance lower-bound. As the entire shrunken range of R_4 falls within R_1's first quadrant, R_4 can be pruned, ensuring that α-skyline paths in R_1 consistently weigh less and cost less than those in R_4.*

5.4 α-FHL Framework

This section presents a theoretical analysis of the α-*FHL* framework.

Let F be a solution to construct *FHL* index. Then, $F(\alpha, G)$ represents a function to build a *FHL* index Λ^α approximately, i.e., α-*FHL* indexing, on a graph G by α. For the specific case α=1, $F(1,G)$ returns the exact *FHL* index, which is denoted as Λ°. Thus, the naive method is $F(\alpha,G)$=$\alpha \otimes \Lambda^\circ$, which means that we obtain Λ^α by allocating α on each label of Λ°. Then, we define $F(\alpha,G)$in the following.

Definition 5.8 (α-***FHL*** **indexing**). *Given $\alpha_1, \alpha_2 \leq \alpha$ and a graph G, the function $F(\alpha,G)$ includes two nested functions: 1) $g(\alpha_1,G)$, an approximate bottom-to-up tree decomposition function that returns*

approximate skyline shortcuts μ^{α} based on the tree structure; 2) $f(\alpha_2, g(\alpha_1, G))$, an approximate top-to-down label assignment function that returns Λ^{α}.

Therefore, $F(\alpha, G)=\Lambda^{\alpha}=f(\alpha_2, g(\alpha_1, G))$, which constructs α-*FHL* by approximating either Λ°, function f, or function g. Our α-*FHL* is thus generalized by the following equation:

$$f(\alpha_2, g(\alpha_1, G))=\begin{cases} F_1(\alpha, G)=\alpha \otimes \Lambda^{\circ} & \text{if } \alpha_1=1, \alpha_2=1 \\ F_2(\alpha, G)=f(\alpha, g(1, G)) & \text{if } \alpha_1=1, \alpha_2=\alpha \\ F_3(\alpha, G)=f(1, g(\alpha, G)) & \text{if } \alpha_1=\alpha, \alpha_2=1 \end{cases} \tag{5.4}$$

One may question why we cannot establish a scenario where $\alpha_1 \alpha_2=\alpha$. This is because, dispersing α would compromise the pruning power in both phases, rendering it insufficient. Hence, we conceptualize F_1 as a baseline that provides valuable insights for analyzing the correctness of queries and defines a lower bound for the index. Both F_2 and F_3 illustrate the impact of approximation on different index phases. Furthermore, given that *skyline tree decomposition* is built upon *contraction hierarchies (CH)*, the design of F_3 can offer technical support for the approximated skyline index on *CH*

5.4.1 F_1 Index

This section introduces $F_1(\alpha, G)=\alpha \otimes \Lambda^{\circ}$ as the baseline of α-*FHL*. When $\alpha=1$, $F_1(1, G)$ reduces to *FHL*, yielding the exact index $1 \otimes \Lambda^{\circ}=\Lambda^{\circ}$. As α increases, the likelihood of pruning more paths in Λ° rises, thus resulting in a smaller $\alpha \otimes \Lambda^{\circ}$ subset of Λ°. We first define the label in $\alpha \otimes \Lambda^{\circ}$ as follows:

Definition 5.9 (F_1 **label).** *In Λ°, each label $L_{\alpha_k}(v, a_i)$ is assigned an approximation ratio $\alpha_k<\alpha$, such that all the skyline paths in $L_{\alpha_k}(v, a_i)$ from v to their ancestor a_i cannot α_k-dominate each other, $\forall v \in V$.*

Then, we can establish the following lemma:

Lemma 5.4. *$L_{\alpha_k}(v, a_i)$ in Λ° is inadequate if it cannot encompass all the α_k-skyline paths from v to a_i, i.e., if $L_{\alpha_k}(v, a_i)$ adopts a larger approximation ratio than α_k. Conversely, $L_{\alpha_k}(v, a_i)$ is not the smallest if it includes additional $\alpha_{k'}$-skyline paths, which can be α_k-dominated, where $\alpha_{k'}<\alpha_k$.*

Example 5.7. *When $\alpha=1.5$, consider a label $l=\{(3,5),(6,2)\}$. The label l is not the smallest if $l=\{(2,6),(3,5),(6,2)\}$, as $(2,6)$ can be α-dominated. Furthermore, $l=(6,2)$ is inadequate.*

Theorem 5.5 (**Approximate Label Error**). *Pruning each label in Λ° by α will result in inaccuracies in the query $q(s, t, C, \alpha)$.*

Proof. This can be illustrated through the following example:

Suppose $L(v, w)=\{(4,4),(3,6)\}$, $L(w, u)=\{(5,6),(3,8)\}$, and w is the unique cutting vertex from v to u. Given a query $q(u, v, C=15, \alpha_0=1.35)$, we approximate each label by $\alpha_0=1.35$: $L(v, w)$ updates to

$\{4,4\}$ as $(3,6)$ is α_0-dominated by $(4,4)$ while $L(w,u)$ remains the same. After concatenation, we get two paths $(9,10)$ and $(7,12)$. As $7 \times 1.35 > 9$, $(7,12)$ is α_0-dominated by $(9,10)$. Thus, only $(9,10)$ remains in $L(v,u)$ and is a result of q. However, if we consider the exact result of $\langle u,w,v \rangle$, we get three exact skyline paths $\{(9,10),(7,12),(6,14)\}$. Because $(9,10)$ can $\alpha_\circ$-dominate $(7,12)$, we drop $(7,12)$ and compare $(9,10)$ with $(6,14)$. As $(6 \times 1.35 < 9)$, $(6,14)$, which satisfies the constraint 15, cannot be α_0-dominated, $(6,14)$ is the correct answer but not $(9,10)$. Therefore, assigning α_0 directly to $L(v,w)$ results in erroneous result. □

Correctness of F_1. Each label $L_{\alpha_k}(v,a_i)$ must: (1) be minimum; and (2) answer all α-*CSP* queries accurately. Condition 1) is straightforwardly demonstrated as F_1 directly assigns an α_k to each exact label of Λ° such that each label cannot α_k-dominate any other. As for condition 2), for any origin-destination (OD) pair (s,t), $\alpha_0 \otimes \Lambda^\circ$ must support a querying method capable of finding all the α_0-skyline paths $\mathcal{P}_{\alpha_0}(s,t)$. This ensures that the α-*CSP* query from s to t with a variable C can be correctly answered.

In α-SPC_1, $\mathcal{P}_{\alpha_1}(s,h_i) \oplus \mathcal{P}_{\alpha_2}(h_i,t)$ must accurately compute $\mathcal{P}_{\alpha_0}(s,h_i,t)$ for $\forall h_i \in H'$. To achieve condition (2), the concatenation of all α_1-skyline paths and α_2-skyline paths must cover all α_0-skyline paths. In other words, the correctness of $\mathcal{P}_{\alpha_0}$ is contingent on α_0, α_1, and α_2. Consequently, we arrive at the following theorem.

Theorem 5.6 (Concatenate Relation). *The α-SPC_1 correctly calculates $\mathcal{P}_{\alpha_0}(s,h_i,t)$ as $\mathcal{P}_{\alpha_1}(s,h_i) \oplus \mathcal{P}_{\alpha_2}(h_i,t)$, given $\alpha_1\alpha_2$ equals α_0.*

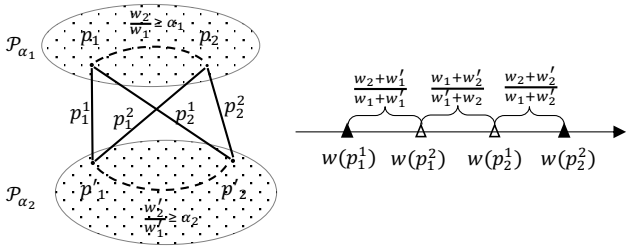
Proof. The calculation $\mathcal{P}_{\alpha_1}(s,h_i) \oplus \mathcal{P}_{\alpha_2}(h_i,t)$ performs all-pair concatenations. This results in a butterfly structure illustrating the concatenation relationships between any two paths in $\mathcal{P}_{\alpha_1}$ and in $\mathcal{P}_{\alpha_2}$. Figure 5.3-(a) presents an example of an extreme butterfly. In this figure, the sets $\{p_1,p_2\}$ and $\{p'_1,p'_2\}$ denote two α_1-skyline and α_2-skyline paths in $\mathcal{P}_{\alpha_1}$ and $\mathcal{P}_{\alpha_2}$, respectively. The notation $w_1/w_2/w'_1/w'_2$ refers to the weights of $p_1/p_2/p'_1/p'_2$, respectively, with $w_1 < w_2$ and $w'_1 < w'_2$. The concatenation generates four paths, $P_c = \{p_1^1, p_1^2, p_2^1, p_2^2\}$, depicted in Figure 5.3. For instance, $p_1^2 = p'_1 \oplus p_2$. In the butterfly structure, p_1^1 and p_2^2 are termed non-cross paths, while p_1^2 and p_2^1 are called cross paths.

This structure elucidates several important aspects. Assume that the smallest gap between w_1 and w_2 occurs in $\mathcal{P}_{\alpha_1}$. This suggests that $\frac{w_2}{w_1}$ is the closest value to α_1 among all pairs in $\mathcal{P}_{\alpha_1}$. Likewise, the smallest gap in $\mathcal{P}_{\alpha_2}$ is found in the pair (p'_1,p'_2), indicating that $\frac{w'_2}{w'_1}$ is the value nearest to α_2. Thus, $\frac{w_2}{w_1} \geq \alpha_1$ and $\frac{w'_2}{w'_1} \geq \alpha_2$. The weights of the concatenated paths are subsequently plotted on the weight axis, as shown in Figure 5.3-(b). The largest gap between any two concatenated paths in P_c is $\frac{w(p_2^2)}{w(p_1^1)}$. Based on these observations, we establish two arguments: (1) $\frac{w(p_2^2)}{w(p_1^1)} \leq \alpha_1\alpha_2$; and (2) $\frac{w(p_2^2)}{w(p_1^1)} \geq \alpha_0$.

Argument (1): $\frac{w(p_2^2)}{w(p_1^1)} \leq \alpha_1\alpha_2$. Firstly, it can be derived that $\frac{w_2+w_1'}{w_1+w_1'} \leq \frac{w_2}{w_1}$ and $\frac{w_2+w_2'}{w_2+w_1'} \leq \frac{w_2'}{w_1'}$. Hence, $\frac{w(p_2^2)}{w(p_1^1)}$ equals $\frac{w_2}{w_1} \cdot \frac{w_2'}{w_1'}$, which is less than or equal to $\alpha_1\alpha_2$. Given that both $\frac{w_2}{w_1}$ and $\frac{w_2'}{w_1'}$ are closest to α_1 and α_2, when $|\mathcal{P}_{\alpha_1}|$ and $|\mathcal{P}_{\alpha_2}|$ approach positive infinity, the limits of $\frac{w_2}{w_1}$ and $\frac{w_2'}{w_1'}$ become α_1 and α_2, respectively. This ensures $\frac{w(p_2^2)}{w(p_1^1)} \leq \alpha_1\alpha_2$.

Argument (2): $\frac{w(p_2^2)}{w(p_1^1)} \geq \alpha_0$. This is proven by contradiction. If $\frac{w(p_2^2)}{w(p_1^1)} < \alpha_0$, none of the paths in the butterfly could be a α_0-skyline path, except for p_2^2. This is because the concatenated paths are verified in cost-increasing order, and the cost $c(p_2^2)$ in P_c is the smallest. Consequently, p_2^2 will prune all other paths. The α-SPC_1 then adds another path p_0 or p_0' to the concatenation. Let's assume the next concatenated path is p_0^2, resulting in a new gap $\frac{w(p_2^2)}{w(p_0^2)}$ which is less than or equal to $\frac{w_2'}{w_0'}$. This implies that $\frac{w_2'}{w_0'} \geq (\alpha_2)^2$, and hence, the valid paths $\mathcal{P}_{\alpha_2}$ are also $(\alpha_2)^2$-skyline paths. Consequently, the completeness of $\mathcal{P}_{\alpha_2}$ diminishes. Therefore, $\frac{w(p_2^2)}{w(p_1^1)}$ must be greater than or equal to α_0.

From Arguments (1) and (2), we can deduce that $\alpha_0 \leq \frac{w(p_2^2)}{w(p_1^1)} \leq \alpha_1\alpha_2$. Hence, the concatenation is correct when α_0 equals $\alpha_1\alpha_2$. □



(a) (b)

Figure 5.3: approximation ratios of α-SPC_1 in F_1

The Operator $\otimes$. The forest structure of α-*FHL* necessitates varying concatenation patterns. For a query (s,t,C,α), if the *OD* pair (s,t) resides within the same tree, α-*SPC* is executed once. Conversely, if $X_s \in T_1$ and $X_t \in T_2$, α-*SPC* is performed three times. Consequently, the operator $\otimes$ on each label in Λ° must facilitate accurate α-*CSP* query processing, even for the longest concatenation pattern. The operator $\otimes$ in F_1 distributes an approximation ratio evenly among each small tree and the boundary tree, i.e., $\alpha \otimes \Lambda^\circ$ approximates each label in Λ° by $\alpha^{1/6}$, ensuring that each label $L_{\alpha^{1/6}}(v,a_i)$ is calculated and each tree receives an allocation of $\alpha^{1/3}$.

Processing of the F_1 Query. The longest concatenation pattern warrants further discussion. Assuming sets H_1 and H_2 comprise all boundary vertices of T_1 and T_2 respectively, the following concatenations are considered: 1) $\mathcal{P}_{\alpha^{1/3}}(s,h_i)$ for $\forall h_i \in H_1$ in T_1; 2) $\mathcal{P}_{\alpha^{2/3}}(s,h'j)$ for $\forall h'j \in H_2$, calculated by $\mathcal{P}_{\alpha^{1/3}}(s,h_i)$ and $\mathcal{P}_{\alpha^{1/3}}(h_i,h'j)$ from T_1 to T_b; and 3) $\mathcal{P}_{\alpha}(s,t)$, computed by $\mathcal{P}_{\alpha^{2/3}}(s,h'j)$ and $\mathcal{P}_{\alpha^{1/3}}(h'_j,t)$ from T_b to T_2.

Analysis of F_1. In a real road network, the pruning power of F_1 on each label proves significant because α is not overly diluted. However, F_1 incurs considerable expense during index construction due to the prerequisite construction of an Λ° prior to approximation. Despite this, through the analysis of F_1, we identify potential issues relating to the allocation of α and elucidate the relationship between the number of concatenations and the allocation of α. This analysis is integral to the design of F_2 and F_3.

5.5 F_2 Index

This section details $F_2(\alpha,G)$=$f(\alpha,g(1,G))$, the pruning labeling index during the *approximate label assignment* phase. F_2 introduces three methods for approximating the labeling index: 1)F_2-E approximates each label based on the tree structure; 2)F_2-θ establishes a threshold θ and approximates labels exceeding this threshold; 3)F_2-R reuses wasted local ratios in subsequent concatenations.

5.5.1 Approximate Label Assignment

Given the forest structure of α-*FHL*, we acknowledge that each tree will accommodate a portion of α. To distinguish between various approximation ratios, we provide the following definitions:

Definition 5.10 (**Approximate Global Ratio**). *Each smaller tree T_i possesses an approximate global ratio, denoted by α_i°. The approximate global ratio for the boundary tree is represented as α_b°. Hence, for any two values α_i° and α_j°, we have $\alpha_i^\circ \alpha_j^\circ \alpha_b^\circ$=$\alpha$.*

Definition 5.11 (**Approximate Local Ratio**). *Within tree T_k, each label $L(v,a_i)$ is assigned an approximate local ratio $\alpha_k(v,a_i)$, where $\alpha_k(v,a_i) \in (1,\alpha_k^\circ]$.*

Hereafter, we refer to $\alpha_k(v,a_i)$ as $\alpha(v,a_i)$ and α_i° as α° if the tree is unspecified.

In $F_2(\alpha,G)$=$f(\alpha,g(1,G))$, F_2 initially constructs the same skyline tree decomposition as in *FHL*. For $\forall v \in V$, it extracts a set of exact skyline shortcuts $P_s(v,u)$, where $u \in N(v)$. Subsequently, F_2 performs a top-to-bottom *Approximate Label Assignment* to compute $\alpha(v,a_i)$-skyline paths for each label $L(v,a_i)$. This leads to the following equation:

$$L(v,a_i)=\mathcal{P}_{\alpha(v,a_i)}=\alpha\text{-}SPC_n(\bigcup_{u\in N(v)} P_s(v,u)\oplus L(u,a_i)) \tag{5.5}$$

where a_i and u are ancestors of v. Additionally, u is a neighbour of v recorded during the graph contraction in *Skyline Tree Decomposition*. Therefore, $L(u,a_i)$=$\mathcal{P}_{\alpha(u,a_i)}(u,a_i)$ is computed before $L(v,a_i)$. The operator α-SPC_n serves to prune the vertices in $N(v)$ and validate the $\alpha(v,a_i)$-skyline paths from the union.

As per Equation 5.5, the paths in $L(v,u)$ may partake in the concatenations for distinct labels as a set of subpaths. This number is tied to traversed hops from v to u in the tree. Moreover, each concatenation of a path can be seen as an approximation operation on it. Consequently, we propose the following theorem:

Theorem 5.7 (**Concatenation and Approximation**). *The number of concatenations a path in $L(v,u)$ undergoes equals the number of times the path is approximated minus one.*

Proof. Theorem 5.6 provides the basis for our proof. Suppose $L(v,a_2)$ results from the concatenation of $L(v,a_1)$ and another label L_1=$\mathcal{P}\alpha_1$. Hence, we have $\alpha(v,a_i)\alpha_1$=$\alpha(v,a_2)$. This means that the paths in $L(v,a_1)$ transform into a set of sub-paths *Psub* in $L(v,a_2)$. To obtain $\alpha(v,a_2)$, P_{sub} is approximated by α_1. When $L(v,a_2)$ concatenates with a label L_2=$\mathcal{P}\alpha_2$, the paths in *Psub* are further approximated by $\alpha(v,a_i)\alpha_1\alpha_2$. As a result, if part of the paths in P_{sub} undergoes n concatenations and are saved as index, they are approximated by $\alpha(v,a_i)\alpha_1\alpha_2...\alpha_n$. □

With reference to Theorem 5.7, we derive the following corollary.

Corollary 5.8 (**Approximate Times Towards Ancestors**). *Given a label $L(v,a_i)$, the maximum number of approximations a path $p \in L(v,a_i)$ undergoes is equivalent to the count of ancestors from v to a_i.*

Therefore, for $p \in \mathcal{P}_{\alpha_\circ}$, assume p is composed of subpaths $\langle p_1,\ldots,p_k\rangle$, we can then express this relationship with the following inequality:

$$w(p_1)\prod_{i=1}^{k}\alpha_i + w(p_2)\prod_{i=2}^{k}\alpha_i + ... + w(p_k)\prod_{i=k}^{k}\alpha_i \leq \alpha_\circ w(p) \tag{5.6}$$

Here, k denotes the maximum number of approximation iterations.

F_2 is designed in adherence to Inequality 5.6. When we concatenate a label $L(v,a_i)$, it is crucial that each concatenated path in $L(v,a_i)$ satisfies Inequality 5.6. Failing to meet this inequality could result in an insufficient labelling index, as the sub-paths on the left side of Inequality 5.6 could be overly approximated, leading to the pruning of several essential skyline paths. Consequently, F_2 assigns an approximation ratio to each label, ensuring that the results of its concatenation align with Inequality 5.6.

5.5.2 F_2-E

Our initial method involves allocating an identical approximate local ratio for each hop on a tree. For a vertex v where X_v has n ancestors and X_{a_1} is the lowest ancestor of X_v, we set $\alpha(v,a_1)=\alpha^{\frac{1}{n}}$. The following theorem encapsulates this process.

Theorem 5.9 (**Even Approximate Local Ratio**). *For a given label $L(v,a_i)$, where the tree height is $\mathcal{H}$, and X_v has k ancestors from X_v to X_{a_i} (including X_v), we allocate $\alpha^{\frac{k}{\mathcal{H}-1}}$ to $L(v,a_i)$.*

F_2-E Indexing. As shown in Algorithm 8, the input is T_G, constructed during the *skyline tree decomposition*. We calculate the labels for each tree node from the root (line 1). For each tree node X_v, we compute labels from v to each ancestor vertex u (lines 3-7). If the number of ancestors between v and u is k, then $\alpha(v,u)=\alpha^{\frac{k}{\mathcal{H}-}}$. We then employ $\alpha(v,u)$ to approximate the concatenated result of $L(v,u)$ in the operator α-SPC_n.

Algorithm 8: Approximate Label Assignment

Input: Tree T_G with shortcuts by index contraction
Output: α-*FHL* Labeling L

1 $X_v \leftarrow$ Tree Root, $Q.insert(X_v)$ **while** $X_v \leftarrow Q.pop()$ **do**
2 $\mathcal{H} \leftarrow$ tree height;
3 **for** $u \in \{u | X_u$ *is ancestor of* $X_v\}$ **do**
4 $k \leftarrow$ the number of ancestors between v to u;
5 $\alpha(v,u)=\alpha^{\frac{k}{\mathcal{H}-1}}$;
6 $L(v,u) \leftarrow \alpha$-$SPC_n((L(v,w) \oplus L(w,u)), \forall w \in X_v \leftarrow \alpha_v$;
7 $L(v) \leftarrow (u, L(v,u))$;
8 $Q.insert(X_u.children), L \leftarrow L \cup L(v)$;

Analysis of F_2-E. The determination of each approximate local ratio hinges upon the established tree structure, thereby linking the approximate power to the tree height. For a label $L(v,u)$, should X_v and X_u be proximate within a tree of substantial height $\mathcal{H}$, $\alpha(v,u)$ will approach 1. Conversely, F_2-E apportions excessive approximation ratios to labels at the base of the tree when k closely approximates n. This can result in smaller-sized labels wasting pruning power. We thus propose two optimized versions based on F_2-E in the ensuing section.

5.5.3 F_2-θ and F_2-R

F_2-θ

In F_2-θ, we introduce a threshold θ during *Approximate Label Assignment*. For a label $L(v,u)$, if its size exceeds θ, an approximate local ratio is allocated. If not, we retain its concatenated result.

$|L(v,u)|$ can be confirmed after an exact concatenation. Nonetheless, this approach necessitates computing all exact skyline paths and subsequently approximating labels that satisfy θ, which can be extremely time-consuming.

Instead of calculating an exact $|L(v,u)|$, F_2-θ determines the number of concatenated paths $\mathcal{N}$ using the sizes $|L(u,h_i)|$ and $|L(h_i,v)|$. It then derives a weight upper bound w_{max} and a weight lower bound w_{min} prior to concatenation. Hence, $\theta{=}\log_{\alpha(v,u)}(w_{max}/w_{min})$, where $\alpha(v,u)$ is the approximate local ratio of $L(v,u)$. We prune $L(v,u)$ if its $N > \log_{\alpha(v,u)}(w_{max}/w_{min})$.

F_2-R

F_2-R serves as an optimized variant of F_2-E. Assuming w is a tree root, the principal idea of F_2-R is to reclaim the pruning power of a label from u to the tree root, then repurpose the reclaimed ratio to compute $L(v,u)$. The reclaimed ratio is denoted as $\alpha.rec$. Accordingly, we propose the following equation.

$$\alpha(v,a_i){=}(\alpha^{\circ})^{\delta/\mathcal{H}-1}\cdot\alpha.rec(a_i,root) \tag{5.7}$$

where $\delta{=}\mathcal{H}-\mathcal{H}_{a_i}$. To compute each local ratio $\alpha(v,a_i)$, we need to derive $\alpha.rec$ via a recycling operator. We emphasize that the recycling operator is only activated on the labels that include the root.

Example 5.8 (**Local Ratio Recycling**). *In Figure 5.1(b), we take tree nodes X_1, X_2, X_3, and X_4 as an example. Suppose that we recycle $\alpha.rec(v_4,v_1)$ from $\alpha_{\circ}^{\frac{1}{7}}$ from $L(v_4,v_1)$, then $\alpha(v_4,v_1){=}\alpha^{\frac{1}{7}}/\alpha.rec(v_{11},v_1)$. When we compute $L(v_3)$, we create two labels $L(v_3,v_1)$ and $L(v_3,v_4)$. We obtain $L(v_3,v_4){=}(\alpha^{\circ})^{\frac{1}{7}\cdot\alpha.rec(v_4,v_1)}$. $\alpha(v_3,v_1)$ is still $\alpha_{\circ}^{\frac{2}{7}}$. Then, we recycle $\alpha.rec(v_3,v_1)$. For computing $L(v_2)$, $\alpha(v_2,v_4)$ is $\alpha_{\circ}^{\frac{2}{7}}\cdot\alpha.rec(v_4,v_1)$ and $L(v_2,v_3)$ is $\alpha_{\circ}^{\frac{1}{7}}\cdot\alpha.rec(v_3,v_1)$. $\alpha(v_2,v_1)$ is $\alpha_{\circ}^{\frac{3}{7}}$ and we execute the recycle operator on it.*

Before we discuss the recycling operator, we define the lower bound of dominance in the following.

Definition 5.12 (**Lower-bound of Domination**). *Given a set of paths in cost-increasing order $\{p_1,p_2,...,p_k,...,p_n\}$, and an approximation ratio α_i, if p_k is α_i-dominated by p_{k-1}, $d_k{=}w(p_{k-1})/w(p_k)$ is a lower bound of p_k. Thus, d_k is the lowest approximation ratio to dominate p_k.*

Recycle Operator is integrated into the α-SPC_1 of each label $L(v,u)$. It calculates the lower bound d_k for each dominated path p_k from v to u and compares d_k with $\alpha(v,u)$. If $d_k{=}\alpha(v,u)$ is found, we do not recycle $\alpha(v,u)$. Otherwise, we select the maximum d_k denoted by d_{max}, and set $\alpha.rec(v,u){=}\alpha(v,u)/d_{max}$.

Example 5.9. *In Figure 5.2(c), α-SPC_1 produces six paths via h_i with a cost-increasing order. The first α_i-dominated path is p_2, we compute $d_2{=}w(p_1)/w(p_2)$. Then, we get rid of p_2 and concatenate p_3. However, p_3 cannot be α_i-dominated by p_1, so we do not compute d_3. The second α_i-dominated path*

is p_5, and d_5=$w(p_4)/w(p_5)$. Subsequently, if d_5<d_2<α_i, we can recycle α_i/d_2 and the concatenate results are not affected.

5.6 F_3 Index

In this section, we discuss $F_3(\alpha, G)$=$f(1, g(\alpha, G))$, with a focus on the *Skyline Tree Decomposition* phase. For every vertex $v \in V$, F_3 approximates the skyline shortcuts $P_s(v,u)$ from v to each neighbour u in the contracted graph, with the restriction that the pruning power of each shortcut cannot exceed α°.

During the contraction process, shortcuts between neighbours u and w fall into three distinct situations: (1)Both (v,u) and (v,w) are original edges of G; (2)(v,u) is an original edge of G, but (v,w) is a shortcut in the contracted graph; and (3)Both (v,u) and (v,w) are shortcuts in the contracted graph.

For situation one, a shortcut can be straightforwardly approximated with a local ratio. However, situations two and three pose challenges due to inequality 5.6. Specifically, $\alpha(v,u)\alpha(v,w) \leq \alpha^\circ$ must be considered for approximating the shortcut of a pair (u,w).

In the contraction process, we aim to establish efficient shortcuts with approximated weights based on the original edge weights w_i. This process relies on a designated contraction order. For each vertex contraction, shortcuts are created and their approximated weights are determined by multiplying the original edge weight with a predefined approximation ratio α_i. When subsequent vertex contractions introduce potential shortcuts between the same pair of nodes, we must update these paths by evaluating their approximated dominance. This assessment requires aligning shortcuts under the same approximation ratio. In cases where a newly introduced shortcut outperforms the existing one in terms of its approximated weight, the approximation ratio of the path is updated. In the scenarios, it is essential to ensure that the updated ratio does not exceed the global ratio α°.

Example 5.10. *Figure 5.4-(a) illustrates the approximation of shortcuts in a partition of G°. We contract vertices in the order v_5, v_2, v_3, v_4, v_1. Initially, we contract v_5(Figure 5.4-(b)), resulting in an approximation of shortcut weight $\alpha_1 w(v_1,v_4)$=$\alpha_1(w_1$+$w_2)$. After contracting v_2(Figure 5.4-(c)), the generated shortcuts yield distinct approximate weights, $\alpha_2 w(v_1,v_3)$=$\alpha_2(w_4,w_7)$, $\alpha_3 w(v_3,v_4)$=$\alpha_3(w_3$+$w_7)$, and $\alpha_4 w(v_1,v_4)$=$\alpha_4(w_3$+$w_4)$. We allocate $\alpha_1(v_1,v_4)$ when contracting v_5, so shortcuts from v_1 to v_4 are updated by verifying the approximate dominance. This requires the alignment of shortcuts with the same ratio, hence α_1=α_4. Consequently, we set α_1=α_2=α_3=α_4. The next contraction v_3 yields $\alpha_5 w(v_1,v_4)$=$\alpha_5(\alpha_2 w(v_1,v_3)$+$\alpha_3 w(v_3,v_4))$. This new shortcut is then combined with previous ones between v_1 and v_4, selecting the path with the largest ratio. This results in the update of local ratio of (v_1,v_4) to $\alpha_5\alpha_2$, where $\alpha_5\alpha_2 > \alpha_1$=$\alpha_2$. Lastly, we ensure $\alpha_5\alpha_2 \leq \alpha$.*

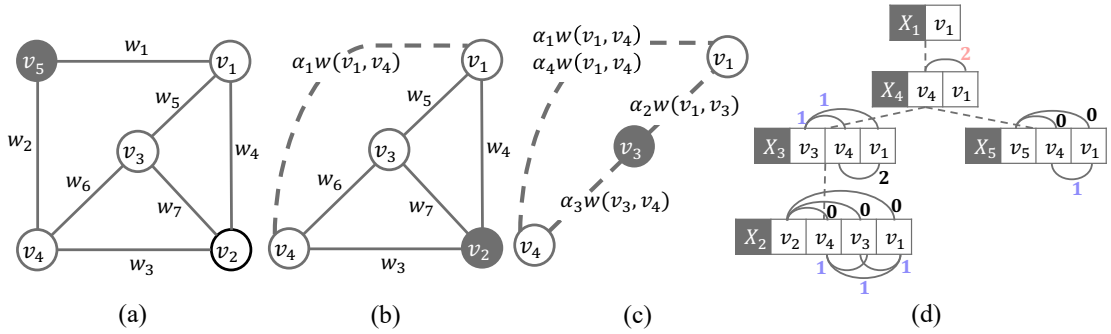


Figure 5.4: Example of F_3 indexing

Suppose we allocate local ratios evenly. In this case, we can determine the local ratio once we establish the greatest number of contractions necessary for computing a shortcut.

In light of this, F_3 initially creates a tree structure for each subgraph but skips the computation on shortcuts. It leads to a constructed tree only encompassing tree nodes with neighbourhood relations. Moving from the bottom to the top of the tree, F_3 computes the number of contractions for any pair (v,u) in every tree node as extra information. Specifically, we denote the number of contractions of any shortcut (v,u) involved as $\lambda(v,u)$. By applying Theorem 5.7, we can deduce that $\lambda(v,u)$ equals the highest approximation times of (v,u). For example, in Figure 5.4-(d), $\lambda(v_1,v_4)$=2, as the computation of a shortcut of (v_1,v_4) go through the rate α_2 and α_5. The computation of $\lambda(v,u)$ follows a cumulative process within the tree structure constructed, as discussed in the following.

Theorem 5.10 (**Cumulative Computation on λ**). *For a tree node X_v consists of $\{w,u\}$, if $\lambda(v,w)$=m and $\lambda(v,u)$=n, then $\lambda(w,u)=\max\{m,n\}+1$.*

Proof. The proof can be derived from Theorem 5.7, considering that each contraction of a shortcut can be seen as a concatenation operation. □

As shown in Figure 5.4-(d), which is the tree structure of Figure 5.4-(a), our computation starts from the leaves, thus we begin with X_2. The neighbours of v_2, namely v_4, v_3, and v_1, have no approximations for (v_2,v_4), (v_2,v_3), and (v_2,v_1), as they are the original edges. Hence, we assign their λ values as zero. According to Theorem 5.10, the other pair of neighbours can be calculated as 1, such as $\lambda(v_4,v_3)$, $\lambda(v_3,v_1)$, and $\lambda(v_4,v_1)$. Next, we move to the other leaf, X_5, and assign $\lambda(v_5,v_4)$ and $\lambda(v_5,v_1)$ as zero, resulting in $\lambda(v_4,v_1)$=1. Upon visiting the parent of X_5, we label $\lambda(v_4,v_1)$=1 in X_4. Similarly, while visiting the parent of X_2, we assign $\lambda(v_3,v_4)$ and $\lambda(v_3,v_1)$ as 1 based on the values in X_2. Consequently, $\lambda(v_4,v_1)$ in X_3 is calculated as 2. When travelling from X_3 to X_4, we update $\lambda(v_4,v_1)$ in X_4 from 1 to 2.

Next, we allocate an $\alpha(v,u)$ to $P_c(u,v)$ in the following:

Theorem 5.11 (**Local Ratio Allocation with λ**). *Assume that the largest λ in a tree T_G is n, and $\lambda(u,w)$=k for a set $P_c(u,w)$ of concatenated paths in X_v, we can allocate $\alpha^{\frac{k}{n}}$ to $P_c(u,w)$ for the dominance operation.*

Example 5.11. *We can assign $\alpha^{\frac{1}{2}}$ to $P(v_4,v_1)$ of X_5. As a result, the pruning power of $P(v_4,v_1)$ in X_4 increases to α because $\lambda(v_4,v_1)$ is updated to 2.*

Analysis of F_3. For each partition in the graph, a tree structure is created to track the number of contractions between each pair. The maximal number of contractions needed to create a shortcut determines the distribution of local ratios. During *skyline label assignment*, the approximate power may be diluted as no further consideration is given to the approximate ratio. Nonetheless, F_3 not only provides a more comprehensive index than F_2 but also offers a viable solution for approximating the *CH*-based index.

5.7 Experiments

We implement all algorithms in C++ with complete optimisations. The experiments are conducted in a 64-bit Ubuntu 18.04.3 LTS with two 8-core Intel Xeon CPU E5-2690 2.9GHz and 186GB RAM.

Datasets. All tests are carried out on four real road networks obtained from the 9^{th} *DIMACS* Implementation Challenge [1]. We use the length of the road as weight w and the travel time t as the cost. We list the information from the networks as follows:

- ***NY*** is a dense grid-like urban area with 264,246 vertices, 733,846 edges and several partitions connected by bridges;
- ***BAY*** has a doughnut shape around *San Francisco Bay* with 321,270 vertices, 800,172 edges;
- ***COL*** has an uneven vertex distribution (dense around Denver but sparse elsewhere) with 435,666 vertices and 1,057,066 edges;
- ***FLA*** is a large network Florida with 1,070,376 vertices and 2,712,798 edges.

Query Set. **Query Set**. For each dataset, we randomly generate three sets of *OD* pairs Q_1 to Q_5, each with 1000 queries. Specifically, we first estimate the diameter d_{max}, which is the longest length of all shortest paths in the graph [103]. Then each Q_i represents a category of *OD* pairs that fall in the distance range $[d_{max}/2^{6-i}, d_{max}/2^{5-i}]$. Afterwards, we assign the query constraints C to these *OD* pairs. First, we calculate the cost range $[C_{min}, C_{max}]$ for each *OD*. This is because if $C<C_{min}$, the optimal result cannot be found, then the query is invalid; and if $C>C_{max}$, the optimal result is the path with the

[1] http://users.diag.uniroma1.it/challenge9/download.shtml

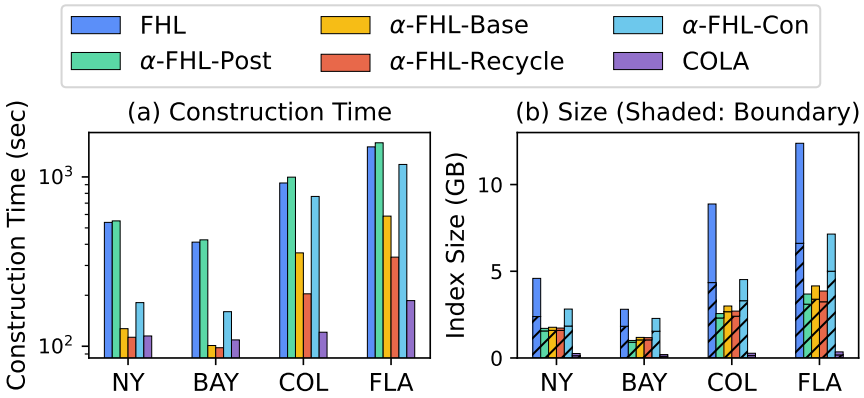


Figure 5.5: Index Construction Performance

shortest distance. Specifically, each *OD* pair has a constraint: $C{=}rC_{max}{+}(1-r)C_{min}$, with $r \in (0,1)$. When r is small, C is closer to the minimum cost; as r increases, C is closer to the maximum cost.

Algorithms. We compare our methods with *Sky-Dijkstra* [13] and *COLA* [38]. As our methods adapt different approximation strategies to *FHL*, we list them as follows.

- **α-*FHL*-Post** is F_1 index approximated on the exact *FHL*;
- **α-*FHL*-Base** is F_2-E, which assign a_i evenly;
- **α-*FHL*-Recycle** is the optimised α-*FHL*, including F_2-R and F_2-θ.
- **α-*FHL*-Con** performs F_3 in index contraction.

5.7.1 Index Size and Construction Time

Figure 5.5-(b) illustrates the index sizes of *FHL*,α-*FHL*s and *COLA* in four different road networks, when we set $\alpha = 1.1$. The index of *FHL* is exacted. Therefore, we compare it with α-*FHL* to show the effectiveness of α-*FHL* in pruning the index. For each bar, the shaded area shows the index size of the small trees, and the non-shaded area is the size of the boundary tree. We can observe that the best pruning result is α-*FHL*-Post, because it prunes the index based on the exact *FHL* index. The index size of α-*FHL*-Recycle is very close to α-*FHL*-Post. Moreover, all methods for approximate label assignments, including α-*FHL*-Base and α-*FHL*-Recycle, provide better results than α-*FHL*-Con, which approximates the index in index contraction. This is because the number of intermediate skyline paths generated in index contraction is much less than the paths populated in the label assignment. Thus, the pruning power of α-*FHL*-Con is restricted. Furthermore, the pruning in the boundary tree is the most significant for α-*FHL*s. On the other hand, *COLA* has the smallest index, as it mainly includes the skyline paths between the boundary vertices.

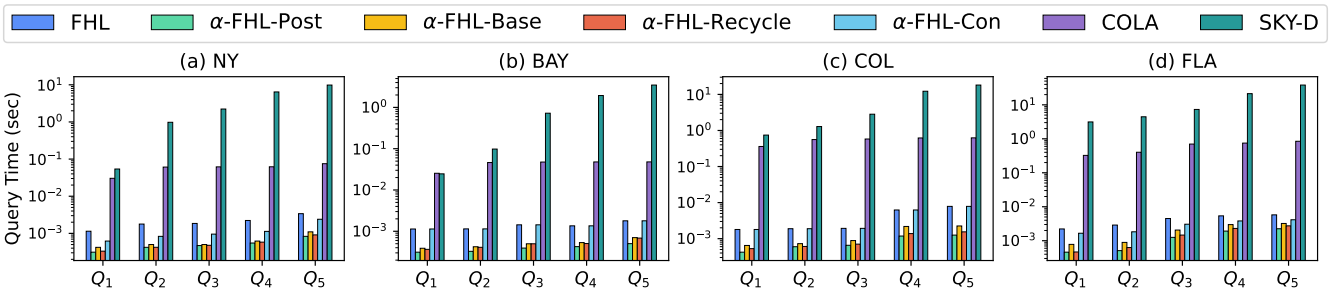


Figure 5.6: Query Time

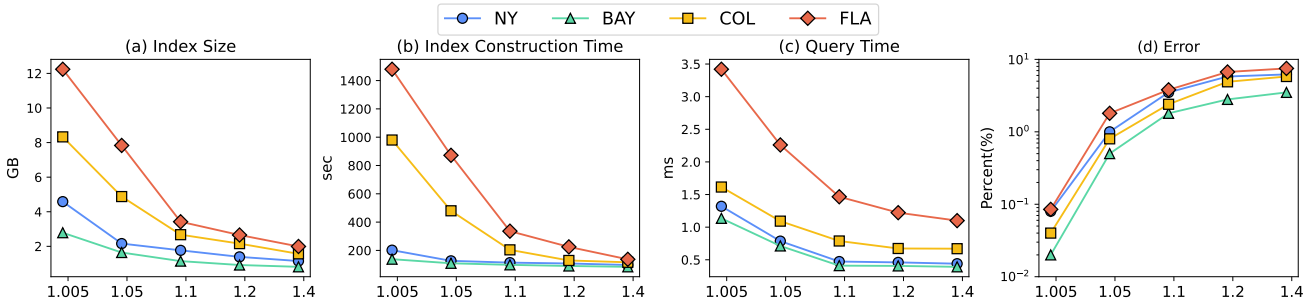


Figure 5.7: α-FHL-Recycle Experiment

Figure 5.5-(a) shows the index construction time of *FHL*,α-*FHL*s and *COLA* in four different road networks when we set $\alpha = 1.1$. α-*FHL*-Post takes the longest time. On the contrary, the indexing time of α-*FHL*-Recycle is the shortest among α-*FHL*s. For *COLA*, only the skyline paths between the boundary vertices are considered. Thus, it has faster pre-processing.

Among the α-*FHL*s, the performance of α-*FHL*-Recycle is the best in index construction, because it takes the shortest time to build the index, which is very close to the best approximate result. Although α-*FHL*-Recycle has a larger index size and longer preprocessing time than *COLA*, α-*FHL*-Recycle supports faster query processing, and index construction is still practical. Consider that α-*FHL*-Recycle can avoid slow online search in query processing, and its indexing cost is worth paying for.

5.7.2 Query Efficiency

Figure 5.6 presents the query processing time for the comparative methods when $\alpha = 1.1$ and $r = 0.5$. We plot the results on the four graphs from Q_1 to Q_5.

There are several observations. First, our α-*FHL*s outperform all other methods by several orders of magnitude. Especially, α-*FHL*-Recycle always answers queries within 1 millisecond, regardless of the query distance, in *NY* and *BAY*. Compared to online search methods, the query distance has less effect on the performance of our methods due to the hop labelling-based index. The query time

has slightly increased along with the distance because queries with larger distances may cross the tress and then experience multiple concatenations. Furthermore, among the α-*FHL*s, α-*FHL* has the best performance. This is because the smallest index size reduces the number of concatenated paths. Therefore, it saves the cost of the dominance operation. Subsequently, the performance of α-*FHL*-Recycle is very close to that of α-*FHL*-Post.

Combine with the performance of α-*FHL*-Recycle in index construction. We determine that α-*FHL*-Recycle is the most practical one among α-*FHL*s, because it has the fastest index contraction time. Moreover, the index size and query processing are similar to those of α-*FHL*-Post. Therefore, we will test α-*FHL*-Recycle with different α in Section 5.7.3.

5.7.3 α-*FHL* with Different α

We experimentally study the impact of α on α-*FHL*-Recycle. In Figure 5.7, we test the index size, constructing time, query execution time, and query accuracy by varying α. We plot the results for four different datasets.

Figure 5.7-(a) shows the index space of α-*FHL*-Recycle on *NY*, *BAY*, *COL* and *FLA*. We observe that the index size decreases when we change α from 1.005 to 1.4. The index sizes are pruned by more than half when $\alpha \geq 1.1$. The improvement in index construction time is also significant, as shown in Figure 5.7-(b). This is because a larger α results in more pruned paths during the label assignment phase. Since α-FHL computes the labels from top to bottom of the tree, the approximated labels are used for the computation of the deeper tree nodes. Increasing α will save time for the concatenation of each label, and thus reduce the indexing time.

In Figure 5.7-(c), we can observe that the query processing time is slightly reduced when we increase α. For explanation, α-*FHL*-Recycle answers queries by concatenating two labels. The increased pruning power results in the reduction of the labels for concatenating. Therefore, there are fewer concatenated paths satisfying our constraint on the attribute cost, thus saving the dominance verification time, especially for queries from different trees.

In the end, we randomly generate 500 queries to test the accuracy of our method. For each query, we compute its relative error by $w(p')/w(p) - 1$, where $w(p')$ is the weight of the exact result obtained by *FHL*, and $w(p')$ is the weight of α-*CSP*. As shown in Figure 5.7-(d), a higher α results in a larger relative error. The good balance of α is from 1.1 to 1.2 for α-*FHL*-Recycle.

5.8 Conclusion

In this paper, we proposed α-*FHL*, which is a novel approximate solution to answer α-*CSP* queries efficiently. In α-*FHL*, we propose approximate skyline path concatenation methods with pruning

techniques to support efficient index construction and query answering. By theoretically analysing the relations between the approximate ratio and tree decomposition structure, we provide effective strategies to prune the labelling index with a theoretical guarantee. Extensive evaluations on real-life road networks demonstrated that our approximate methods outperform the state-of-the-art *CSP* approaches on α-*CSP* query processing. Compared with the exact *FHL* index, our new methods significantly reduce the index construction time and size.

Chapter 6

Conclusion

In this thesis, we study constrained shortest paths with related problems and propose efficient methods to answer different types of *MCSP* queries, including *CSP*, *fixed MCSP*, *flexible MCSP*, and α-*CSP*.

In Chapter 3, we study the *CSP* and fixed *MCSP* problems. Based on our literature review, the issue that mainly affects the efficiency of query answering is the skyline path search. Some index-free methods may spend hours answering *MCSP* queries in a large-scale road network, even for the simplest version, i.e., *CSP* queries. State-of-the-art index-based solutions, such as *CSP-CH* and *COLA*, use some heuristics or hierarchical structures to speed up query processing. However, their querying methods are still based on online skyline path search and have to visit a large number of vertices, which limits their query performance. Therefore, we propose the efficient *FHL* index to totally avoid the expensive skyline path search, and then achieve efficient *CSP* query answering. Then, we extend *FHL* to the fixed *MCSP* by improving the skyline path concatenation operator in multi-dimensional space.

In Chapter 4, we propose *FHL-Cube* to answer flexible *MCSP* queries, which supports any combination of constraints setting. Our idea is to organise the skyline path index with different dimensions by cubes. The cuberisation operator derives the higher dimensional skyline paths from the 2-dimensional skyline paths for generating a hierarchical cube called skyline path derivation. Instead of expensive skyline computation of increased complexity in higher dimensions, we compute the 2D dimensional skyline paths at a small cost. Then we derive all the higher dimensional skyline paths without the computation on skyline path validation.

In Chapter 5, we study the approximate *CSP* queries. We adopt different strategies to compute the *FHL* index approximately. Compared to the exact version, our α-*FHL* achieves faster indexing and smaller memory consumption by sacrificing minor accuracy. It is the first work to analyse the correctness of allocating approximate ratios in *Tree Decomposition*, and provides the theoretical guarantee and guideline for *Tree Decomposition*-based solutions to answer approximate-related queries.

Future Research Direction

Specifically, we outline the main relevant research directions in the following. The first direction focuses on the skyline path index in dynamic road networks. Specifically, we study the index maintenance of the skyline path index on tree decomposition when edge weights increase or decrease in the road network. Given its focus on multi-criteria optimization, this poses a greater challenge for labeling in tree decomposition-based skyline path index compared to tree decomposition-based shortest path index. For example, a weight change could transform a previously non-skyline path into a new skyline path, invalidate a current skyline path, or even alter the dominance relationships among multiple paths. Currently, this work is nearly finished and ready to submit.

Secondly, we will consider both additive and non-additive criteria in the MCSP problem. Additive criteria, like travel time and distance, are combined using a weighted sum, and the FHL indices focus on these to identify optimal paths. Non-additive criteria, such as lane occupancy rate or time interval of green light, are different and can't be simply aggregated. Our goal is to design an algorithm that finds the path with the least weight, subject to constraints on both the summed additive criteria and individual non-additive criteria values for each edge in the path.

Next, we will address the distribution of criteria. Different areas have various structures, densities, criteria, and transportation focuses (e.g., rural and urban areas). Thus, I'm considering an indexing method to build indexes locally based on area-specific criteria and manage them globally. An area-specific graph partition method is also considered. Compared to an index considering all criteria at once, an area-specific method could benefit the index in terms of space consumption and construction.

We also interested in multi-dimensional road network embedding, which considers complex spatial and topological properties of road networks, such as road hierarchies, traffic patterns, and geographical constraints. The techniques aim to transform this complex structure into a continuous and compact vector representation, capturing the inherent properties and relationships among network elements. This will benefit various downstream tasks and improve state-of-the-art methods using traditional network representations.

The last direction is to identify the skyline cube using a learning model with skyline derivation. In high-dimensional space, skyline points no longer provide interesting insights, as there are too many. Therefore, it is interesting to design an online model to deduce valuable combinations of criteria and classify skyline entries. Additionally, an offline model is designed based on skyline derivation rules, aiming to ensure online classification accuracy and discover more logic deduction rules. Considering that skyline derivation in high-dimensional databases may be less efficient and the learning-based model might compromise accuracy, the purpose of this work is to explore how the components can benefit each other, leading to enhanced accuracy and efficiency.

Bibliography

[1] E. W. Dijkstra, A note on two problems in connexion with graphs, Numerische mathematik 1 (1) (1959) 269–271.

[2] L. Li, M. Zhang, W. Hua, X. Zhou, Fast query decomposition for batch shortest path processing in road networks, in: ICDE, 2020.

[3] M. Zhang, L. Li, X. Zhou, An experimental evaluation and guideline for path finding in weighted dynamic network, Proceedings of the VLDB Endowment 14 (11) (2021) 2127–2140.

[4] L. Li, S. Wang, X. Zhou, Time-dependent hop labeling on road network, in: ICDE, 2019, pp. 902–913. doi:10.1109/ICDE.2019.00085.

[5] L. Li, S. Wang, X. Zhou, Fastest path query answering using time-dependent hop-labeling in road network, TKDE (2020).

[6] L. Li, W. Hua, X. Du, X. Zhou, Minimal on-road time route scheduling on time-dependent graphs, PVLDB 10 (11) (2017) 1274–1285.

[7] L. Li, K. Zheng, S. Wang, W. Hua, X. Zhou, Go slow to go fast: minimal on-road time route scheduling with parking facilities using historical trajectory, The VLDB Journal 27 (3) (2018) 321–345.

[8] J. D. Adler, P. B. Mirchandani, Online routing and battery reservations for electric vehicles with swappable batteries, Transportation Research Part B: Methodological 70 (2014) 285–302.

[9] N. Jozefowiez, F. Semet, E.-G. Talbi, Multi-objective vehicle routing problems, European journal of operational research 189 (2) (2008) 293–309.

[10] N. Shi, S. Zhou, F. Wang, Y. Tao, L. Liu, The multi-criteria constrained shortest path problem, Transportation Research Part E: Logistics and Transportation Review 101 (2017) 13–29.

[11] T. Korkmaz, M. Krunz, Multi-constrained optimal path selection, in: INFOCOM, Vol. 2, 2001, pp. 834–843.

[12] H. De Neve, P. Van Mieghem, A multiple quality of service routing algorithm for pnni, in: 1998 IEEE ATM Workshop Proceedings, 1998, pp. 324–328.

[13] G. Tsaggouris, C. Zaroliagis, Multiobjective optimization: Improved fptas for shortest paths and non-linear objectives with applications, Theory of Computing Systems 45 (1) (2009) 162–186.

[14] P. Van Mieghem, F. A. Kuipers, On the complexity of qos routing, Computer communications 26 (4) (2003) 376–387.

[15] S. Chen, K. Nahrstedt, An overview of quality of service routing for next-generation high-speed networks: problems and solutions, IEEE network 12 (6) (1998) 64–79.

[16] Z. Wang, J. Crowcroft, Bandwidth-delay based routing algorithms, in: GLOBECOM, Vol. 3, IEEE, 1995, pp. 2129–2133.

[17] H.-P. Kriegel, M. Renz, M. Schubert, Route skyline queries: A multi-preference path planning approach, in: ICDE 2010, IEEE, 2010, pp. 261–272.

[18] Q. Gong, H. Cao, P. Nagarkar, Skyline queries constrained by multi-cost transportation networks, in: ICDE, IEEE, 2019, pp. 926–937.

[19] D. Ouyang, L. Yuan, F. Zhang, L. Qin, X. Lin, Towards efficient path skyline computation in bicriteria networks, in: DASFAA, Springer, 2018, pp. 239–254.

[20] P. Hansen, Bicriterion path problems, in: Multiple criteria decision making theory and application, Springer, 1980, pp. 109–127.

[21] J. M. Jaffe, Algorithms for finding paths with multiple constraints, Networks 14 (1) (1984) 95–116.

[22] G. Y. Handler, I. Zang, A dual algorithm for the constrained shortest path problem, Networks 10 (4) (1980) 293–309.

[23] L. Lozano, A. L. Medaglia, On an exact method for the constrained shortest path problem, Computers Operations Research 40 (1) (2013) 378 – 384.

[24] A. Sedeño-Noda, S. Alonso-Rodríguez, An enhanced k-sp algorithm with pruning strategies to solve the constrained shortest path problem, Applied Mathematics and Computation 265 (2015) 602 – 618. doi:https://doi.org/10.1016/j.amc.2015.05.109.

[25] M. R. Garey, D. S. Johnson, Computers and intractability, Vol. 174, freeman San Francisco, 1979.

[26] R. Hassin, Approximation schemes for the restricted shortest path problem, Mathematics of Operations research 17 (1) (1992) 36–42.

[27] A. Juttner, B. Szviatovski, I. Mécs, Z. Rajkó, Lagrange relaxation based method for the qos routing problem, in: INFOCOM, Vol. 2, IEEE, 2001, pp. 859–868.

[28] D. H. Lorenz, D. Raz, A simple efficient approximation scheme for the restricted shortest path problem, Operations Research Letters 28 (5) (2001) 213–219.

[29] F. Kuipers, A. Orda, D. Raz, P. Van Mieghem, A comparison of exact and ε-approximation algorithms for constrained routing, in: NETWORKING, Springer, 2006, pp. 197–208.

[30] S. Storandt, Route planning for bicycles—exact constrained shortest paths made practical via contraction hierarchy, in: ICAPS, 2012.

[31] R. Geisberger, P. Sanders, D. Schultes, D. Delling, Contraction hierarchies: Faster and simpler hierarchical routing in road networks, in: WEA, Springer, 2008, pp. 319–333.

[32] M. Zhang, L. Li, W. Hua, X. Zhou, Stream processing of shortest path query in dynamic road networks, TKDE (2020).

[33] M. Zhang, L. Li, W. Hua, X. Zhou, Efficient 2-hop labeling maintenance in dynamic small-world networks, in: ICDE, IEEE, 2021.

[34] M. Zhang, L. Li, W. Hua, R. Mao, P. Chao, X. Zhou, Dynamic hub labeling for road networks, in: ICDE, IEEE, 2021.

[35] E. Cohen, E. Halperin, H. Kaplan, U. Zwick, Reachability and distance queries via 2-hop labels, SIAM Journal on Computing 32 (5) (2003) 1338–1355.

[36] T. Akiba, Y. Iwata, Y. Yoshida, Fast exact shortest-path distance queries on large networks by pruned landmark labeling, in: SIGMOD, ACM, 2013, pp. 349–360.

[37] D. Ouyang, L. Qin, L. Chang, X. Lin, Y. Zhang, Q. Zhu, When hierarchy meets 2-hop-labeling: Efficient shortest distance queries on road networks, in: SIGMOD, 2018, pp. 709–724.

[38] S. Wang, X. Xiao, Y. Yang, W. Lin, Effective indexing for approximate constrained shortest path queries on large road networks, PVLDB 10 (2) (2016) 61–72.

[39] Z. Liu, L. Li, M. Zhang, W. Hua, P. Chao, X. Zhou, Efficient constrained shortest path query answering with forest hop labeling, in: ICDE, IEEE, 2021.

[40] Z. Liu, L. Li, M. Zhang, W. Hua, X. Zhou, Fhl-cube: multi-constraint shortest path querying with flexible combination of constraints, Proceedings of the VLDB Endowment 15 (11) (2022) 3112–3125.

[41] Z. Liu, L. Li, M. Zhang, W. Hua, X. Zhou, Multi-constraint shortest path using forest hop labeling, The VLDB Journal (2022) 1–27.

[42] R. Bellman, Dynamic programming princeton university press princeton, New Jersey Google Scholar (1957).

[43] E. Denardo, Dynamic programming: models and applications, 2003.

[44] M. Sniedovich, Dynamic programming: foundations and principles, CRC press, 2010.

[45] M. Sniedovich, Dijkstra's algorithm revisited: the dynamic programming connexion, Control and cybernetics 35 (3) (2006) 599–620.

[46] S. E. Dreyfus, An appraisal of some shortest-path algorithms, Operations research 17 (3) (1969) 395–412.

[47] T. A. J. Nicholson, Finding the shortest route between two points in a network, The computer journal 9 (3) (1966) 275–280.

[48] J. Maue, P. Sanders, D. Matijevic, Goal-directed shortest-path queries using precomputed cluster distances, Journal of Experimental Algorithmics (JEA) 14 (2010) 3–2.

[49] P. E. Hart, N. J. Nilsson, B. Raphael, A formal basis for the heuristic determination of minimum cost paths, IEEE transactions on Systems Science and Cybernetics 4 (2) (1968) 100–107.

[50] A. V. Goldberg, C. Harrelson, Computing the shortest path: A search meets graph theory, in: SODA, Society for Industrial and Applied Mathematics, 2005, pp. 156–165.

[51] R. J. Gutman, Reach-based routing: A new approach to shortest path algorithms optimized for road networks., ALENEX/ANALC 4 (2004) 100–111.

[52] A. Madkour, W. G. Aref, F. U. Rehman, M. A. Rahman, S. Basalamah, A survey of shortest-path algorithms, arXiv preprint arXiv:1705.02044 (2017).

[53] M. Potamias, F. Bonchi, C. Castillo, A. Gionis, Fast shortest path distance estimation in large networks, in: Proceedings of the 18th ACM conference on Information and knowledge management, 2009, pp. 867–876.

[54] J. Kleinberg, A. Slivkins, T. Wexler, Triangulation and embedding using small sets of beacons, in: 45th Annual IEEE Symposium on Foundations of Computer Science, IEEE, 2004, pp. 444–453.

[55] G. V. Batz, R. Geisberger, S. Neubauer, P. Sanders, Time-dependent contraction hierarchies and approximation, in: International Symposium on Experimental Algorithms, Springer, 2010, pp. 166–177.

[56] T. Kieritz, D. Luxen, P. Sanders, C. Vetter, Distributed time-dependent contraction hierarchies, in: International Symposium on Experimental Algorithms, Springer, 2010, pp. 83–93.

[57] R. Geisberger, P. Sanders, D. Schultes, C. Vetter, Exact routing in large road networks using contraction hierarchies, Transportation Science 46 (3) (2012) 388–404.

[58] T. Akiba, Y. Iwata, K.-i. Kawarabayashi, Y. Kawata, Fast shortest-path distance queries on road networks by pruned highway labeling, in: 2014 Proceedings of the Sixteenth Workshop on Algorithm Engineering and Experiments (ALENEX), SIAM, 2014, pp. 147–154.

[59] Y. Yano, T. Akiba, Y. Iwata, Y. Yoshida, Fast and scalable reachability queries on graphs by pruned labeling with landmarks and paths, in: Proceedings of the 22nd ACM international conference on Information & Knowledge Management, 2013, pp. 1601–1606.

[60] M. Jiang, A. W.-C. Fu, R. C.-W. Wong, Y. Xu, Hop doubling label indexing for point-to-point distance querying on scale-free networks, PVLDB 7 (12) (2014) 1203–1214.

[61] H. L. Bodlaender, A tourist guide through treewidth, Acta cybernetica 11 (1-2) (1994) 1.

[62] L. Chang, J. X. Yu, L. Qin, H. Cheng, M. Qiao, The exact distance to destination in undirected world, VLDB Journal 21 (6) (2012) 869–888.

[63] W. Li, M. Qiao, L. Qin, Y. Zhang, L. Chang, X. Lin, Scaling distance labeling on small-world networks, in: SIGMOD, 2019, pp. 1060–1077.

[64] F. Wei, Tedi: efficient shortest path query answering on graphs, in: Graph Data Management: Techniques and Applications, IGI Global, 2012, pp. 214–238.

[65] W. Li, M. Qiao, L. Qin, Y. Zhang, L. Chang, X. Lin, Scaling up distance labeling on graphs with core-periphery properties, in: SIGMOD, 2020, pp. 1367–1381.

[66] H. C. Joksch, The shortest route problem with constraints, Journal of Mathematical analysis and applications 14 (2) (1966) 191–197.

[67] L. Lozano, A. L. Medaglia, On an exact method for the constrained shortest path problem, Computers & Operations Research 40 (1) (2013) 378–384.

[68] J. Gao, H. Qiu, X. Jiang, T. Wang, D. Yang, Fast top-k simple shortest paths discovery in graphs, in: CIKM, 2010, pp. 509–518.

[69] J. Y. Yen, Finding the k shortest loopless paths in a network, management Science 17 (11) (1971) 712–716.

[70] J. Hershberger, S. Suri, Vickrey prices and shortest paths: What is an edge worth?, in: Proceedings 42nd IEEE symposium on foundations of computer science, IEEE, 2001, pp. 252–259.

[71] A. Sedeño-Noda, Ranking one million simple paths in road networks, Asia-Pacific Journal of Operational Research 33 (05) (2016) 1650042.

[72] J. C. N. Climaco, E. Q. V. Martins, A bicriterion shortest path algorithm, European Journal of Operational Research 11 (4) (1982) 399–404.

[73] W. M. Carlyle, J. O. Royset, R. Kevin Wood, Lagrangian relaxation and enumeration for solving constrained shortest-path problems, Networks: An International Journal 52 (4) (2008) 256–270.

[74] L. Wang, L. Yang, Z. Gao, The constrained shortest path problem with stochastic correlated link travel times, European Journal of Operational Research 255 (1) (2016) 43–57.

[75] J. E. Beasley, N. Christofides, An algorithm for the resource constrained shortest path problem, Networks 19 (4) (1989) 379–394.

[76] X. Zhang, Y. Zhang, Y. Hu, Y. Deng, S. Mahadevan, An adaptive amoeba algorithm for constrained shortest paths, Expert Systems with Applications 40 (18) (2013) 7607–7616.

[77] R. Muhandiramge, N. Boland, Simultaneous solution of lagrangean dual problems interleaved with preprocessing for the weight constrained shortest path problem, Networks: An International Journal 53 (4) (2009) 358–381.

[78] A. C.-C. Yao, A lower bound to finding convex hulls, Journal of the ACM (JACM) 28 (4) (1981) 780–787.

[79] D. Delling, A. V. Goldberg, I. Razenshteyn, R. F. Werneck, Graph partitioning with natural cuts, in: IPDPS, 2011, pp. 1135–1146.

[80] N. Jing, Y.-W. Huang, E. A. Rundensteiner, Hierarchical encoded path views for path query processing: An optimal model and its performance evaluation, IEEE Transactions on Knowledge and Data Engineering 10 (3) (1998) 409–432.

[81] S. Jung, S. Pramanik, An efficient path computation model for hierarchically structured topographical road maps, IEEE Transactions on Knowledge and Data Engineering 14 (5) (2002) 1029–1046.

[82] K. Braekers, A. Caris, G. K. Janssens, Bi-objective optimization of drayage operations in the service area of intermodal terminals, Transportation Research Part E: Logistics and Transportation Review 65 (2014) 50–69.

[83] X. Gandibleux, F. Beugnies, S. Randriamasy, Martins' algorithm revisited for multi-objective shortest path problems with a maxmin cost function, 4OR 4 (1) (2006) 47–59.

[84] F. Guerriero, R. Musmanno, Label correcting methods to solve multicriteria shortest path problems, Journal of optimization theory and applications 111 (3) (2001) 589–613.

[85] A. J. Skriver, K. A. Andersen, A label correcting approach for solving bicriterion shortest-path problems, Computers & Operations Research 27 (6) (2000) 507–524.

[86] H.-P. Kriegel, P. Kröger, P. Kunath, M. Renz, T. Schmidt, Proximity queries in large traffic networks, in: ACM GIS, 2007, pp. 1–8.

[87] D. Kossmann, F. Ramsak, S. Rost, Shooting stars in the sky: An online algorithm for skyline queries, in: VLDB'02: Proceedings of the 28th International Conference on Very Large Databases, Elsevier, 2002, pp. 275–286.

[88] D. Papadias, Y. Tao, G. Fu, B. Seeger, An optimal and progressive algorithm for skyline queries, in: SIGMOD, 2003, pp. 467–478.

[89] N. Roussopoulos, S. Kelley, F. Vincent, Nearest neighbor queries, in: Proceedings of the 1995 ACM SIGMOD international conference on Management of data, 1995, pp. 71–79.

[90] G. R. Hjaltason, H. Samet, Distance browsing in spatial databases, ACM Transactions on Database Systems (TODS) 24 (2) (1999) 265–318.

[91] C. Böhm, H.-P. Kriegel, Determining the convex hull in large multidimensional databases, in: International Conference on Data Warehousing and Knowledge Discovery, Springer, 2001, pp. 294–306.

[92] J. Pei, W. Jin, M. Ester, Y. Tao, Catching the best views of skyline: A semantic approach based on decisive subspaces, Space (X, Y) 100 (4) (2005) 1.

[93] J. Pei, A. W.-C. Fu, X. Lin, H. Wang, Computing compressed multidimensional skyline cubes efficiently, in: 2007 IEEE 23rd International Conference on Data Engineering, IEEE, 2007, pp. 96–105.

[94] C. Raïssi, J. Pei, T. Kister, Computing closed skycubes, Proceedings of the VLDB Endowment 3 (1-2) (2010) 838–847.

[95] S. Wang, W. Lin, Y. Yang, X. Xiao, S. Zhou, Efficient route planning on public transportation networks: A labelling approach, in: SIGMOD, 2015, pp. 967–982.

[96] H. Wu, J. Cheng, S. Huang, Y. Ke, Y. Lu, Y. Xu, Path problems in temporal graphs, PVLDB 7 (9) (2014) 721–732.

[97] N. Robertson, P. D. Seymour, Graph minors. ii. algorithmic aspects of tree-width, Journal of algorithms 7 (3) (1986) 309–322.

[98] J. Chomicki, P. Godfrey, J. Gryz, D. Liang, Skyline with presorting, in: ICDE, Vol. 3, 2003, pp. 717–719.

[99] P. Godfrey, R. Shipley, J. Gryz, et al., Maximal vector computation in large data sets, in: Proceedings of the VLDB Endowment, Vol. 5, Citeseer, 2005, pp. 229–240.

[100] Y. Lu, C. Shahabi, An arc orienteering algorithm to find the most scenic path on a large-scale road network, in: Proceedings of the 23rd SIGSPATIAL International Conference on Advances in Geographic Information Systems, 2015, pp. 1–10.

[101] X. Zhang, Z. Liu, M. Zhang, L. Li, An experimental study on exact multi-constraint shortest path finding, in: ADC 2021, Dunedin, New Zealand, Springer, 2021, pp. 166–179. doi:10.1007/978-3-030-69377-0_14.
URL https://doi.org/10.1007/978-3-030-69377-0_14

[102] D. Delling, D. Wagner, Pareto paths with sharc, in: International Symposium on Experimental Algorithms, Springer, 2009, pp. 125–136.

[103] U. Meyer, P. Sanders, δ-stepping: a parallelizable shortest path algorithm, Journal of Algorithms 49 (1) (2003) 114–152.

[104] S. Jung, S. Pramanik, Hiti graph model of topographical road maps in navigation systems, in: Proceedings of the Twelfth International Conference on Data Engineering, IEEE, 1996, pp. 76–84.

[105] G. Karypis, et al., Family of multilevel partitioning algorithms (1997).

[106] T. Chondrogiannis, J. Gamper, Pardisp: A partition-based framework for distance and shortest path queries on road networks, in: 2016 17th IEEE International Conference on Mobile Data Management (MDM), Vol. 1, IEEE, 2016, pp. 242–251.

[107] R. Zhong, G. Li, K.-L. Tan, L. Zhou, Z. Gong, G-tree: An efficient and scalable index for spatial search on road networks, IEEE Transactions on Knowledge and Data Engineering 27 (8) (2015) 2175–2189.

[108] F. Pellegrini, Scotch: Static mapping, graph, mesh and hypergraph partitioning, and parallel and sequential sparse matrix ordering package, 2007, Cited on (2005) 3.

[109] D. Delling, A. V. Goldberg, T. Pajor, R. F. Werneck, Customizable route planning in road networks, Transportation Science 51 (2) (2017) 566–591.

[110] J. Hahn, New york city to google: Reduce the number of left turns in maps navigation directions, https://www.foxnews.com/auto/new-york-city-to-google-reduce-the-number-of-left-turns-in-maps-naviga

[111] 9th dimacs implementation challenge - shortest paths, http://users.diag.uniroma1.it/challenge9/download.shtml.

[112] M. Zhang, L. Li, X. Zhou, An experimental evaluation and guideline for path finding in weighted dynamic network, Proceedings of the VLDB Endowment 14 (11) (2021) 2127–2140.

[113] J. Li, C. Ni, D. He, L. Li, X. Xia, X. Zhou, Efficient knn query for moving objects on time-dependent road networks, The VLDB Journal (2022) 1–20.

[114] Z. Luo, L. Li, M. Zhang, W. Hua, Y. Xu, X. Zhou, Diversified top-k route planning in road network, Proceedings of the VLDB Endowment 15 (11) (2022) 3199–3212.

[115] H. Liu, C. Jin, B. Yang, A. Zhou, Finding top-k shortest paths with diversity, TKDE 30 (3) (2017) 488–502.

[116] T. Chondrogiannis, P. Bouros, J. Gamper, U. Leser, D. B. Blumenthal, Finding k-shortest paths with limited overlap, The VLDB Journal (2020) 1–25.

[117] J. Li, X. Xiong, L. Li, D. He, C. Zong, X. Zhou, Finding top-k optimal routes with collective spatial keywords on road networks, in: 2023 IEEE 39th International Conference on Data Engineering (ICDE), IEEE, 2023.